

Received 11 December 2025, accepted 24 December 2025, date of publication 30 December 2025, date of current version 20 January 2026.

Digital Object Identifier 10.1109/ACCESS.2025.3649768

 SURVEY

Telemetry and Agentic AI: Foundations for Optical Network Automation

SERGIO CRUZES^{id}, (Member, IEEE)

Optical Network Engineering, Ciena Office Brazil, Ciena Corporation, São Paulo, Brazil

e-mail: scruzes@ciena.com

ABSTRACT Optical networks are moving toward real-time, autonomous operation, driven by the need to detect impairments earlier, react faster, and coordinate control across multi-vendor environments. This paper examines how telemetry, machine learning, digital twins, and emerging agentic AI components jointly support this transition. It consolidates recent advances in streaming telemetry, edge processing, anomaly detection, and hybrid LLM–ML control, and clarifies how these technologies interact within closed automation loops. The study develops a unified architectural view in which telemetry acts as the sensory layer, AI and ML models provide inference and prediction, digital twins maintain alignment with the physical network, and agentic AI-based structures coordinate decisions. The paper also identifies common trends in the current literature, including the adoption of standardized data models, distributed agents, intent-based operation, and real-time AI pipelines. Together, these insights outline the foundations required for scalable and autonomous optical network automation.

INDEX TERMS Optical networks, telemetry, automation, agentic AI, digital twin, large language models.

I. INTRODUCTION

Optical networks now support traffic volumes that grow year after year. Their operation involves heterogeneous equipment, strict physical constraints, and rapid changes in demand. These conditions make traditional management methods difficult to maintain, so automation has become a central requirement for preserving performance and scalability [1].

Automation enables operators to handle routing, spectral assignment, resource allocation, and failure recovery in a more systematic way, while reducing the need for manual intervention and supporting real-time decisions in complex multi-vendor environments [1]. Recent advances further extend this capability: machine-learning methods introduce data-driven control, digital twins (DTs) offer continuously updated virtual replicas for simulation and planning, and large language models add semantic reasoning and intent interpretation to operational workflows [2].

Telemetry forms the bridge between the physical environment and automated control functions. It provides continuous

measurements of signal quality, device state, and traffic conditions that supply the information needed by AI-driven controllers and planning tools [3]. In modern architectures, high-rate telemetry streaming, often based on Google remote procedure call (gRPC), gRPC network management interface (gNMI), in-memory processing, and distributed preprocessing, enables rapid collection and analysis of operational data, which is essential for real-time automation loops.

Without accurate and timely telemetry, advanced mechanisms cannot act with confidence. This is especially important as interest grows in autonomous decision structures that can interpret events, plan actions, and carry out operational tasks with limited human involvement. These structures, often described as agentic AI, introduce software agents that observe conditions, evaluate possible responses, and coordinate actions throughout the network [2]. Such agents can combine the semantic reasoning of LLMs with domain models and policy rules, enabling them to carry out configuration tasks, negotiate resources, and adapt strategies as network conditions evolve. This work brings together telemetry, structured data models, DT systems, and agentic-based control in a single framework, showing how

The associate editor coordinating the review of this manuscript and approving it for publication was Muguang Wang^{id}.

these elements operate jointly to support higher levels of autonomy in optical networks.

This paper combines a survey of existing work with an architectural synthesis. The individual components discussed, telemetry pipelines, edge processing, streaming platforms, DT systems, machine-learning models, large language models, and agentic-based control mechanisms, are all derived from prior literature and operational practice. No new protocol, algorithm, or control mechanism is proposed. The contribution of this work lies in the synthesized framework of the authors, which integrates components derived from literature into a unified architectural view that clarifies their roles, interactions, and dependencies within closed-loop optical network automation. Throughout the paper, the technologies surveyed provide the building blocks, while the proposed integration reflects how these blocks can be combined to support scalable, intent-driven, and reliable autonomous operation.

The transition toward more autonomous optical networks also introduces several challenges. Reliable automation depends on high-quality data [4], scalable infrastructure, sufficient processing capacity to support real-time analytics [5], and decision mechanisms that can be trusted in operational environments [6]. These aspects require careful design, but they also mark a clear research agenda for the coming years.

This paper examines how telemetry and agentic AI form the basis of next-generation automation frameworks for optical networks. Telemetry provides the foundation for situational awareness. Agents use this information to diagnose conditions, adjust configuration, and coordinate multi-step tasks. DTs validate actions before they affect live traffic, and large language models help interpret operator intent and explain system behavior. By combining these elements, it becomes possible to build automation systems that are more efficient, predictable, and easier to operate.

The main contributions of this work are threefold. First, it outlines how a unified framework could link telemetry, structured data models, DTs and agentic-based control, illustrating how these elements together may enable higher levels of autonomous operation in optical networks. Second, it details how telemetry can be arranged as a complete data layer with streaming, fusion, and quality functions that provide reliable inputs to AI based control loops. Third, it explains how an agentic AI architecture may be organized for optical networks, with coordinated roles for monitoring, analysis, resource planning, and action checking through DT environments.

The remainder of this paper is organized as follows. Section II reviews previous work on telemetry and agentic AI and explains how these areas support automation in optical networks. Section III reviews telemetry as the basis of the data layer and details streaming architectures, data fusion and data quality. Section IV presents DT systems used for validation, planning, and synchronization with the physical network. Section V summarizes the role of large

language models in interpretation, reasoning, and operator interaction. Section VI describes the agentic AI framework and the coordination of agents that operate on telemetry and intent. Section VII covers key operational tasks, including quality of transmission (QoT) estimation, failure handling, resource optimization, and soft reconfiguration. Section VIII discusses the main challenges that affect scalable automation. Section IX outlines future research directions. Section X describes common trends observed in telemetry, agentic AI and automation, and Section XI concludes the paper.

II. RELATED WORK ON TELEMETRY AND AGENTIC AI

Recent research on optical telemetry reveals several consistent themes in the literature. Reference [13] provides a summary of research work related to telemetry architectures in optical communications. It shows that monitoring architectures increasingly combine optical indicators with packet-layer telemetry to improve visibility across layers. Early work explores Kafka-based reporting pipelines, yet another next generation (YANG) extensions for on-demand data retrieval, and analytic platforms capable of processing Internet protocol flow information export (IPFIX) protocol and spectrum data. Several studies demonstrate the advantages of gRPC-based streaming from disaggregated devices, reporting lower overhead and faster update cycles than the network configuration protocol (NETCONF). High-rate telemetry agents integrated into transponders, reconfigurable optical add-drop multiplexers (ROADMs), and monitoring blades further extend visibility by exposing fine-grained physical-layer metrics. Telemetry has also been applied to mode selection, soft-failure diagnosis, and control-plane support during unstable conditions. However, while scalability and interoperability with open optical devices have been widely studied, reliability under stress remains an open challenge, with few works addressing dependable delivery when networks are congested or partially impaired.

Reference [14] reviews work that spans ML models, optical performance monitoring (OPM), data sources, and telemetry mechanisms. Supervised, unsupervised, and reinforcement learning techniques have been widely used for QoT prediction, nonlinear impairment mitigation, failure handling, and routing, modulation, and spectrum assignment (RMSA). Accuracy improves substantially when detailed physical-layer data are included. OPM continues to evolve from classical analog and digital techniques toward digital signal processing (DSP)-driven and ML-aided methods, improving the precision of impairment and quality estimation. Operators routinely collect large volumes of operational and device-level metrics, yet real datasets are rarely shared because of confidentiality and security concerns, and public datasets often lack topology and device configuration details. Telemetry-related studies examine gRPC streaming, YANG retrieval, modular device agents, and Kafka-based architectures capable of handling high-rate updates. However, most solutions target isolated aspects of telemetry performance,

such as transport speed, feature extraction, or vendor integration. Few treat telemetry as a unified substrate for ML-driven control or DT systems, leaving gaps in end-to-end data organization, cross-domain visibility, and stable delivery during high-load conditions.

Reference [3] highlights the central role of telemetry in automation, failure detection, and digital-twin development. Optical spectra and in-phase and quadrature (IQ) constellations have repeatedly been shown to be essential for detecting anomalies, filter shifts, and bit error rate (BER) trends. Telemetry in optical networks exhibits all five Vs of Big Data: high volumes, fast rates (velocity), diverse formats (variety), data quality (veracity), and the need to extract value. Existing architectures range from centralized data lakes to hierarchical and event-driven models. gRPC provides efficient encoding for structured telemetry but limits flexibility due to schema rigidity, while Kafka-based streaming (a distributed event-streaming platform) better accommodates heterogeneous data sources. Scalability is a persistent issue, particularly in disaggregated environments where many devices generate high-frequency measurements. Although powerful streaming and aggregation techniques have been demonstrated, fewer works address architectures that remain flexible, scalable, and capable of delivering consistent multi-source data for ML, DTs, and real-time control.

In addition to telemetry research, reference [15] traces the evolution from classical agent models based on symbolic logic and predefined rules to modern learning-driven agents. Traditional agents could perform structured tasks, but lacked adaptability, learning, and contextual reasoning. With the rise of large-scale generative models, agents gained tool-use capabilities, memory, and multi-step reasoning. Reference [16] extends this perspective, reviewing frameworks that support planning, coordination, and multi-agent collaboration. Applications now span robotics, healthcare, logistics, and industrial automation. In the literature, recurring challenges include reliable long-term reasoning, coordination among multiple agents, evaluation strategies for complex decision loops, and safe integration with real-time telemetry and control systems.

Together, these works show a growing convergence between telemetry innovations and agentic intelligence. High-rate, multi-source telemetry is becoming essential for ML, DTs, and AI-driven control, while agentic systems increasingly depend on structured, reliable, and real-time data streams to deliver autonomous behavior.

To help readers navigate the dense body of prior work, Table 1 groups representative references already cited in this section by technical theme.

III. TELEMETRY AS THE CORE DATA LAYER

A. ROLE OF TELEMETRY IN NETWORK OPERATION

Real-time automation depends on a continuous, low-latency view of both the physical and logical state of the network. Telemetry provides this view by streaming time-series measurements to centralized and distributed control and AI

platforms [3]. These streams support tasks such as power balancing, alarm classification, anomaly detection, topology discovery, and early failure mitigation [3].

To be useful in closed-loop control, telemetry must carry accurate measurements and operationally meaningful indicators. Metrics such as signal-to-noise ratio (SNR) offer a practical proxy for transmission quality and can reveal early degradations in coherent long-haul systems [17]. Richer observables from optical spectrum analyzers and coherent DSP outputs, including IQ samples and derived constellation features, expose filter misalignment, chromatic dispersion, nonlinear impairments, and the onset of soft failures. Although these data types enable advanced applications such as anomaly classification, root cause analysis, and link distance estimation, they also raise challenges in data volume, parsing overhead, and dimensionality reduction [3]. In practice, telemetry must describe system health with enough fidelity that risks can be anticipated, faults can be localized, and configuration changes can be planned with confidence.

B. STREAMING ARCHITECTURES

Telemetry pipelines typically begin at the device, where measurements are acquired and often preprocessed locally before being pushed into a distributed monitoring and control framework [3], [14]. Modern deployments commonly follow a publish-subscribe model. Optical elements or node-level agents publish measurements, while collectors, software-defined networking (SDN) controllers, analytics services, and automation agents subscribe to relevant topics.

In field deployments, lightweight agents such as Telegraf (an open-source, plugin-based telemetry agent for collecting, processing, and forwarding metrics) can run on servers near ROADMs, transponders, and amplifiers to normalize heterogeneous formats and forward streams to time-series databases such as InfluxDB [14]. This allows each node to store and process parameters such as BER, OSNR, power levels, laser frequencies, and chromatic dispersion locally, supporting edge analytics such as QoT prediction and anomaly detection.

For wide-area transport and fan-out to multiple consumers, event streaming systems such as Apache Kafka decouple producers from consumers [13], [14]. Kafka organizes measurements into topics and partitions, enabling parallel ingestion, replication for fault tolerance, and replay of recent history for diagnostics. This design supports high-throughput, low-latency streaming and scales to multi-consumer automation pipelines in optical networks [13]. As a result, streaming architectures reduce reaction time, distribute processing, and prevent overload at single collection points.

C. TELEMETRY PROTOCOLS

Telemetry in modern optical networks relies on a layered stack that combines data modeling, control interfaces, transport mechanisms, and storage engines [18]. This modular

TABLE 1. Representative literature grouped by technical theme.

Theme	Representative references (already cited)
Telemetry architectures and streaming	Kafka- and gRPC-based telemetry pipelines, hierarchical and event-driven designs, scalability analysis [3], [13], [14].
Machine learning for optical networks	QoT estimation, failure detection, impairment monitoring, ML-assisted OPM [1], [14].
Digital twins and validation	DT-supported planning, validation of control actions, synchronization with physical networks [2], [3].
Agentic AI and multi-agent systems	Agent architectures, coordination, planning, and autonomy in complex systems [15], [16].
Telemetry-driven automation	Closed-loop control, real-time analytics, integration of telemetry with AI and control planes [3], [14].

design supports heterogeneous equipment and enables timely delivery of measurements for operational decisions.

At the modeling layer, YANG provides a structured and vendor-agnostic method to define the configuration and state data [19]. These models are commonly accessed via NETCONF or the RESTful CONfiguration protocol (RESTCONF) [20], which allow controllers and management systems to retrieve and modify device state in a schema-driven manner.

For high-frequency monitoring, gRPC has become a preferred option for telemetry streaming. Operating over HTTP/2, it supports efficient remote procedure calls and bidirectional communication [21]. When combined with Protocol Buffers, gRPC encodes telemetry messages into compact binary structures suitable for rapid transmission from ROADMs, transponders, and amplifiers [22]. Deploying gRPC telemetry in operational systems typically requires binding YANG-defined objects to concrete device instances, including identity, location, role, and hardware characteristics, so that each stream can be aligned with topology and service context.

Representational state transfer (REST) application programming interfaces (APIs) remain important for north-bound communication and lower-rate configuration queries. Although not optimized for continuous streaming, REST interfaces provide readable access to configuration and status using standard HTTP methods [14]. Each request is processed independently, which simplifies integration but limits suitability for high-rate time-series delivery.

Beyond these mechanisms, additional messaging and transport protocols expand pipeline flexibility. Kafka enables topic-based streaming with strong decoupling between producers and consumers [13]. Lightweight protocols such as Message Queuing Telemetry Transport (MQTT) and WebSocket (a protocol for persistent, bidirectional communication over TCP, commonly used for real-time data streaming) provide low-overhead channels well suited to constrained edge environments [23], [24]. In-memory systems such as Redis support fast pub-sub messaging and short-term caching, enabling agents and controllers to exchange transient updates with minimal latency [25]. Together, these layers deliver structured high-rate measurements with controllable overhead, enabling the visibility required for automation and closed-loop control.

To show how these modeling languages, control interfaces, and transport mechanisms interact in practice, Figure 1 summarizes an end-to-end telemetry protocol stack for optical networks. The diagram organizes the ecosystem from device and on-box agents up to control applications and digital-twin platforms, and highlights how YANG-based abstractions, NETCONF and RESTCONF access, gRPC streaming, and message-oriented backbones cooperate to distribute telemetry at scale.

D. DATA QUALITY ISSUES

The effectiveness of telemetry-driven automation depends strongly on data quality. Operators face irregular sampling, missing measurements, vendor-specific conventions, and limitations of legacy equipment [3], [26]. High-dimensional observables such as optical spectra and IQ constellations intensify these challenges because they require compression and feature extraction to be manageable at scale [3], [27].

Distributed telemetry frameworks address these problems through local preprocessing and intelligent aggregation. Supervised feature extraction, autoencoder-based compression, and statistical summarization reduce high-volume streams while preserving information needed for analysis [3]. Reported experiments show that these methods can shrink telemetry volume by more than 99% while still supporting sub-second processing, which is aligned with real-time automation requirements [3].

These optimizations introduce trade-offs. Over-aggregation can hide subtle anomalies, and edge processing requires compute capacity that may not exist on older or resource-limited devices [28]. Maintaining and updating ML models on many distributed nodes adds operational complexity, particularly when heterogeneous edge locations run different pipeline components [28]. Similar issues appear in multi-user telemetry platforms, where dashboards must avoid cross-tenant leakage, and the platform must include controls that prevent accidental overwrites or data loss [29]. For these reasons, telemetry pipelines must include validation, normalization, and harmonization steps so that analytics and automation consume reliable, consistent signals. Implementations based on Elasticsearch (a distributed search and analytics engine for storing and querying large volumes of time-series and event data) and Kibana (a visualization and dashboarding tool for exploring and monitoring data stored

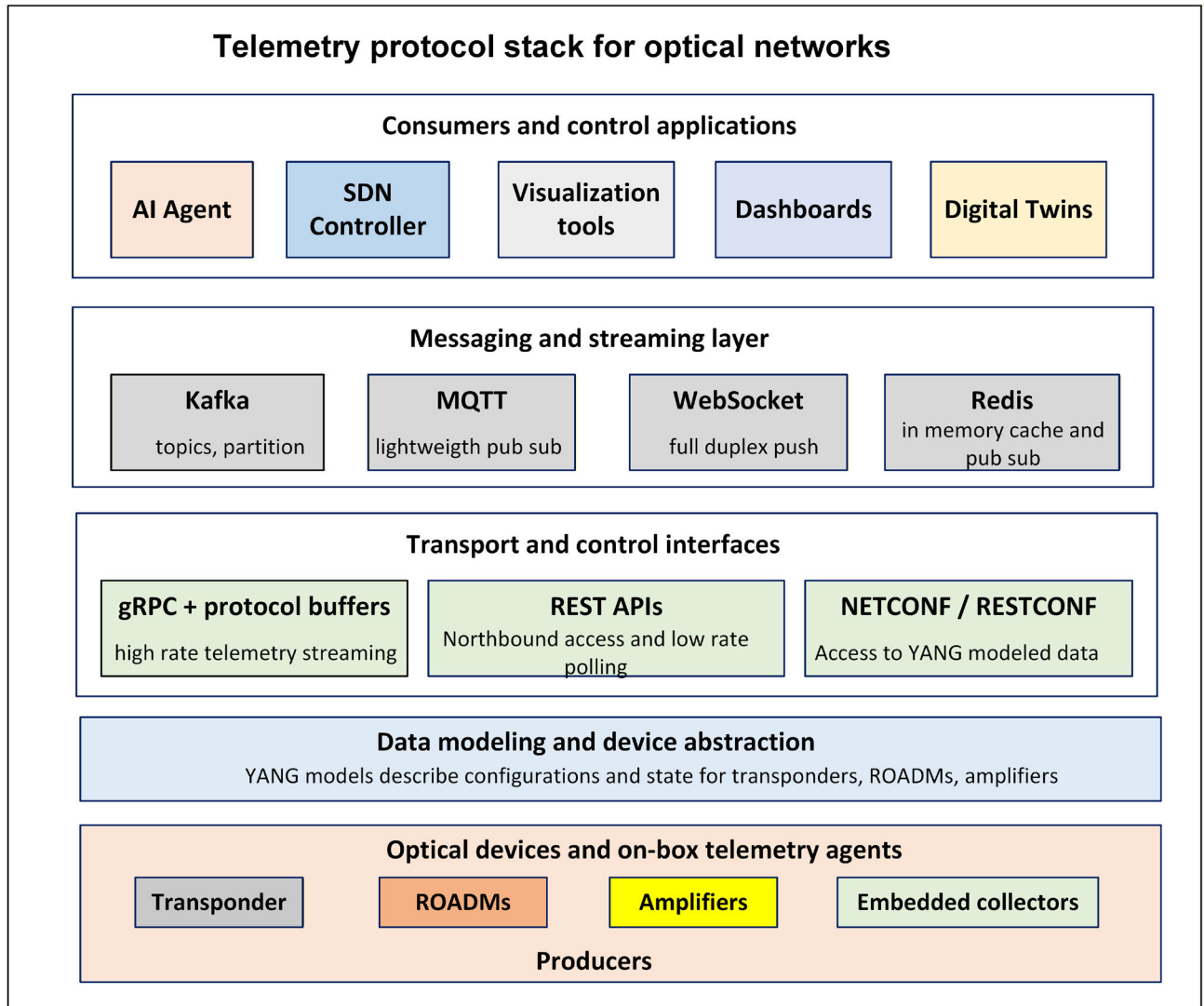


FIGURE 1. End-to-end telemetry protocol stack showing producers, modeling layers, transport and control interfaces, messaging mechanisms, and consumer applications that support real-time monitoring and automation in optical networks.

in Elasticsearch) make these challenges visible in practice, including index separation, access control, and governance for shared platforms [27], [29]. Multi-layer optical telemetry studies also emphasize that accurate state representation is necessary for automated decision logic and anomaly analysis [26]. Additional messaging frameworks such as MQTT are often adopted where bandwidth headroom is limited and lightweight transport is preferred [30].

E. DATA FUSION

Data fusion combines telemetry and management information into a unified operational view. In optical systems, fusion commonly spans physical-layer measurements, device status, alarms, SDN events, and topology information. Unified telemetry platforms show that fusion improves correlation across packet and optical domains and supports automated

decision workflows [14]. Streaming platforms such as Kafka organize inputs into separate topics, for example per device, per layer, or per service, enabling parallel ingestion and low-latency delivery [14]. Time-series databases such as InfluxDB [31] and event-oriented stores such as Elasticsearch maintain synchronized histories, supporting continuous monitoring and fast retrieval of past events. Visualization tools such as Grafana and Kibana present aggregated dashboards for operators and have been evaluated in multi-layer optical telemetry and high-rate metric streaming settings [26], [27], [29].

Standards are central to reliable fusion. Interfaces such as transport application programming interface (T-API) and Telecom Infra Project (TIP) telemetry specifications define common models for performance indicators, impairment monitoring, and equipment state, reducing integration effort

in multi-vendor environments. T-API uses YANG models together with RESTCONF or gRPC to expose topology, performance metrics, alarms, and provisioning data in a vendor-neutral format [32]. TIP TR-547 and TR-548 provide reference agreements that define schemas and architectural guidelines for real-time telemetry ingestion and processing [32], [33]. This standards-based approach supports semantic and temporal alignment across heterogeneous sources, improving the reliability of fused inputs consumed by analytics and automation modules.

Fusion also supports advanced analytics. Frameworks that combine T-API streams with time-series caches can reconstruct dense KPI windows from sparse update policies, enabling unsupervised anomaly detection with clustering methods such as DBSCAN [34]. In this way, fusion supports both real-time monitoring and offline analysis.

F. TELEMETRY AS THE TRIGGER FOR CONTROL ACTIONS

In closed-loop automation, telemetry is not only a reporting mechanism but also the primary trigger for control actions. SDN controllers, AI engines, and agentic-AI frameworks subscribe to streaming measurements and react when predefined conditions are observed. Changes in received power, drops in optical signal-to-noise ratio (OSNR), increases in BER, constellation distortions, and latency deviations can all serve as control signals. When such variations occur, processing pipelines can initiate fault localization, adjust amplifier gain or equalization profiles, reconfigure lightpaths, or refresh QoT estimates to maintain service guarantees. This aligns with work emphasizing pervasive telemetry as the foundation for event-driven optical automation and real-time control [3], [13].

Telemetry-driven workflows often rely on layered or hybrid architectures. Edge agents perform local filtering, compression, and feature extraction, while centralized managers coordinate global decisions [3]. Processed streams feed AI inference pipelines in which different models share the workload. Lightweight ML models such as decision trees and random forests can handle simple classification tasks, while LLM-based components support intent parsing, orchestration, and operator interaction [35], [36]. Techniques such as hardware acceleration (use of specialized processors to speed up computation) [37], model quantization (reduction of numerical precision of model parameters to decrease computation and memory cost) [38], speculative decoding (using smaller models to propose several token completions in parallel) [39], [40], and batching (processing multiple inference requests together to improve throughput and resource utilization) [41] help keep end-to-end latency within operational limits.

The telemetry manager closes the loop by adjusting sampling rates, thresholds, and feature sets based on conditions and the needs of downstream controllers [3]. In this sense, telemetry acts as the sensory layer of the automation framework, continuously feeding DT systems, AI models,

and software agents with structured and timely information, enabling intent-based automation and closed-loop control in optical networks [2]. This stream supports adaptive control, soft-failure detection, and resource optimization even in disaggregated multi-vendor environments [3], [34]. By ensuring observability directly drives action, telemetry enables actionable synchronization between DTs, ML pipelines, and LLM agents in operational settings [2], [34].

Before introducing the architecture figures in the following subsections, it is helpful to summarize the key layers found in current telemetry systems. Table 2 consolidates the main functions, placements, and enabling methods used to support real-time automation in optical networks.

G. DISTRIBUTED EDGE-CENTRIC TELEMETRY ARCHITECTURE

Distributed telemetry architectures place part of the monitoring and preprocessing logic close to optical devices, while retaining centralized coordination for control and analytics. As shown in Figure 2, transponders, amplifiers, and ROADMs stream measurements that are first handled by lightweight node-level agents [14]. These agents perform cleaning, normalization, and short-term buffering using local time-series storage, enabling fast checks and simple inference without immediate dependence on remote controllers. This edge-centric placement reduces reaction time and limits unnecessary upstream traffic, while preserving global correlation through centralized platforms.

Event-driven streaming backbones link edge processing with centralized consumers. Kafka-based pipelines, illustrated in Figure 3, decouple telemetry producers from consumers and support the scalable ingestion of high-rate optical metrics [13]. Node agents publish structured streams that are asynchronously consumed by SDN controllers, AI agents, DT functions, and visualization tools. Partitioning and replication enable parallel processing, resilience to failures, and replay of recent history for diagnostics, without imposing tight coupling between devices and applications.

Although edge processing reduces latency, raw physical-layer telemetry such as optical spectra and IQ samples can still overwhelm transport and storage if forwarded without reduction. Compression-based distributed telemetry, shown in Figure 4, addresses this by extracting features and summaries at the node level [3]. Supervised feature selection and autoencoder-based reduction shrink high-volume observables into compact representations that preserve the information needed for QoT estimation, anomaly detection, and early-warning tasks. The resulting summaries are cached locally and forwarded through low-latency interfaces before integration with centralized storage, alarm systems, and analytics.

These architectural elements converge when telemetry drives hybrid ML-DT-LLM automation pipelines. In such systems, end-to-end synchronization between telemetry sources, numerical inference, DT state, and higher-level reasoning becomes a primary requirement. Purely centralized

TABLE 2. Layers and functions in a hybrid telemetry system for optical networks.

Layer	Main role	Methods and tools (with references)
Edge agents	Collect device data and perform first-stage reductions. Support simple ML tasks near the source.	Data adapters, supervised feature extraction, autoencoder compression, delta encoding [3], [27], Redis pub-sub [25], local time-series buffers [14], lightweight anomaly checks [28].
Local processing	Convert high-volume inputs into compact summaries. Remove redundant or low-variance samples.	Statistical summaries [3], threshold-based reporting [3], windowed averaging [14], harmonization routines for legacy devices [26].
Transport layer	Move reduced measurements to control platforms with low delay and stable throughput.	gRPC with Protocol Buffers [21], [22], Kafka topics and partitions [13], MQTT [23], Redis exchange [25], WebSocket push updates [24].
Telemetry manager	Maintain state history, correlate streams, and supply data to SDN and AI modules.	InfluxDB time-series storage [14], Elasticsearch for logs and events [29], local caches [25], validation and normalization routines [26].
AI and control	Use fused telemetry to support QoT, alarms, power balance, and resource placement. Provide feedback to adjust sampling.	LLMs and DT-based analysis [2], decision trees and random forests [35], quantized models [36], speculative decoding [40], batching [41], GPU acceleration [37].
Standards and schemas	Preserve a consistent and vendor-neutral data view across systems.	YANG, NETCONF, RESTCONF [19], OpenConfig and OpenROADM [20], T-API, TIP TR-547 and TR-548 [32], [33].

or batch-oriented ML processing is often insufficient under the timing constraints imposed by real-time control loops. Recent work therefore combines ML inference with DT-based validation and LLM-assisted orchestration, seeking a balance between speed, safety, and interpretability [42].

To summarize both architectural and operational trade-offs, Table 3 compares ML-only automation pipelines with hybrid ML–DT–LLM approaches as reported in the literature. The comparison covers decision structure and adaptation, and highlights practical differences in latency, scalability, reliability, and operational risk for closed-loop optical network automation.

Distributed telemetry architectures mitigate timing limitations by combining edge preprocessing with low-latency ingestion frameworks such as Apache Kafka [43], [44], MQTT [23], and gRPC streaming [44]. Kafka supports high-throughput, fault-tolerant streaming [45], enabling measurement delivery intervals of the order of 100 ms. Bandwidth can be further reduced through delta encoding and incremental updates, which forward only changes relative to prior states [46]. In-memory data stores such as Redis provide rapid access to shared context, supporting timely DT updates and LLM queries [47].

Efficient coordination among LLMs, numerical ML models, and DT components is essential to keep control loops responsive. Compact serialization and RPC mechanisms reduce message sizes and transmission overhead [48], [49]. Modular designs allow LLM components to focus on orchestration and explanation while delegating physics-based validation and numerical inference to specialized modules [50]. Shared-memory buffers and message queues further reduce synchronization overhead for co-located processes [51]. For latency-sensitive tasks, edge ML models can detect anomalies locally and suppress transmission of full-resolution telemetry when not required [28]. Selected DT functions, including lightweight physics solvers, may also be executed at the edge using runtimes such as WebAssembly (a portable, low-level binary instruction format designed for fast, sandboxed execution of code across platforms) [52], [53].

Meeting strict timing targets also requires optimizing the LLM inference path. Quantization [36], parameter-efficient fine tuning, speculative decoding [40], context caching [54], and continuous batching [55] reduce inference latency and resource consumption while preserving accuracy. Hybrid pipelines that combine LLM reasoning with lightweight classifiers such as Random Forests can further reduce decision latency by 40–60 percent [56], and hardware acceleration using modern graphics processing units (GPUs) supports token generation rates compatible with near real-time operation [57]. Candidate actions generated by agents are typically validated within a DT environment before deployment, for example using tools such as GNPpy to assess feasibility without impacting live traffic [58]. When combined, distributed preprocessing, efficient streaming, in-memory state management, and optimized inference have been reported to reduce end-to-end latency by factors of the order of 3–5 compared to fully centralized designs on experimental and prototype platforms [3], [13], [59]. These values should be interpreted as indicative ranges observed in specific implementations rather than universal bounds.

To clarify the design trade-offs discussed in this section, Table 4 compares centralized and distributed telemetry architectures reported in the literature. The comparison highlights differences in data volume, latency, scalability, and fault tolerance, based on experimental and empirical results from recent optical networking studies.

IV. DIGITAL TWINS

A. PURPOSE OF DIGITAL TWINS

A DT system is a dynamic virtual model that mirrors a physical system and is kept aligned with reality through continuous data updates [60]. In optical communications, this idea has evolved into a DT framework that represents the behavior of links, devices, and services by combining network design information with real-time measurements [61]. Typical DT frameworks group their functions into three main blocks: real-time monitoring and software sensing, mirror modeling, and automatic control [61].

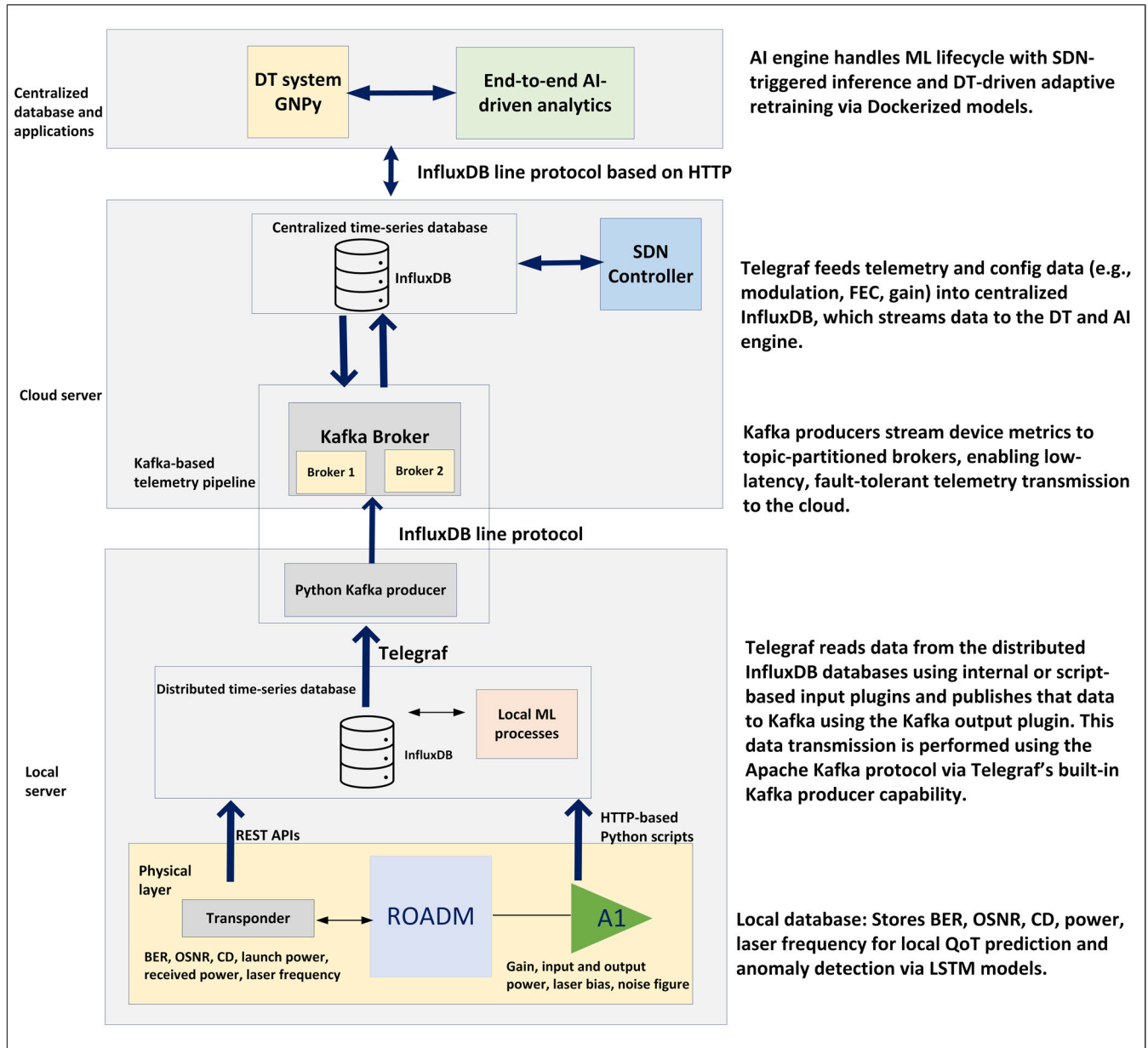


FIGURE 2. Unified system for real-time monitoring and telemetry [14].

Real-time monitoring gathers traffic patterns, device health, fiber impairments, and environmental conditions, while mirror modeling uses these data to maintain an up-to-date virtual copy of the network [62], [63]. Automatic control closes the loop by using the twin to evaluate actions such as reconfiguration, margin tightening, and failure recovery before applying it to live traffic [61]. In this way, the DT provides a controlled space in which operators and AI agents can explore what-if scenarios, test operating points, and plan changes without disturbing the physical network [64].

Figure 5 shows the main components of a DT twin for optical networks and how the information moves between them. At the bottom, the physical layer represents the

operational optical network, where devices generate real-time measurements on state, performance, alarms, and status. These measurements flow upward into the data acquisition module and database, which collect and organize all incoming telemetry.

The mirror modeling block uses these data to maintain a virtual representation of the network. It combines numerical methods, analytical models, machine learning, and physics-informed techniques to estimate behavior and predict how the network will respond to different conditions.

LLMs interact with this modeling block by receiving model outputs and providing predictions or strategies. They support tasks such as failure handling, device configuration, performance estimation, and parameter selection.

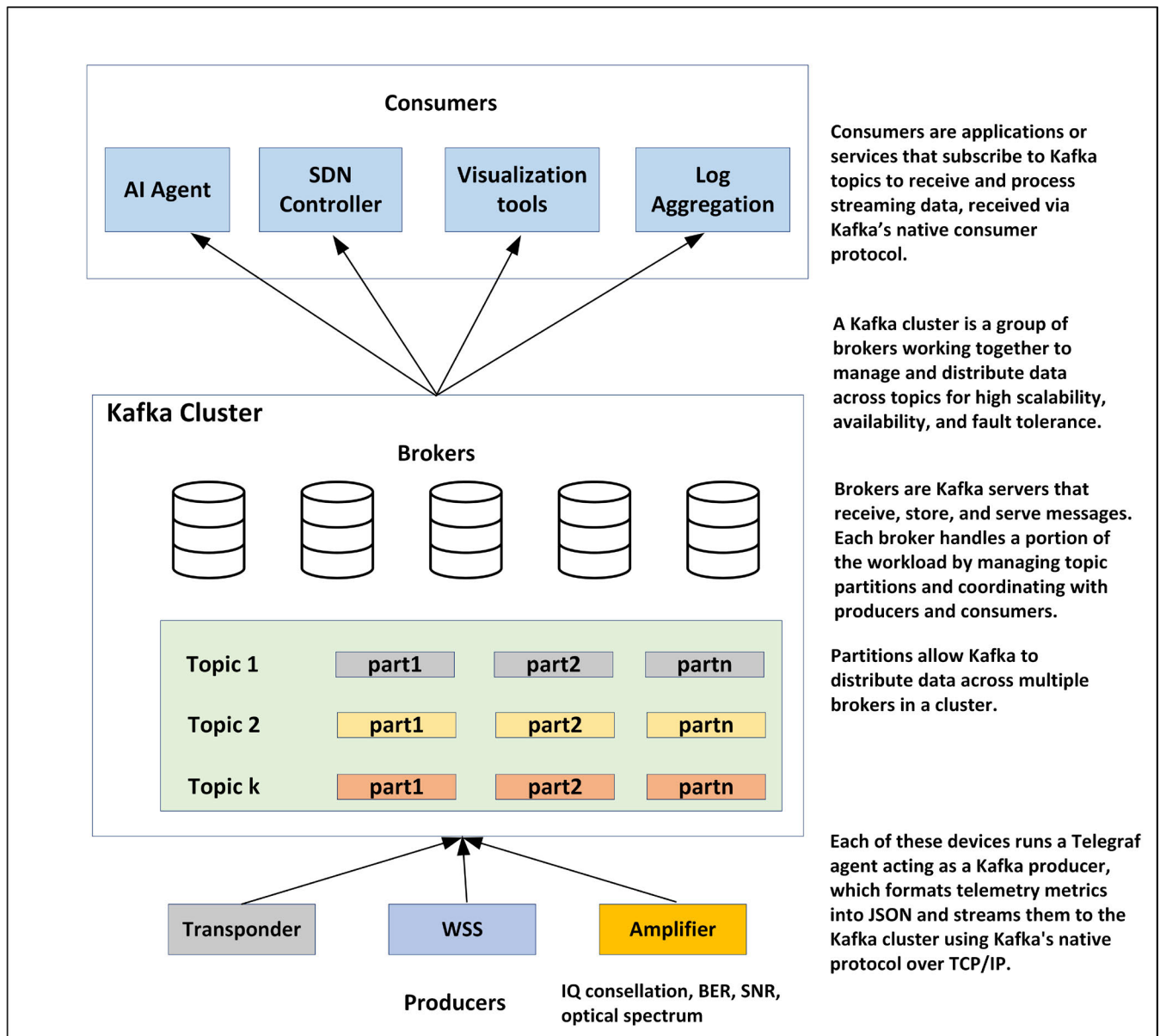


FIGURE 3. Kafka-based telemetry system [13].

The SDN block receives these strategies and translates them into specific control actions. It forms the link between the modeling results and the operational network, handling configuration updates and coordinating responses.

On the right, the optimization block represents the closed-loop control functions. These include tuning low-margin networks, configuring equipment, managing faults, optimizing resources, and planning expansions.

Information then returns to the physical layer, forming the continuous cycle of monitoring, modeling, and control that defines the DT's operation.

B. FAST MODELING FOR REAL-TIME USE

To support automation, the DT must offer models that are accurate enough to be useful and fast enough to run within

operational time scales. Mirror modeling in optical networks typically relies on three families of methods: numerical or analytical models, data-driven machine learning, and physics-informed machine learning [61].

Analytical and numerical tools, such as Gaussian noise (GN) models for QoT estimation and split-step Fourier methods (SSFM) for fiber propagation, can describe a wide range of linear and nonlinear effects with high fidelity [64]. Data-driven approaches, including neural networks for fiber channels or amplifier behavior, provide faster inference once trained and can adapt to conditions not fully covered by analytical formulas [61]. Physics-informed ML combines both views, embedding constraints from propagation physics into learning models so they remain consistent with known laws while still leveraging data [61].

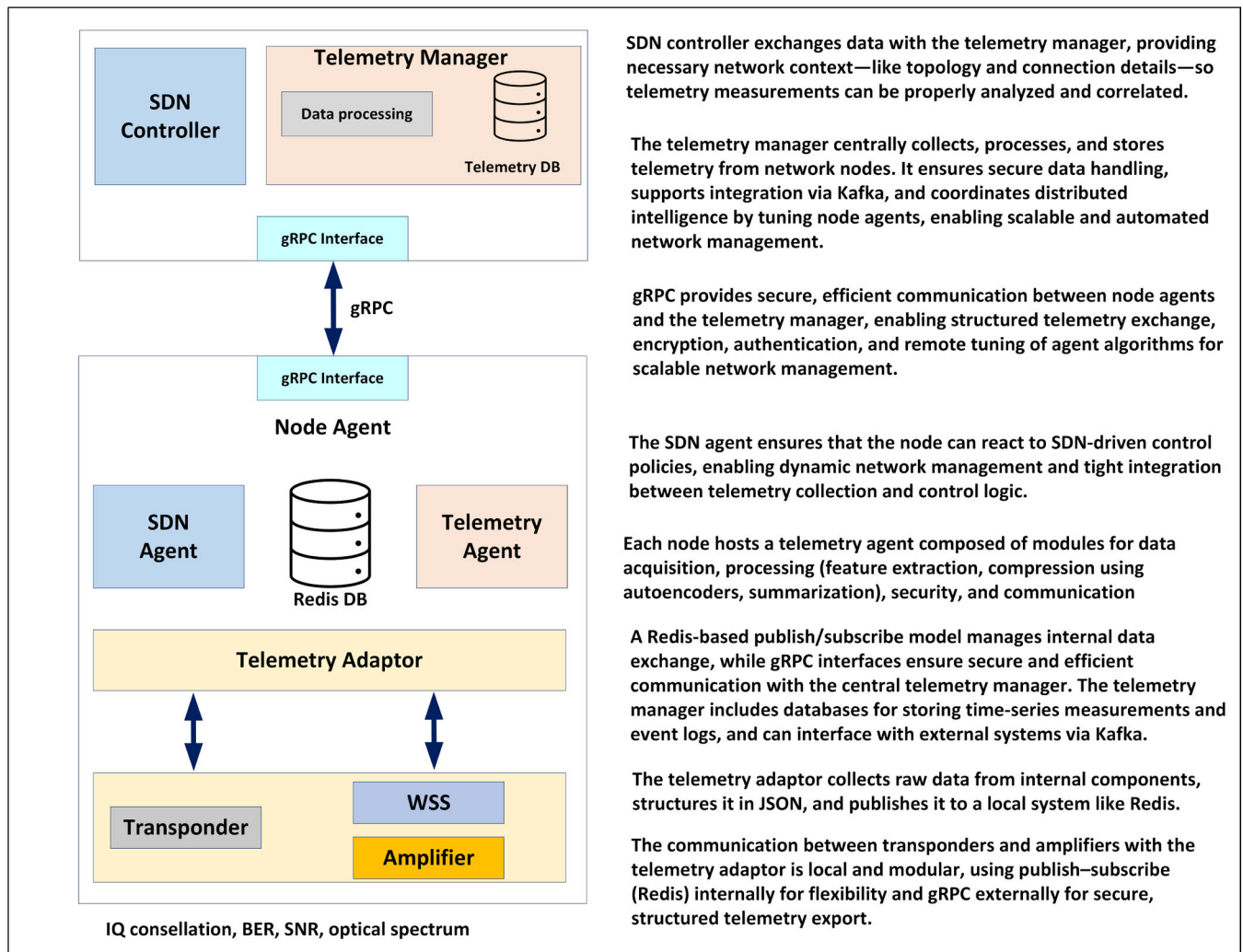


FIGURE 4. Distributed telemetry pipeline [3].

TABLE 3. ML-only vs. hybrid ML-DT-LLM automation pipelines in optical networks: architectural and operational trade-offs.

Aspect	ML-only pipeline	Hybrid ML-DT-LLM pipeline
Decision structure	Direct inference from telemetry using trained models [14], [109].	ML inference combined with DT validation and LLM-guided planning [2], [61].
Latency	Lowest inference latency due to single-stage execution; limited safeguards [14].	Higher end-to-end latency due to validation and coordination, mitigated by edge preprocessing and asynchronous execution [64].
Scalability	Scales well for isolated tasks, but centralized inference and retraining can become bottlenecks as network size grows [14].	Horizontal scalability enabled by distributed telemetry, modular agents, and parallel DT simulations [15], [61].
Reliability	Sensitive to model drift and unseen conditions; prediction errors may propagate directly to control actions [109].	Higher reliability through physics-based DT checks, policy constraints, and staged decision paths [61], [64].
Operational risk	Higher risk when actions are applied without explicit safety validation [109].	Reduced risk due to pre-deployment validation, explainability, and controlled execution [2].
Explainability	Limited to model-level indicators such as feature importance or confidence scores [109].	Human-readable explanations grounded in intent, telemetry context, and DT outcomes [2], [70].
Adaptation to new scenarios	Requires retraining or manual tuning when conditions change [14].	Adapts via telemetry-driven DT updates and LLM-guided workflow adjustment [2], [64].
Support for closed-loop automation	Suitable for local or narrow control loops [14].	End-to-end loops spanning perception, planning, validation, and execution [2], [61].

Generative methods such as GANs generative adversarial networks (GANs) and variational autoencoders (VAEs) can

extend the data available to these models by synthesizing realistic operating conditions, including rare or extreme

TABLE 4. Centralized vs. distributed telemetry in optical networks: qualitative and quantitative comparison.

Aspect	Centralized telemetry (cloud or NOC centric)	Distributed telemetry (edge centric, hierarchical)
Data path and placement	Devices stream most raw measurements to a central collector or data lake, concentrating ingest and analytics load [3].	Node-level agents preprocess locally, keep short windows in local stores, and forward reduced streams to central platforms for global correlation, DT, and automation [3], [14].
Data volume and bandwidth (example figures)	With 50 nodes (illustrative core network), rich telemetry at high granularity can reach 15.64 TB/day (5.58 PB/year) [3].	Local reduction can cut daily volume, e.g., 541 GB/day down to 3.74 GB/day with feature extraction and compression, and to 0.21 GB/day with summarization [3].
Latency and streaming performance (reported)	Central ingest and storage can become bottlenecks under high-rate feeds, especially with spectra and IQ transported upstream [3].	Kafka-based pipelines report producer-to-consumer latency around 50 ms while sustaining about 4000 messages/s [13]. A unified platform combining distributed and centralized InfluxDB with Kafka reports end-to-end latency below 0.05 s with optimized partitioning [14].
Scalability under many producers and consumers	Scaling often requires vertical expansion and careful backpressure, otherwise hot spots appear [3].	Partitioning and replication distribute load across brokers and consumer groups. Reported scalability tests reached 3920 messages/s with multiple consumer groups and CPU below 50% on brokers in the worst case [13].
Flexibility for heterogeneous data	Strong schemas help consistency but can complicate onboarding of mixed streams such as KPIs plus events and logs [3].	Event streaming accommodates heterogeneous sources more easily, while edge agents normalize vendor-specific formats before publishing [3], [14].
Fault tolerance and resiliency	Central collector failures can impair observability unless redundancy and failover are engineered [3].	Replication and broker failover preserve delivery during failures, with no message loss reported in tests [13].
Compute footprint and operational trade-offs	Lower edge compute needs, but higher transport and central compute, and expensive storage growth if high-dimensional telemetry is retained [3].	Requires compute at nodes for feature extraction and summarization, reducing upstream bandwidth and central storage; reported processing times were feasible for near-source operation [3].
Fit for DT and closed-loop automation	DT synchronization can lag if ingestion queues build up or data is too heavy for the required cadence [3].	Reduced and prioritized streams plus low-latency transport enable faster DT refresh; unified platforms explicitly target feeding AI engines and DT functions from Kafka and time-series stores [14].

cases [65], [66]. This helps the DT explore congestion, peak loads, security-related events, and recovery scenarios that may not appear often in historical records [66]. At the same time, operational deployments must respect limits on central processing unit (CPU) and memory, especially when DT functions run in distributed environments. If models are too detailed, they may become too slow for real-time use; if they are oversimplified, they may ignore important wideband or device-specific effects [64], [67]. Practical DT designs therefore mix lightweight models for fast decisions with more complex simulations that run less frequently or off the critical path.

Depending on timing requirements, resource availability, and deployment constraints, DT functions may execute either centrally in cloud or network operation center (NOC) environments, or closer to the network edge. The literature reports clear trade-offs between these execution models in terms of latency, scalability, computational load, and suitability for real-time control. Table 5 summarizes these differences, highlighting how edge-assisted and cloud-based DT execution complement each other in practical optical network deployments.

C. TELEMETRY AS A SOURCE OF UPDATES

The DT depends on telemetry to stay synchronized with the physical network. Monitoring functions collect measurements from coherent receivers, inline monitors, SDN controllers, and environmental sensors to track changes in traffic, signal quality, and device status [61], [62]. A growing

share of these measurements now comes from software-based sensing in DSP chains, which reduces the need for dedicated hardware probes and scales better for large networks [61].

Before entering DT, raw inputs pass through data fusion pipelines that clean, normalize, and align heterogeneous sources [63]. Typical steps include desensitization, noise removal, naming harmonization, and normalization of units. Sampling and augmentation stages reduce redundancy, balance data distributions, and generate synthetic samples when necessary, improving the robustness of downstream models [63]. Time-series and event databases, such as InfluxDB [26] and Elasticsearch [29], preserve synchronized views of key parameters and alarms that the DT uses to reconstruct the current operating point.

During operation, telemetry streams from transceivers and inline monitors refine DT parameters, adjust fiber and device models, and update margin estimates in near real-time [64]. End-to-end margin analysis uses these updated models to quantify how much tolerance remains against noise, nonlinearities, and hardware variations. Adaptive learning components inside the DT can then retrain or fine-tune models when new patterns emerge, so that the virtual representation evolves along with the network [28], [64]. Without this continuous feedback from telemetry, the DT would quickly drift away from reality and lose its value for automation.

Figure 6 outlines the typical data fusion stages used before telemetry enters a DT system. Raw measurements from receivers, controllers, and inline monitors are first

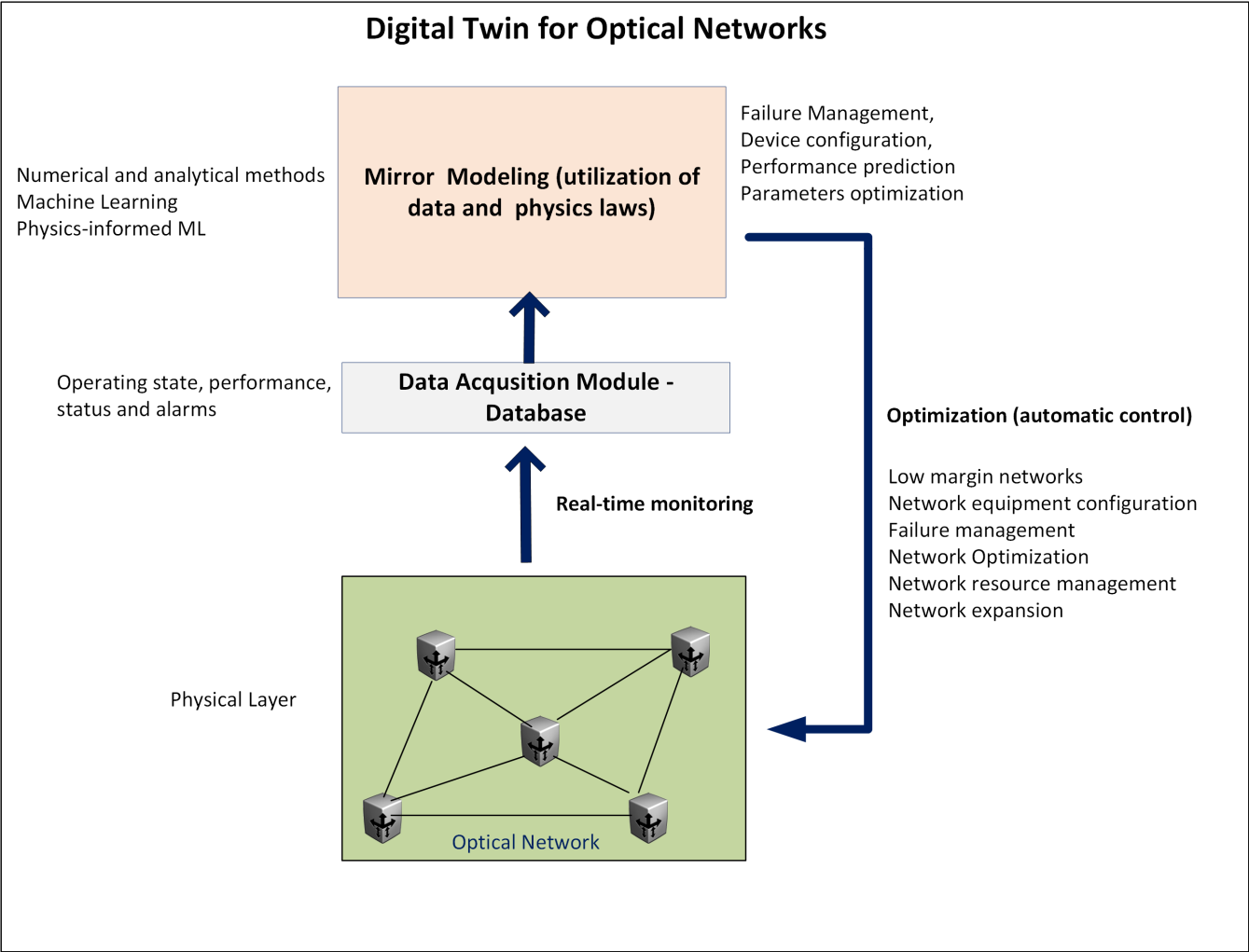


FIGURE 5. Digital twin for optical networks.

TABLE 5. Comparison of edge and cloud execution of DT functions in optical networks.

Aspect	Edge-based DT execution	Cloud-based DT execution
Execution location	DT components deployed close to transceivers, ROADMs, or local controllers, often co-located with telemetry agents [61].	DT components executed in centralized cloud or NOC environments with global network visibility [61].
Latency	Low latency due to proximity to telemetry sources, enabling near real-time validation and control [64].	Higher end-to-end latency due to transport and centralized processing, suitable for non-critical control tasks [64].
Model complexity	Lightweight analytical models and fast ML inference favored to meet operational timing constraints [64], [67].	Complex numerical simulations and physics-based solvers supported due to higher compute availability [64].
Compute and memory constraints	Limited CPU and memory resources in distributed or embedded environments [67].	Abundant compute and storage resources available for large-scale simulation and optimization [67].
Scalability	Horizontal scalability achieved by distributing DT functions across nodes [64].	Vertical scalability through centralized resource expansion, with potential ingest bottlenecks [64].
Telemetry dependence	Relies on locally preprocessed or summarized telemetry streams [28], [64].	Consumes fused, historical, and multi-layer telemetry aggregated from the network [63].
Support for real-time control	Well suited for fast QoT checks, margin tracking, and agent validation [61].	Better suited for planning, optimization, and what-if analysis outside critical loops [61].
Resilience to network impairments	Can continue operating during partial loss of connectivity to centralized systems [67].	Requires stable connectivity for timely telemetry updates and synchronization [67].
Typical DT functions	Fast QoT estimation, anomaly validation, local configuration checks [68], [69].	Network-wide planning, detailed QoT simulation, and long-horizon optimization [68], [69].

passed through cleaning and desensitization. Naming and unit harmonization make inputs consistent between vendors.

Sampling and augmentation remove redundancy and balance the dataset, while normalization prepares values for

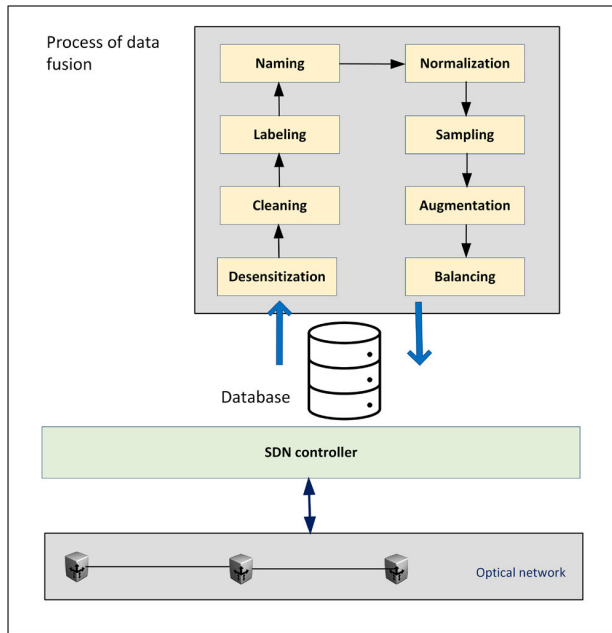


FIGURE 6. Data fusion pipeline for DT-based optical network monitoring.

downstream models. Once processed, these aligned streams provide the DT with a reliable view of the current network state, enabling accurate modeling and real-time updates. This pipeline ensures that the twin stays synchronized with the physical environment even when data sources are diverse.

D. DIGITAL TWIN AS A VALIDATOR FOR AGENTS

In an automation framework based on agents, the DT acts as a validation and planning engine. Before applying a change, an agent can query the DT to test whether a proposed configuration meets QoT requirements, respects device limits, and maintains sufficient margin [61]. For example, open tools such as GNPY estimate generalized SNR and other physical-layer indicators by combining fiber properties, amplifier settings, and topology information in a vendor-neutral way [68], [69]. Integrated into a DT, such tools allow agents to evaluate new lightpaths, amplifier profiles, or spectrum allocations virtually, then commit only those actions that pass predefined thresholds.

Generative components can further enrich this process by simulating a range of possible future states. GAN-based scenario generation allows the DT to explore how the network might behave under traffic surges, failures, or parameter drifts and to prepare mitigation plans in advance [66]. In turn, the automatic control layer uses DT outputs to drive device configuration, network reconfiguration, and recovery procedures, while keeping human intervention to a minimum [61].

Field experience shows that using DTs as validators brings both benefits and constraints. It improves confidence in agent decisions and supports low-margin designs, but it also requires robust telemetry pipelines, fast synchronization, and careful parameter identification in live environments

[64], [67]. Uncertainties caused by temperature changes, infrastructure aging, and manufacturing variability must be captured by DT models; otherwise, virtual predictions may misrepresent real conditions [67]. When properly calibrated and fed with high-quality telemetry, the DT becomes a powerful filter that helps agents avoid unsafe commands and enables more reliable closed-loop automation in disaggregated, multi-vendor optical networks.

Figure 7 illustrates the main elements of a transmission-focused DT. It brings together the effects introduced by fiber, amplifiers, filtering stages within ROADMs, and transceiver behavior into a single modeling structure. The fiber representation accounts for the two broad categories of propagation impairments. Linear effects, such as chromatic dispersion (CD), polarization mode distortion (PMD), loss mechanisms, and polarization dependent loss (PDL), describe gradual signal degradation as light travels along the span [2]. Nonlinear contributions, including Kerr-driven interactions such as self-phase modulation (SPM), cross-phase modulation (XPM), and four-wave mixing (FWM), as well as inelastic scattering processes such as stimulated Raman scattering (SRS) and stimulated Brillouin scattering (SBS) [2]. These nonlinear impairments grow with optical power and channel density, and therefore interact directly with linear noise mechanisms.

Optical amplifiers are explicitly modeled through their gain characteristics and noise figure, which determine the level of amplified spontaneous emission (ASE) added at each span. ASE constitutes the dominant linear noise source in long-haul and metro systems and plays a central role in defining the achievable OSNR. The overall transmission performance is therefore governed by a trade-off between linear noise accumulation, dominated by ASE, and nonlinear interference generated by high launch powers and dense WDM operation. The DT captures this balance to identify operating points that maximize end-to-end performance rather than optimizing either impairment class in isolation. Filtering effects from wavelength-selective switches and add-drop stages are included because they shape channel spectra, introduce passband narrowing, and affect inter-channel interactions. Transceiver contributions, including modem noise and DSP limitations, are represented as part of the end-to-end chain.

By combining linear noise sources, nonlinear interference, and component-specific characteristics, the DT can estimate generalized SNR (GSNR), evaluate performance margins, and perform what-if analysis for configuration changes. This capability allows controllers and agents to assess routing decisions, modulation formats, launch power settings, and amplifier configurations in a safe virtual environment before applying changes to the operational network [2].

V. LARGE LANGUAGE MODELS

A. INTENT PROCESSING

Large language models act as an interface between humans and automation systems by turning natural language requests

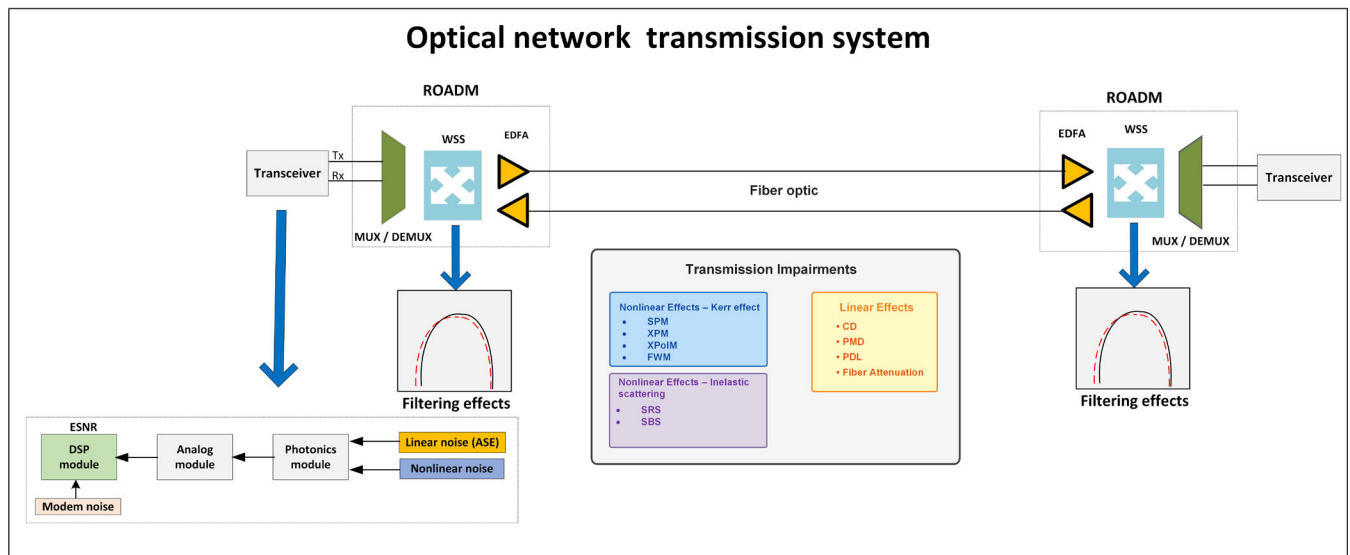


FIGURE 7. Optical transmission system architecture with modeling considerations for DT creation [2].

into structured actions [70]. Built on transformer architectures [71], they work on tokenized text embeddings and can map free-form operator instructions to specific tasks [72], such as querying a telemetry store, checking QoT, or planning a reconfiguration. Through fine-tuning [73] and prompt design [74], LLMs can be adapted to understand the vocabulary and patterns common in optical networking, including device types, service names, and operational procedures.

Figure 8 gives a simplified view of the transformer backbone used by most current large language models. The input text is first broken down into tokens and mapped to embeddings, then combined with positional information so that the model can keep track of word order [71]. Stacked attention blocks allow each token to focus on the parts of the sentence that matter most to the task, while feedforward layers refine these internal representations. Depending on the design, encoder stacks specialize in understanding input sequences [71], decoder stacks specialize in generation [71], and encoder-decoder models combine both roles for tasks such as translation and summarization [71], [72]. In the context of optical networks, LLM-centric architectures can interpret operator intent and convert high-level requests into executable actions [73], a capability that enables controllers to orchestrate telemetry collection, QoT validation, and reconfiguration tasks without relying on manually defined rule chains [70]. In practice, this intent-to-action flow allows an LLM agent to extract semantically relevant goals from natural-language input [73] and translate them into structured workflows that span measurement acquisition, performance assessment, and network-state updates, ultimately supporting closed-loop decision pipelines for provisioning, optimization, and failure response in disaggregated optical environments [70].

In this role, an LLM receives an operator query, interprets the intent, and produces a machine-readable description of what should be done, for example, a list of checks, parameters to retrieve, or configuration changes to be evaluated by downstream agents [2], [70]. Techniques such as in-context learning [75] and retrieval-augmented generation [76] help the model ground its answers in domain documentation and recent network data without requiring full retraining [77]. As a result, LLMs can help operators express objectives in a simple language while still driving precise and reproducible actions.

B. REASONING SUPPORT

Beyond intent translation, LLMs provide a reasoning layer over heterogeneous information sources. During inference, a model can read structured telemetry values, alarm text, log snippets, and maintenance notes, and then synthesize them into an explanation of current conditions [2], [70]. For QoT assessment, an LLM can combine margin reports and historical patterns to estimate whether a lightpath is likely to meet performance requirements without running full simulations in every case [70].

In fault management, LLMs can correlate alarm sequences with changes in BER, OSNR, or amplifier gain and propose likely causes such as tilt, filter misalignment, or fiber cuts [78], [79]. Chain-of-thought prompting and related reasoning techniques help the model make its intermediate steps explicit, which can be valuable when operators need to review or challenge its suggestions [80], [81]. LLMs can also propose candidate actions, such as rerouting, spectrum adjustment, or modulation changes, which are then passed to agents, DTs, or traditional ML modules for validation and execution [2], [82]. In this way, the model becomes a support

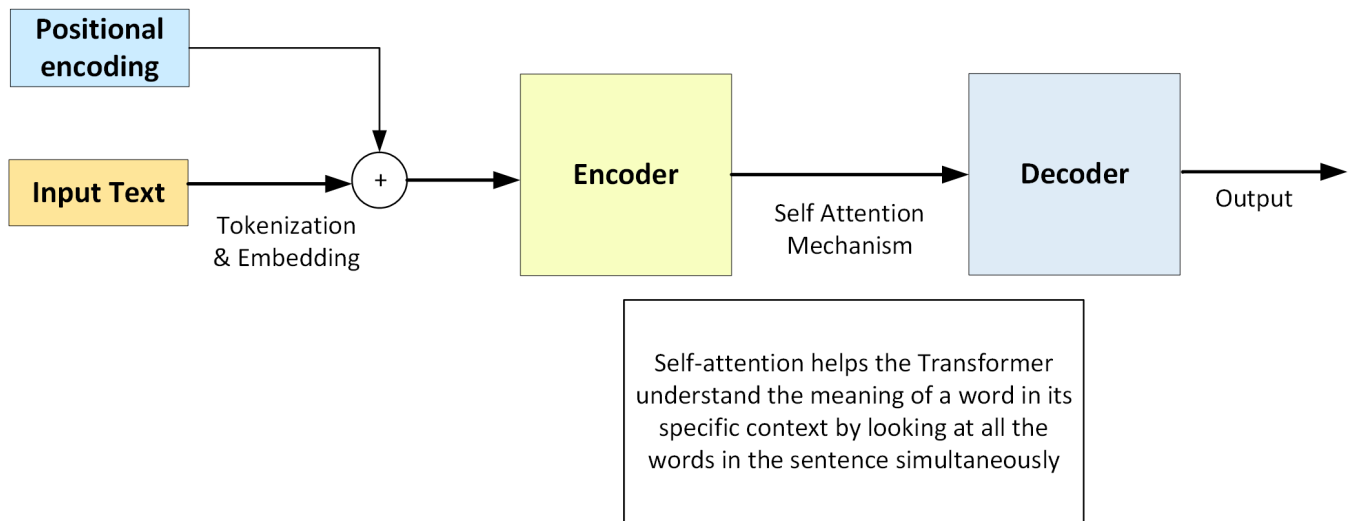


FIGURE 8. Transformer architecture.

tool for diagnosis and decision-making rather than a direct controller.

C. LIMITATIONS

LLMs do not have direct access to the physical state of the network and cannot sense the system on their own. They depend entirely on the quality, coverage, and freshness of the data they receive through telemetry streams, logs, and external knowledge bases [2], [70]. If these inputs are incomplete, inconsistent, or outdated, the model may produce answers that sound convincing but do not reflect real conditions, a behavior commonly described as hallucination [79], [83].

Training and fine-tuning also introduce limitations. Domain adaptation requires curated datasets and careful data cleaning, which can be costly in environments with sensitive or proprietary information [73], [84]. Even when adapted, LLMs can be vulnerable to biases, adversarial prompts, and jailbreak attempts that push them toward unsafe or undesired outputs [83], [85]. From a systems perspective, continuous inference over high-frequency data can place heavy demands on compute and energy resources, especially if large models are used in real-time loops [2], [82], [86]. These constraints reinforce the need to place LLMs in supervised positions in the architecture, where other components verify their recommendations before any action reaches the control plane.

D. ROLE IN TELEMETRY WORKFLOWS

Within telemetry-centric automation, LLMs operate as high-level interpreters and coordinators rather than low-level streaming engines. Telemetry pipelines and time-series databases deliver structured metrics, while log and alarm systems provide unstructured text. The LLM consumes these inputs and produces summaries, groupings, and action plans

for operators and agents [2], [14]. For example, the model can filter repetitive alerts, cluster related events, and express the situation in concise language that highlights what changed, where, and since when [70], [87]. It can also transform natural language instructions into queries over telemetry stores or into configuration templates that are then checked and applied by SDN controllers and agentic frameworks [69], [88]. In hybrid AI workflows, LLMs coordinate with classical models by delegating purely numerical tasks to decision trees, random forests, or specialized QoT estimators, while focusing on tasks that require explanation and multi-step reasoning [35], [79]. Because of these characteristics, LLMs naturally fit at the top of telemetry workflows. They sit above streaming protocols and databases, read the processed outputs, and communicate with humans and agents. This placement allows them to add value in interpretation and coordination while keeping direct control actions under the supervision of DTs, rule-based systems, and other ML components that are tightly bound to physical-layer constraints [2], [70], [82].

VI. AGENTIC AI

A. CONCEPT AND DEFINITION

Agentic AI interprets automation as a collection of autonomous software entities that observe the network, interpret the context, and act within a delimited scope to satisfy operational objectives [15]. An AI agent is a computational process that receives input from its environment, processes this information, and selects actions that support a given goal, such as maintaining service quality or restoring a lightpath. Agentic AI extends this notion by organizing multiple agents in a shared environment where they exchange information, share context, and factor complex objectives into smaller tasks that can be handled in parallel [15].

In optical networks, an LLM based agent corresponds to a particular instance of this concept that integrates a large language model with domain specific tools, telemetry feeds, SDN APIs, and DT functions [2]. This agent parses intents, reasons about network state, and executes steps for service provisioning, QoT validation, alarm triage, and failure recovery by grounding decisions in current measurements and domain knowledge.

A recent survey on agentic systems highlights a key insight that directly reinforces the framework view described in this manuscript [16]. The study shows that agentic AI is not simply an extension of single LLM agents, but a broader paradigm defined by structured planning, explicit goal decomposition, memory, tool interaction, and coordinated multi-agent behavior. These properties allow agents to operate over longer time horizons, refine decisions through feedback, and collaborate to complete tasks that exceed the reach of an isolated entity. This perspective aligns with the architecture described here, where autonomous components use telemetry, DTs, and SDN interfaces to reason about network state and work collectively toward operational objectives.

B. MULTI-AGENT COLLABORATION AND ADVANTAGES

Within this paradigm, several specialized agents collaborate to achieve high level operational goals that exceed the capabilities of any single entity. Traditional automation in optical networks often relies on rule based logic or local optimization routines, which are effective for static tasks but limited in flexibility when conditions evolve. Agentic AI instead allows agents to adapt behavior based on streaming telemetry, cross agent coordination, and feedback from DT simulations and operator input [15].

Each agent acquires situational awareness through telemetry streams, logs, alarms, and status queries that expose metrics such as BER, OSNR, received power, and spectral occupancy [89]. These data feeds are processed by perception modules that normalize formats and attach metadata such as device identifiers and timestamps [90], [91]. Reasoning modules then combine this context with policies and intents to derive actions. The execution modules interface with SDN controllers, element managers, and DT platforms to apply changes or test candidate configurations. This decomposition allows complex workflows, such as coordinated reconfiguration after a fiber cut, to be split into explainable, auditable responsibilities distributed across agents.

To clarify the differences between traditional automation approaches and agentic AI, Table 6 compares rule-based automation with agent-driven systems as discussed in the literature. The comparison highlights differences in adaptability, coordination, scalability, and suitability for real-time, closed-loop operation in optical networks, based on reported design characteristics and operational experience.

1) MICROSERVICES ANALOGY AND EVENT-DRIVEN ARCHITECTURE

The structure of an agentic AI system resembles a microservice architecture, where each component plays a focused role and can be developed, deployed, and scaled independently [92]. Unlike conventional microservices, however, agents reason about state, adjust policies, and learn from feedback over time [93], [94]. If coordination is not handled carefully, large deployments can become fragmented and difficult to manage, much like microservice ecosystems without shared communication or resilience patterns.

To address these challenges, agentic AI adopts an event driven architectural style. Instead of relying solely on request response interactions, agents operate in an ecosystem where they publish and subscribe to events that describe telemetry updates, alarms, state changes, and planning decisions [89], [95]. A streaming backbone ingests measurements from device telemetry, DSP observables, OSAs, and SDN events, and then exposes these measurements as topics or event streams that agents can selectively consume [89]. This approach offers loose coupling, asynchronous processing, and the ability to introduce new agents or modify existing ones without disrupting ongoing workflows.

C. MULTI-AGENT SYSTEMS AND COORDINATION

The shift from individual agents to multi-agent systems (MAS) mirrors the evolution from monolithic controllers to distributed control planes in large networks [92]. A single agent has limited coverage and cannot maintain sufficient knowledge of all services, devices, and policies in a multi domain optical infrastructure. MAS architectures instead distribute responsibilities so that agents can specialize in functions such as QoT estimation, routing and spectrum assignment (RSA), failure diagnosis, or policy enforcement. This distribution supports task delegation, parallel execution, and dynamic role assignment as network conditions change [90].

Coordination in MAS favors indirect, message based communication over tight, point to point dependencies. Agents publish events related to anomalies, configuration proposals, and validation results. Other agents subscribe to relevant event types and react according to their roles. For example, a monitoring agent may publish an event when it detects a drop in received power or a trend in BER that indicates degradation. A fault analysis agent subscribes to these events, gathers additional measurements, and correlates them with historical patterns to identify likely root causes such as filter misalignment or amplifier tilt [78]. A QoT agent reevaluates the margins of affected lightpaths and flags those close to their limits [70]. Routing and spectrum agents can then explore alternative paths and slot allocations, while a DT agent simulates candidate solutions.

The coordination loop summarized in Figure 10 illustrates this event-driven collaboration. Telemetry from DSP units, optical line amplifiers, optical spectrum analyzers (OSAs),

TABLE 6. Comparison of rule-based automation and agentic AI in optical network management.

Aspect	Rule-based automation	Agentic AI
Decision model	Predefined rules and static logic triggered by fixed thresholds or events [90].	Goal-driven decision making with reasoning, planning, and adaptive policies [15], [93].
Adaptability to changing conditions	Limited adaptability; rules must be manually updated when conditions or policies change [90].	Dynamic adaptation based on telemetry, feedback, and learned experience [15], [94].
Context awareness	Local or narrowly scoped context, often limited to individual devices or alarms [89].	Global and shared context across agents, including historical state, intent, and DT feedback [2].
Handling of complex workflows	Difficult to scale beyond linear or predefined workflows [90].	Complex, multi-step workflows decomposed across cooperating agents [15], [100].
Coordination and scalability	Centralized or tightly coupled logic with limited horizontal scalability [92].	Distributed multi-agent coordination using event-driven communication and loose coupling [92], [99].
Use of telemetry	Telemetry used mainly as triggers for rule execution [89].	Telemetry continuously feeds perception, reasoning, and learning loops [2], [89].
Support for DTs	DTs used offline or as external validation tools [61].	DTs integrated into decision loops for continuous validation and planning [2], [61].
Explainability and auditability	High transparency at rule level, but limited explanation for emergent behavior [90].	Decisions can be explained through intent, reasoning traces, and DT outcomes [2], [93].
Learning and evolution	No inherent learning; behavior changes only through manual rule updates [90].	Learning from feedback, operator input, and performance outcomes [38], [94].
Operational scope	Best suited for static tasks and well-defined failure scenarios [90].	Suitable for dynamic environments, multi-domain coordination, and autonomous operation [15].

and SDN controllers is continuously collected and published onto an event bus that transports state updates to monitoring, analysis, planning, and control agents. In this context, amplifiers refer to in-line optical amplifiers, such as EDFAs and Raman amplifiers, which expose operational parameters including gain settings, output power, alarms, and OSNR-related indicators through embedded monitoring functions or associated DSP measurements. These amplifier-related observables are abstracted as telemetry events and are correlated with DSP-derived metrics such as BER and constellation statistics before being consumed by higher-layer agents. This architecture minimizes tight coupling between agents and physical devices, allows new agents to join the workflow by subscribing to relevant event topics, and increases resilience by enabling redundant or backup agents to assume responsibilities when another component becomes unavailable [90], [94].

D. END-TO-END WORKFLOW EXAMPLE: FIBER DEGRADATION TO SDN ACTION

To make the agentic loop concrete, this subsection outlines a representative end-to-end workflow in which gradual fiber degradation triggers a coordinated response. The example follows the sequence: degradation in the field, telemetry anomaly, multi-agent reasoning, DT validation, and SDN action.

Step 1. Physical degradation and early symptoms. A fiber segment gradually accumulates extra attenuation or experiences a spectral tilt. Early indicators appear as slow OSNR reduction and increasing pre-FEC BER on one or more coherent channels. In parallel, the spectrum monitor may reveal a drift in the noise floor or an emerging ripple pattern.

Step 2. Telemetry anomaly event creation. A monitoring agent subscribes to QoT and optical telemetry topics and

maintains per-lightpath baselines and trend features. When it detects a sustained deviation, for example, an OSNR drop beyond a learned trend bound or a monotonic BER increase, it publishes an *anomaly event* containing a timestamp, affected services, recent KPIs windows, and confidence metadata [3], [89]. Confidence metadata captures indicators of the reliability and certainty of an anomaly detection, such as confidence scores, data quality metrics, and temporal stability.

Step 3. Multi-agent reasoning and hypothesis ranking. A fault analysis agent consumes the anomaly event, queries additional context such as amplifier setpoints, ROADM filter configuration, and recent maintenance actions, and generates ranked hypotheses. For example, it may separate the likely fiber degradation from filter tightening or EDFA gain drift by comparing multi-span signatures, power profiles, and the spatial pattern of degradations [78]. In parallel, a QoT agent recomputes margins and flags services closest to violation [70]. If the event indicates spectrum pressure or fragmentation, an RSA agent prepares candidate reroutes or slot reallocations.

Step 4. DT validation and feasibility checks. Candidate actions are not applied directly. A DT agent runs feasibility checks by simulating proposed changes, for example, power rebalancing, modulation downgrade, route change, or spectrum shift. DT evaluates constraints such as OSNR margin, nonlinear penalties, filter cascade limits, and equipment capabilities, and returns a pass or fail result plus expected margins [2], [61]. Unsafe actions are discarded, and the remaining candidates are ranked according to risk and expected improvement.

Step 5. Controlled SDN execution and verification. An execution agent converts the selected candidate into a structured change request for the SDN controller, including guardrails such as max power deltas, rollback conditions,

and a post-change verification plan. The SDN controller applies the change, and the monitoring agent verifies recovery by checking whether KPIs return to the expected region. If verification fails, the workflow triggers a rollback and escalates to an operator with an evidence bundle, including telemetry plots, DT results, and the final action trace [2], [93].

The workflow also incorporates safeguards against common failure modes in agentic systems. Conflicting agent decisions are handled through DT-based validation and ranking, which enforces a single feasible action path and prevents concurrent or incompatible changes from being applied [96]. Stale telemetry is mitigated by associating timestamps and freshness thresholds with telemetry and anomaly events; actions are blocked when required inputs exceed predefined age limits [97]. DT desynchronization is detected through continuous reconciliation between simulated and measured KPIs; when divergence exceeds predefined limits, DT parameters are refreshed or candidate actions are revalidated before execution [61], [97]. Finally, LLM hallucinations are managed by restricting the LLM role to intent parsing, hypothesis explanation, and plan orchestration, while all safety-critical decisions are validated by external numerical models and digital twin simulations before execution, reflecting the need for non-LLM verification mechanisms in safety-critical systems [2], [98]. These mechanisms ensure that semantic reasoning complements, rather than replaces, physics-based validation and control.

This example illustrates why agentic AI is more than alarm triage. It is a closed loop with explicit event generation, distributed reasoning, DT-based validation, and controlled actuation over SDN interfaces.

E. FAILURE MODES AND SAFEGUARDS IN AGENTIC AUTOMATION

Agentic systems introduce new failure modes because decisions are distributed across multiple agents and may involve probabilistic inference and learned policies. This subsection summarizes common failure modes and practical safeguards.

Conflicting agent decisions. Conflicts occur when agents propose incompatible actions, for example a power-control agent increasing launch power while a nonlinear-risk agent recommends the opposite, or an RSA agent moving spectrum while a protection agent tries to hold the current configuration. *Safeguards:* enforce a single *commit point* through an orchestrator or policy agent that resolves conflicts using priorities, constraints, and objective functions. Use action locking per resource, for example per lightpath or per ROADM degree, and require DT validation for all actions that change the physical layer settings [15], [93].

Stale telemetry and delayed events. Staleness can lead to actions that are optimal for an old state. This is common when sampling policies change, collectors lag, or backpressure builds in the streaming layer. *Safeguards:* attach timestamps and freshness scores to all events, reject actions if the

telemetry window exceeds a staleness threshold, and trigger a fast re-sample or on-demand query before committing. Maintain heartbeat topics for key telemetry feeds and degrade to conservative policies when freshness is poor [3], [13].

DT desynchronization from the physical network. A DT may drift if topology, calibration, or equipment state changes are not reflected quickly, which can cause false passes or false fails in validation. *Safeguards:* couple DT state updates to the same event streams used by control, and version DT inputs, topology, and device models. Require periodic reconciliation, for example, comparing predicted versus observed OSNR or power evolution, and downgrade the DT authority when residual errors exceed a limit [2], [61].

LLM hallucinations and unsafe reasoning traces. LLMs can produce plausible but incorrect explanations, incorrect parameter names, or invalid action sequences if the context is incomplete. *Safeguards:* restrict the LLM to planning and explanation while delegating numerical inference and validation to ML models and DT checks. Use retrieval-augmented grounding for facts, schema validation for structured outputs, and tool gating that prevents direct device commands from free-form text. Apply human-in-the-loop escalation for high-impact actions and require DT pass results before SDN execution [2], [61], [70].

Cascading automation errors. Even correct local actions can cascade if applied repeatedly across many services, for example, oscillations in power control or repeated reroutes that amplify fragmentation. *Safeguards:* rate-limit actions per domain, introduce cool-down timers, and add global coordination constraints at the orchestrator. Use post-action verification and rollback policies, and maintain incident-level memory so that the system does not repeat a failed strategy [93], [94].

These safeguards turn agentic AI into a controlled closed loop, where autonomy is bounded by freshness checks, DT validation, policy arbitration, and auditable execution.

F. LLMs AS THE REASONING CORE

Large language models act as the main reasoning engines in the agentic stack. They provide semantic interpretation of intents, contextual synthesis of telemetry and topology information, and translation of natural language goals into structured plans that agents can execute [2]. An operator can, for instance, specify a request such as “restore the path with highest GSNR subject to latency below 5 ms.” The LLM parses the goal, identifies the affected services and constraints, and produces a sequence of abstract actions that include QoT validation, path search, and risk assessment [2].

These plans are grounded in network reality through integration with DTs and telemetry pipelines. The LLM uses retrieved measurements, historical traces, and model output to refine its suggestions. Before changes are applied to the live network, candidate actions are executed in the DT, which acts as a high fidelity replica of the optical infrastructure capable of simulating power evolution, OSNR, nonlinear effects, and equipment constraints. The combination of LLM reasoning

and DT validation reduces the risk of unsafe actions and supports explainable decision making for operators [61], [70].

G. CORE CAPABILITIES OF AUTONOMOUS AGENTS

Autonomous agents in this framework are built around a set of core capabilities that support closed loop operation [93], [94]:

- **Role** defines the agent function and domain expertise, such as alarm correlation, QoT evaluation, RSA, or policy enforcement. The role restricts which telemetry signals are relevant and which tools the agent can invoke.
- **Perception** describes how the agent acquires situational awareness. Typically, it consumes telemetry streams, log feeds, and SDN events through streaming middleware or APIs and transforms them into internal representations [90], [91].
- **Reasoning** captures the decision making process. It can range from rule based logic to LLM driven planning or reinforcement learning policies, depending on the complexity and criticality of the task [2], [93].
- **Memory** comprises short term context, such as the current incident or workflow state, and long term knowledge, such as past incidents, learned policies, and human feedback [94]. This memory allows agents to reuse prior experience and maintain continuity across interactions.
- **Action** refers to the execution of tasks. Agents may send commands to SDN controllers, interact with element managers, update configuration databases, trigger DT simulations, or call external tools for analytics [93].
- **Learning** enables agents to refine their behavior. Feedback can be obtained from DT outcomes, operator corrections, or performance metrics and used to adjust rules, update model parameters, or tune policies over time [38], [94].

Fig. 9 includes these capabilities as logical modules in the overall agentic AI framework, showing how perception, reasoning, memory, and action interact with the LLM layer, DT simulation, and SDN controllers.

H. MULTI-AGENT DEPLOYMENT PATTERNS

In practice, multi-agent deployments follow well known coordination patterns that determine how responsibilities are distributed and how conflicts are avoided [99]. In the orchestrator worker pattern, a main agent receives a high level intent, decomposes it into subtasks, and delegates these subtasks to specialized workers responsible for routing, QoT, spectrum control, or alarm handling [100]. Hierarchical arrangements use upper layers to define policies and objectives, while lower layers apply them to specific domains, regions, or devices. Blackboard or shared space architectures allow agents to interact indirectly by reading and writing to a common data structure that stores system state, candidate actions, and partial results [90]. Market oriented schemes, when needed, allocate tasks through bidding or negotiation.

These patterns are implemented on top of the streaming backbone that transports telemetry, alarms, and decisions. Locally deployed, fine tuned LLMs can be embedded within task oriented agents, such as a spectrum control agent that evaluates fragmentation or a management agent that classifies alarms and suggests corrective actions [91]. The coordination logic that binds these agents together forms the MAS layer in Fig. 9, and the resulting multi-agent workflow is summarized in Fig. 10.

I. LLM INTEGRATION LAYER AND WORKFLOW

The LLM integration layer forms the interface between operator intents, domain knowledge, and agent level actions. It combines a multimodal encoder, which can process textual descriptions, tabular telemetry, and topology sketches, with retrieval augmented generation (RAG) components that query documentation, configuration databases, and historical logs [2]. Parameter efficient tuning methods such as LoRA allow operators to adapt general purpose LLMs to optical networking tasks without full fine tuning [76], [101].

The intent driven workflow proceeds as follows. First, the operator issues a request, which is parsed by intent classifiers and entity extractors that map natural language to services, devices, and metrics [74], [77]. The LLM then constructs a structured plan that enumerates the steps required to achieve the objective of the operator, including assigning subtasks to the appropriate agents [70], [88]. This plan is published as a sequence of events or actionable tasks for the multi-agent system (MAS), enabling parallel or pipeline execution of configuration logic [70]. Each agent executes its part of the workflow while coordinating through shared context and event streams, and may invoke the LLM when semantic reasoning is needed for task explanation, summarization of telemetry trends, or generation of human-readable operational reports [70], [87]. For latency sensitive operations, communication between agents and the LLM layer uses low overhead protocols such as gRPC and efficient context management [91].

J. PERFORMANCE EFFICIENCY AND OPTIMIZATION

Performance constraints are central in optical network environments, where control actions must respect tight timing bounds. To meet these requirements, the framework relies on low bit quantization, selective fine tuning, and careful placement of models across CPUs and GPUs [38]. Critical components, such as channel setup and failure remediation, may use compact LLMs or non-LLM models for real-time inference, while a larger central model handles more complex reasoning and dialogue when time allows.

Validation in the DT environment further improves efficiency by filtering unsafe or unproductive actions before they reach the live network. By combining streaming telemetry, compact reasoning modules, and pre-computed policies, the system maintains near real-time responsiveness

while still taking advantage of LLM based reasoning when needed [2], [38].

K. ARCHITECTURAL GUIDELINES FOR AGENTIC AI IN OPTICAL NETWORKS

Recent work in autonomous networking suggests that agentic AI-based intelligence in optical transport systems tends to adopt layered and modular designs. Although this manuscript does not propose or validate a new agentic AI framework, the surveyed literature consistently exhibits a set of architectural patterns that occur in independent studies and deployments. Figure 9 therefore presents a reference architecture synthesized from these works, organized into four functional layers that support intent-driven and cognitively assisted operation, and used in this paper as an explanatory model rather than a prescriptive design.

At the base, the event-driven and data streaming layer collects real-time telemetry from transponders, ROADMs, amplifiers, OSAs, and SDN controllers and delivers structured updates to consumers through topics or event streams, for example, using Kafka-centric streaming stacks integrated with Spark Streaming (a distributed stream-processing framework for real-time data analytics) [89], [99]. Above this foundation, the MAS layer coordinates specialized agents responsible for tasks such as spectrum allocation, QoT evaluation, failure recovery, monitoring, and resource optimization [15], [100]. Each agent integrates the core capabilities described in Section VI-G and interacts with peers through events rather than direct calls, following event driven and holonic EDA patterns that decouple producers and consumers of services [92], [99].

The LLM integration layer provides semantic reasoning, intent parsing, and knowledge grounding. It translates operator goals into structured plans and assists agents with interpretation, summarization, and explanation [2], [93]. At the top, the control and validation infrastructure enforces safety by routing candidate actions through DT simulations and policy checks before SDN commands reach the production network [38], [61]. Low latency protocols and event based middleware ensure that this loop remains within the operational time scales required by restoration and optimization procedures [91].

Rather than prescribing a fixed design, these layers reflect architectural considerations that future agentic systems should incorporate to support reliable, explainable, and intent-aligned automation in multi-vendor optical environments.

L. AGENTIC AI SUMMARY

Table 7 summarizes the main elements of the agentic AI framework for optical networks. It consolidates conceptual foundations, event driven and streaming principles, MAS coordination, core agent capabilities, LLM integration, deployment patterns, and performance strategies [38], [91], [93], [94], [99], [100]. The table also links each aspect to representative methods and references, providing a compact

reference for how LLM enabled agents interact with SDN controllers, DT modules, and telemetry pipelines to deliver intent based, resilient, and scalable network management.

VII. OPERATIONAL TASKS

A. QoT MONITORING

Telemetry and agentic AI turn QoT monitoring into a continuous, closed loop. Agents subscribe to QoT indicators such as BER, Q-factor, OSNR, GOSNR (nonlinear Gaussian OSNR), GSNR, and effective SNR (ESNR) and track how linear and nonlinear impairments affect active lightpaths [102], [103], [104]. These indicators capture attenuation, chromatic dispersion, polarization effects, and nonlinear phenomena such as self-phase modulation (SPM), four-wave mixing (FWM), stimulated Raman scattering (SRS), and stimulated Brillouin scattering (SBS), giving a quantitative view of signal health across the network [102], [103].

For each service, agents maintain a view of the residual margin and how it evolves over time. When telemetry indicates that Q-factor or GSNR margins are shrinking, agents flag a potential degradation and consult QoT estimators to assess proximity to a service violation [105]. Analytical QoT models based on the Gaussian Noise (GN) family provide fast, physics-grounded predictions of nonlinear interference and maximum reach [106]. These models deliver estimates with low computational overhead compared to full split-step Fourier method (SSFM) simulation and have been refined with enhanced correction terms that improve accuracy across multi-span, coherent links [107]. In parallel, ML-based QoT estimators infer degradation trends directly from telemetry and remain effective even when link parameters are uncertain or partially unavailable [108]. Experimental comparison further shows that ML models can outperform analytical approaches in field scenarios where parameter uncertainty is high [109]. This complementarity allows agents to combine fast analytical forecasting with ML-driven robustness, supporting more reliable margin prediction in dynamic optical networks.

DTs amplify this process by simulating alternative routes, modulation formats, and power profiles. Agents can ask the DT whether moving a lightpath to a different path, adjusting spectrum, or tightening margins remains feasible under current conditions [61]. GAN-based and hybrid ML-physics models enrich DT with synthetic scenarios and refined parameters, improving the accuracy of QoT prediction in rare or extreme cases [110]. LLMs then help summarize the QoT status for operators, explain why a path is at risk, and produce QoT-aware action proposals that downstream agents can test and execute [70].

B. FAILURE MANAGEMENT

Failure management benefits directly from fast telemetry and coordinated agents [78]. Soft and hard failures affect lightpaths in different ways, from gradual QoT degradation to sudden loss of signal [78], [111]. Soft failures include filter

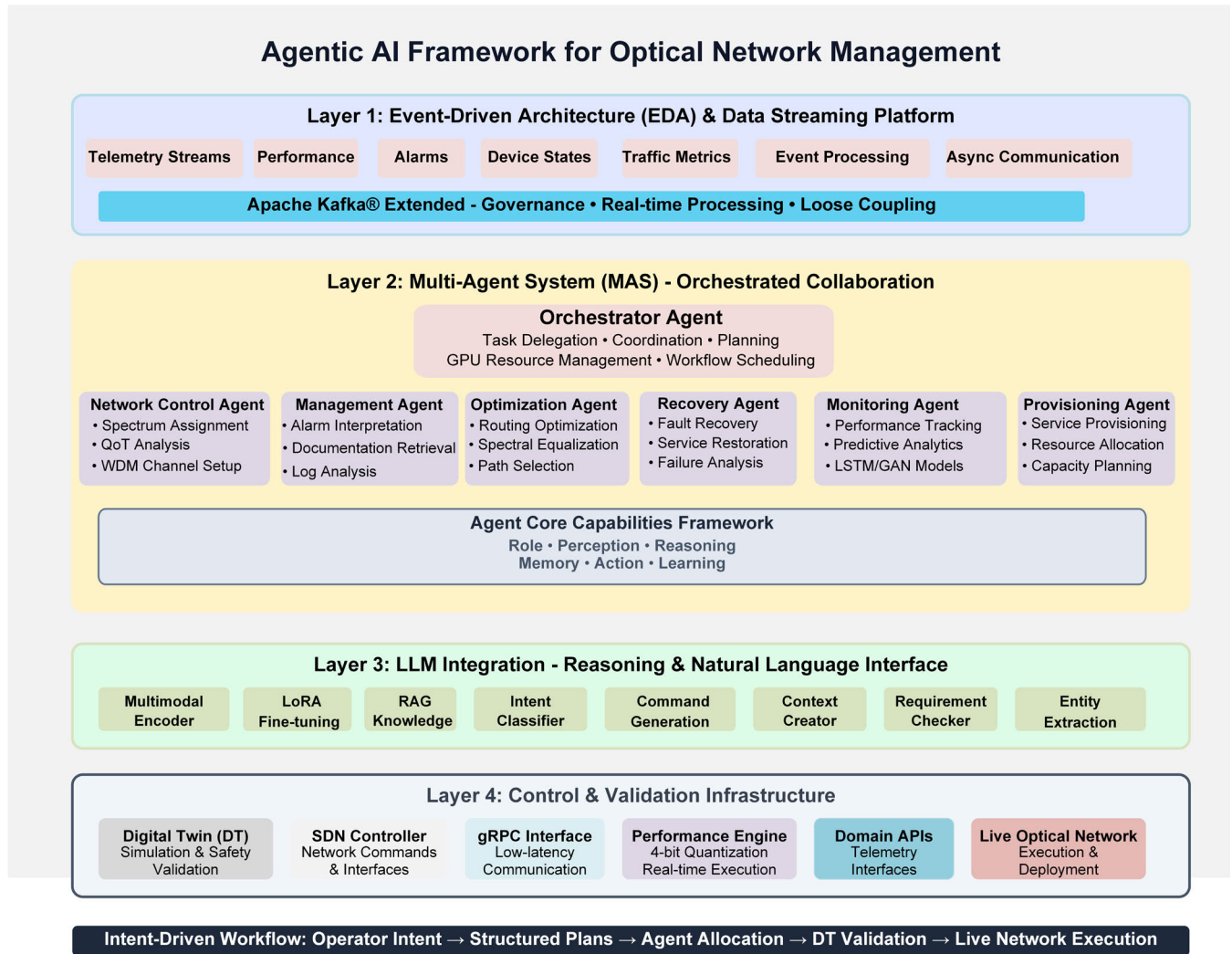


FIGURE 9. Reference agentic AI framework for optical network automation synthesized from literature.

shift, filter tightening, extra attenuation, and amplifier drift, which evolve slowly and may not trigger alarms immediately [112]. Hard failures, by contrast, cover disruptive events such as fiber cuts, power loss, and hardware faults in WSSs, amplifiers, or transceivers [78], [113].

Agents dedicated to failure detection use time-series ML models such as LSTM and GRU to follow OSNR, power, and related metrics and detect early deviations from normal network behavior [114], [115]. Support vector machines (SVMs) and clustering-based methods then analyze high-dimensional telemetry and traffic indicators to separate anomalies from normal fluctuations [116]. GANs expand training sets with synthetic fault patterns to improve robustness when real failure samples are scarce [117]. Meanwhile, constellation-based feature extraction from coherent receivers provides fine-grained impairment visibility, enabling soft-failure detection earlier than threshold-only monitoring [118].

Recent studies also show that coherent-receiver telemetry can help failure localization by reconstructing how

optical power evolves along the entire lightpath [61]. Instead of streaming raw samples, the receiver derives a compact longitudinal power profile and sends it through standard streaming interfaces, while the controller combines this profile with stored span parameters to identify where attenuation or filtering irregularities occur. This enriched view helps distinguish span-related issues from node-level problems such as misaligned WSS filters and enables faster classification of soft failures through profile-to-reference comparison. This approach provides a practical example of distributed preprocessing and telemetry-driven diagnosis for self-healing optical networks [119].

Once a deviation is detected, prediction agents estimate how the fault will evolve. Encoder-decoder LSTM models can forecast QoT trends and estimate time-to-failure, giving operators and agents a window to reroute or repair before hard failure occurs [120], [121]. DT-based frameworks compare measured signals and constellations against

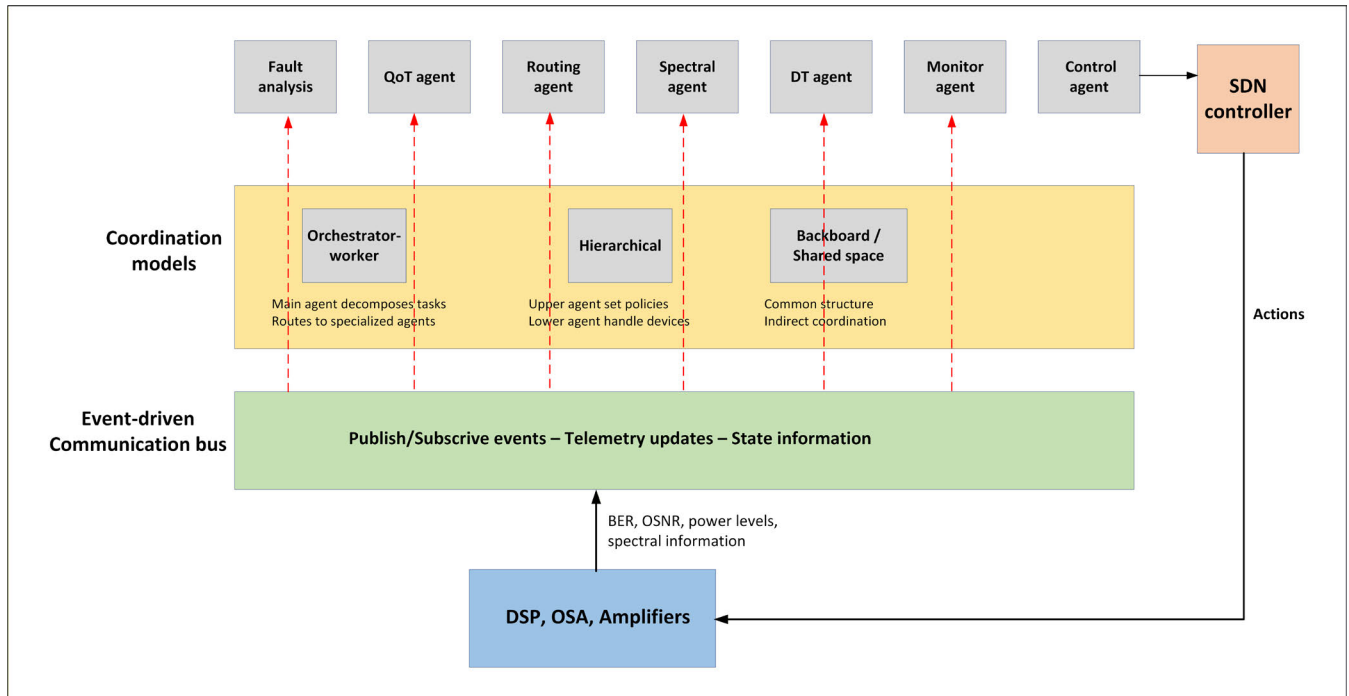


FIGURE 10. Event driven multi-agent coordination loop in optical networks.

simulated expectations to classify failure type and refine localization [64], [118].

Localization agents combine alarm correlation [122], spectrum analysis [123], BER behavior [124], and OTDR traces to pinpoint the faulty span or device [121]. LLMs assist in these workflows by reading large volumes of log messages, configuration files, and ticket histories, then suggesting likely root causes and summarizing remediation options in natural language [78]. Recovery agents finally use path computation, restoration policies, and DT validation to select new routes or reconfigure amplifiers, while DT checks whether the chosen plan is safe before live execution [60]. Figure 11 can depict this hybrid loop in which ML models detect and localize, DTs simulate, LLMs explain, and agents execute recovery actions.

C. RESOURCE OPTIMIZATION

Telemetry and agentic AI also support continuous resource optimization, particularly for routing and spectrum assignment, a central topic in elastic and flex-grid optical networks [125]. As traffic grows and becomes more dynamic, agents must decide how to allocate wavelengths, slots, and transponder formats so that blocking probability stays low and spectrum is used efficiently. These decisions are traditionally handled through RSA formulations [125] and more recent reinforcement learning approaches that learn allocation strategies under variable traffic patterns [126].

Figure 12 illustrates this closed loop. Telemetry, traffic data, and topology information feed the monitoring block, which maintains up-to-date views of spectrum occupancy and network conditions. These inputs flow into analytical models,

ML predictors, and DT systems that perform ongoing QoT estimation. The RSA decision engine then evaluates routing choices, spectral placement, and efficiency, using these QoT results to avoid paths with insufficient margins. Once a configuration is selected, the SDN controller, ROADM plane, and transceiver settings apply the change to the network. The output is a QoT-compliant lightpath, improved spectrum use, reduced blocking, or restoration actions with verified performance.

Optimization agents analyze traffic matrices, current spectrum occupancy, and QoT margins to recommend new paths or rearrangements. Deep reinforcement learning (DRL) agents learn policies that balance load, pick routes, and assign spectrum based on rewards tied to blocking rate, utilization, and QoT outcomes [127]. Diffusion models and other generative methods can extend exploration by generating candidate configurations and traffic patterns that go beyond those seen during training, helping DRL avoid local optima [117].

LLMs support these agents by processing topology descriptions, policy rules, and QoT summaries to produce human-readable recommendations and by translating high-level intents into structured optimization goals. Their ability to interpret network state and assist in routing and resource allocation tasks has been examined in recent studies that survey ML-based routing methods and the evolution of learning-driven resource allocation in optical networks [128]. Work on the integration of LLMs into optical automation shows that transformer-based models can parse operator intents, summarize complex states, and

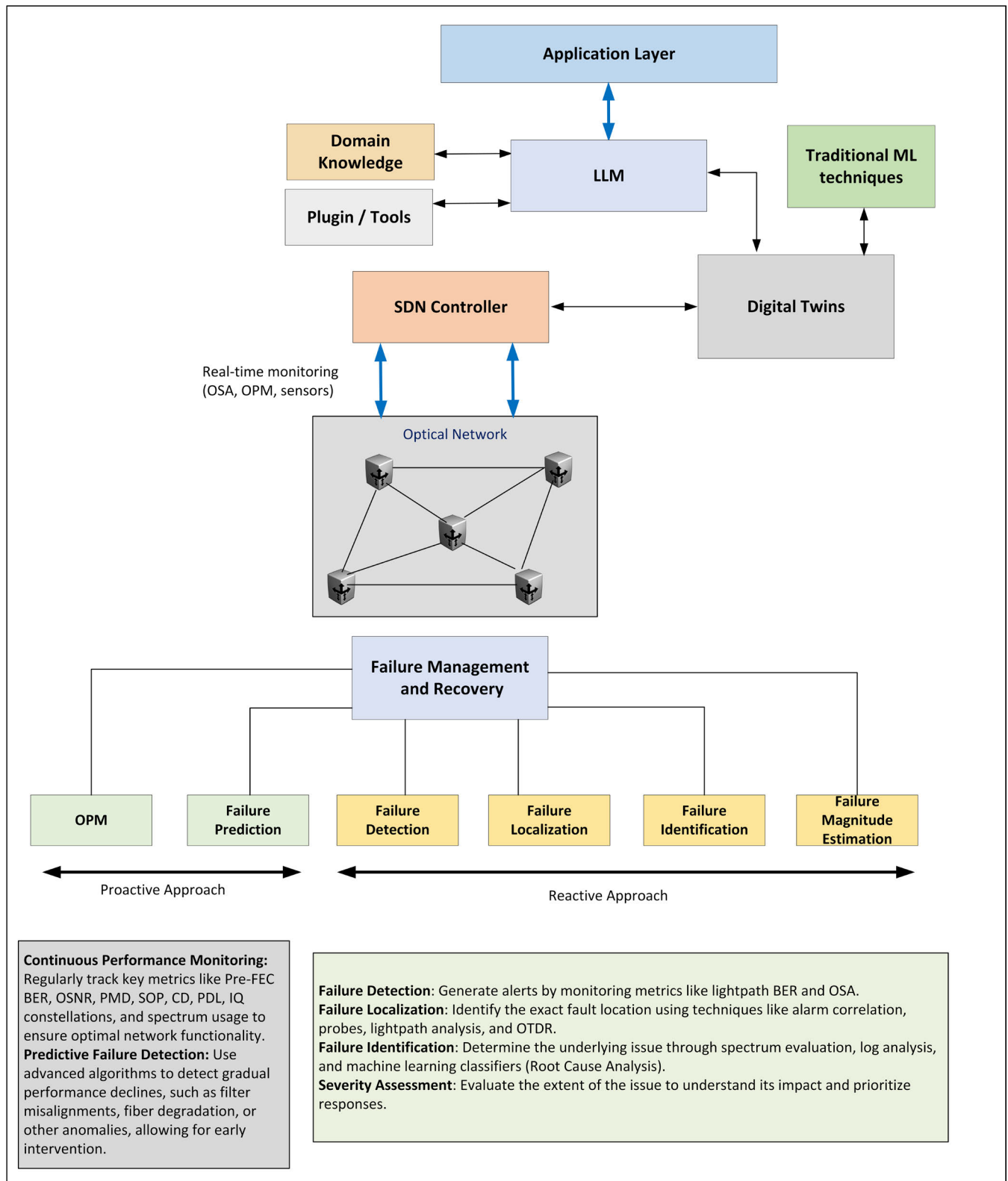


FIGURE 11. Failure management - hybrid approach.

generate configuration steps that align with traffic and QoT constraints, improving the workflow of optimization agents

in dynamic environments [2]. DTs provide the safe testbed for these changes, allowing agents to trial defragmentation

TABLE 7. Agentic AI in optical networks: condensed perspective.

Aspect	Purpose / What It Enables	Key Methods / Examples	References
Concept and definition	Explain how autonomous agents evolve into coordinated AI systems with domain reasoning and execution capabilities.	LLM assisted agents for provisioning, QoT evaluation, and service restoration in optical networks.	[15]
Multi-agent collaboration	Achieve complex multi step objectives by distributing responsibilities across specialised agents that share context.	Telemetry driven perception, distributed decision loops, coordinated execution through SDN and DT modules.	[15]
Microservices analogy and EDA	Promote modularity, loose coupling, and scalability in large automation deployments.	Event driven communication, streaming backbone, decoupled producers and consumers.	[89], [92], [95]
Multi-agent systems (MAS)	Distribute workload, enable parallel processing, and improve robustness to failures.	Indirect coordination via event streams, flexible role assignment, decentralized task execution.	[90], [92]
LLMs as reasoning core	Provide semantic interpretation and convert intents into executable plans validated through DT.	Intent parsing, structured plan generation, DT based pre checks, explanation of decisions for operators.	[2]
Core capabilities of agents	Define essential functions needed for autonomous and adaptive behavior.	Roles, perception modules, reasoning routines, memory stores, action interfaces, and learning loops.	[93], [94]
Deployment patterns	Organize cooperation among agents for efficient and scalable operation.	Orchestrator worker chains, hierarchical layers, blackboard collaboration, market based assignments.	[91], [99], [100]
LLM integration layer	Connect operator intents to domain tasks while improving alignment and efficiency.	Multimodal encoder, LoRA based tuning, RAG pipelines, prompt and context management, intent classifiers.	[2], [91]
Performance efficiency	Maintain near real-time operation under compute and memory constraints.	Low bit quantization, parameter efficient tuning, DT validation, gRPC based low latency communication.	[38]
Framework architecture	Support end to end intent driven orchestration with validation and controlled execution.	EDA backbone, MAS coordination, LLM reasoning layer, DT simulation checks, SDN based enforcement.	[2]

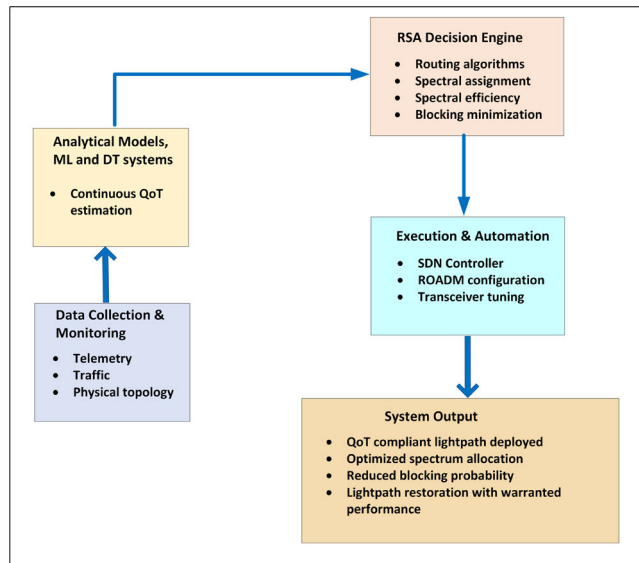


FIGURE 12. RSA closed loop.

moves, alternative paths, or modulation upgrades without affecting live traffic [61]. When the DT confirms that the QoT constraints are met, agents send the chosen configuration to the SDN controllers for the actual deployment.

In practice, this loop runs continuously: telemetry updates QoT and utilization views [5], agents propose adjustments [126], DTs validate them [2], and controllers apply

small configuration changes [5]. Over time, this reduces blocking, improves spectral efficiency [126], and keeps network operation closer to its optimal region with limited human intervention [5].

D. SOFT RECONFIGURATION

Soft reconfiguration refers to gradual adjustments that improve performance without changing the underlying topology. Instead of adding new links or nodes, agents tune how existing resources are used: power levels, spectral placement, baud rates, and modulation formats.

Telemetry delivers a fine-grained view of channel power, OSNR, GSNR, Q-factor, and traffic load, which agents use to identify imbalances and underutilized margins [103], [129]. Power-control agents adjust amplifier setpoints and spectral equalization profiles to smooth power across channels, lower unnecessary margins, and reduce energy use while keeping QoT above thresholds [5], [103]. QoT-aware agents can suggest baud-rate or modulation changes for specific services, increasing capacity on high-quality paths and reducing order for paths that approach physical limits [102], [105].

These actions are staged and verified through DTs. Before applying a soft change, the agent submits the proposed adjustment to the DT, which checks how the new power or spectrum configuration affects the level of linear and nonlinear impairment [61], [106]. If the simulated QoT remains within the limits for all affected lightpaths, the

change can proceed, often in small steps, so that additional telemetry confirms the expected outcome [61].

LLMs assist by explaining planned reconfigurations to operators, summarizing the expected impact, and documenting the sequence of changes in plain language [70]. Combined, telemetry, DTs, and agentic AI enable a style of incremental optimization where the network is constantly fine-tuned, rather than reconfigured only during large maintenance windows [70], [78].

E. NETWORK AUTONOMY LEVELS IN TELEMETRY AND AGENTIC AI FRAMEWORKS

The transition toward autonomous operation is commonly described through the six levels defined by the TM Forum [20], [130]. These levels range from fully manual management to end-to-end autonomous behavior, and they help position telemetry and agentic AI within a broader automation roadmap. As autonomy increases, data-driven decision-making becomes more important, and closed-control loops evolve from simple threshold actions to continuous intent-based operation.

At Level 0, the network relies entirely on human intervention. Operators perform configuration, supervision, and recovery actions manually. This mode, while now unusual in large infrastructures, shows the limitations of manual processes such as slow detection of failures and service degradation.

Level 1 introduces basic support functions that handle repetitive or well-defined tasks. These functions follow static rules and fixed workflows and help reduce errors and delays, but operators remain in full control of the lifecycle.

At Level 2, controllers apply predefined policies for a wider range of functions. Telemetry is collected more consistently, enabling early forms of feedback loops. Automation covers selected fulfillment and assurance tasks, though operators still validate changes that affect service stability.

A distinct shift appears at Level 3. Networks begin to adjust their behavior based on real-time observations, analytics, and early AI components. Routine actions such as standard reconfiguration or typical recovery steps require less manual involvement. Telemetry streams become essential for maintaining situational awareness and for triggering updates when conditions change [130].

Level 4 represents a high degree of autonomy where AI-driven analysis and cross-domain orchestration support decisions that are made in near real-time [131]. Operators express high-level intent, and the network translates that intent into concrete actions, evaluates them using models and DTs, and enforces the chosen workflow. Migration to this level requires joining multiple automation layers, aligning operational teams, and integrating advanced capabilities such as machine learning and multi-agent control.

At Level 5, the system manages its own lifecycle. It configures, repairs, optimizes, and protects itself through continuous feedback loops. DTs and agentic AI frameworks

coordinate cross-domain decisions, while humans intervene only in exceptional cases related to policy or compliance. Telemetry at this stage is comprehensive, streaming continuously from all layers and across vendor boundaries.

This hierarchy outlines how telemetry and agentic AI contribute to autonomy. Telemetry supplies accurate, real-time insight that becomes indispensable above Level 1, while agentic AI introduces distributed reasoning that interprets conditions, applies policies, and executes multi-step workflows. Together, these elements support the move from reactive management at intermediate levels to proactive intelligence and self-directed operation at the higher levels. To give a structured view of how these capabilities evolve across the six levels, Table 8 summarizes the key characteristics associated with each stage. The table highlights how telemetry maturity and the use of agentic AI expand as networks progress from manual processes at L0 to fully autonomous behavior at L5.

The progression can also be visualized more clearly through Figure 13, which shows how telemetry functions and agentic AI roles expand across the autonomy scale. The figure illustrates the shift from operator-centric control at the lower levels to multi-agent, intent-driven operation supported by DTs at the higher levels.

F. DATA MODELS FOR TELEMETRY DRIVEN AND AGENTIC AI AUTOMATION

Autonomous behavior requires a consistent way to describe resources, services, policies, and operational state. Data models provide this structure by giving agents, controllers, and DTs a common understanding of the network [131]. They connect raw telemetry with the semantic context needed for intent execution, reasoning, and closed-loop operation. Without these models, automation would be fragmented and limited to isolated workflows tied to specific vendors or domains.

Resource models describe both physical and virtual components of the network. They capture configuration settings, operational limits, relationships between elements, and performance indicators. These models allow autonomous controllers to evaluate capacity, detect conflicts, plan routing or wavelength placement and understand how changes in one part of the network affect the rest of the system [131], [132]. In multi-vendor optical environments, resource modeling languages such as YANG and T-API help normalize information exported by different platforms so that orchestration systems, telemetry collectors and DTs can interpret it consistently [133].

Service models represent how services are defined and what they must deliver. They include functional attributes, expected performance, protection rules, and customer related requirements. These structures allow autonomous systems to understand what is being delivered, what constraints apply, and what level of quality must be maintained. When telemetry shows a deviation from the expected behavior, the service

TABLE 8. Summary of autonomous network levels in telemetry- and agentic AI-driven operation, adapted from the TM Forum framework [130].

Level	Key characteristics
L0 – Manual	No automation. Operators handle configuration, monitoring, troubleshooting, and recovery manually. Telemetry use is limited and often ad hoc.
L1 – Assisted	Basic automation supports individual tasks through fixed scripts and workflows. Telemetry is polled at low frequency for dashboards and threshold alarms. Operators remain responsible for all decisions.
L2 – Partial automation	Centralized controllers apply static policies for selected services. Telemetry becomes more structured and enables simple feedback loops. Portions of fulfillment and assurance workflows are automated, with operators validating key actions.
L3 – Conditional autonomy	Networks adapt to observed conditions using analytics and early AI. Telemetry streams feed real-time monitoring platforms. Closed-control loops manage routine reconfiguration and common recovery patterns. Operators focus on exceptions and on refining policies.
L4 – High autonomy	AI-driven analysis and cross-domain orchestration support near real-time decisions across network domains [131]. Rich telemetry aligns information across layers and vendors. Intent-based interfaces express high-level goals, while DTs and multi-agent systems validate and execute actions. Human involvement is mostly supervisory.
L5 – Full autonomy	Networks operate with self-configuration, self-healing, self-optimization, self-protection, and self-learning. Telemetry spans all lifecycle stages and feeds AI-driven DTs and multi-agent controllers. Most workflows are zero-touch, with human involvement limited to rare exceptions.

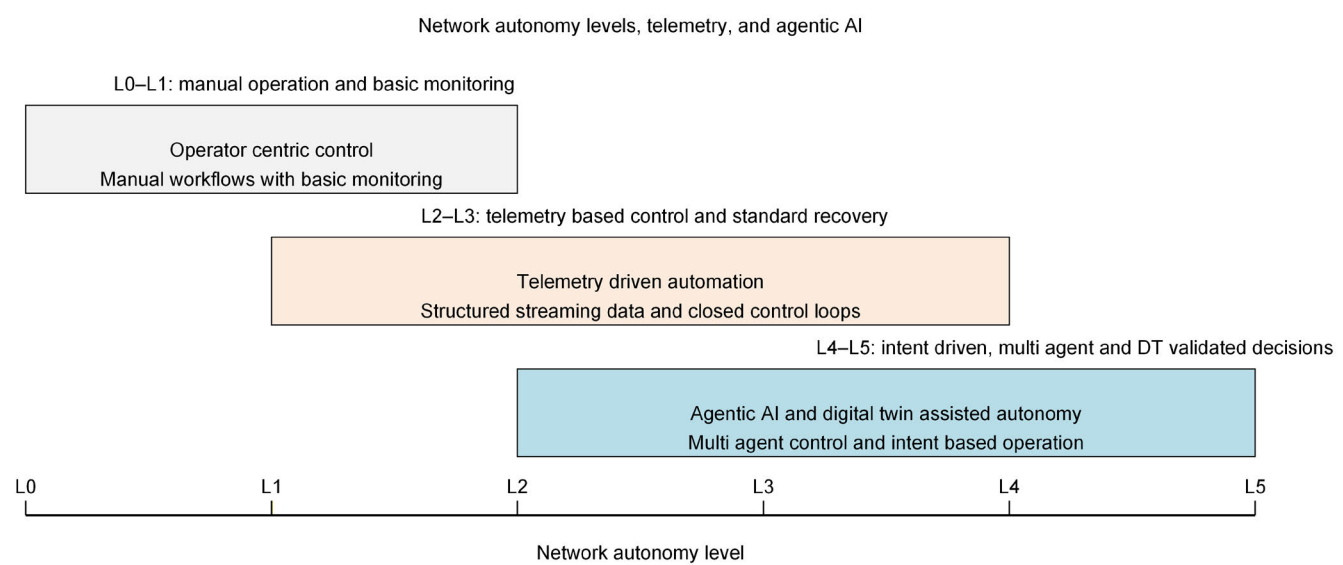


FIGURE 13. Relationship between network autonomy levels, telemetry evolution, and agentic AI functions [20], [130], [131].

model guides the selection of corrective actions and enables agents to coordinate restoration across domains [134].

Intent models express business goals and operational policies in a format that machines can evaluate. They capture desired outcomes, performance objectives, security requirements, and compliance rules. These models are essential for translating high level operator input into concrete actions. Intent frameworks rely on them to check whether a proposed configuration meets policy constraints and whether the actions taken by agents still align with the overall objective expressed by the user or service provider [135].

Operational models organize the information needed for day to day management. They include fault records, performance measurements, configuration and inventory details, security policies, accounting data, and audit trails [131]. These models enable predictive maintenance, automated healing, performance tracking, and security enforcement.

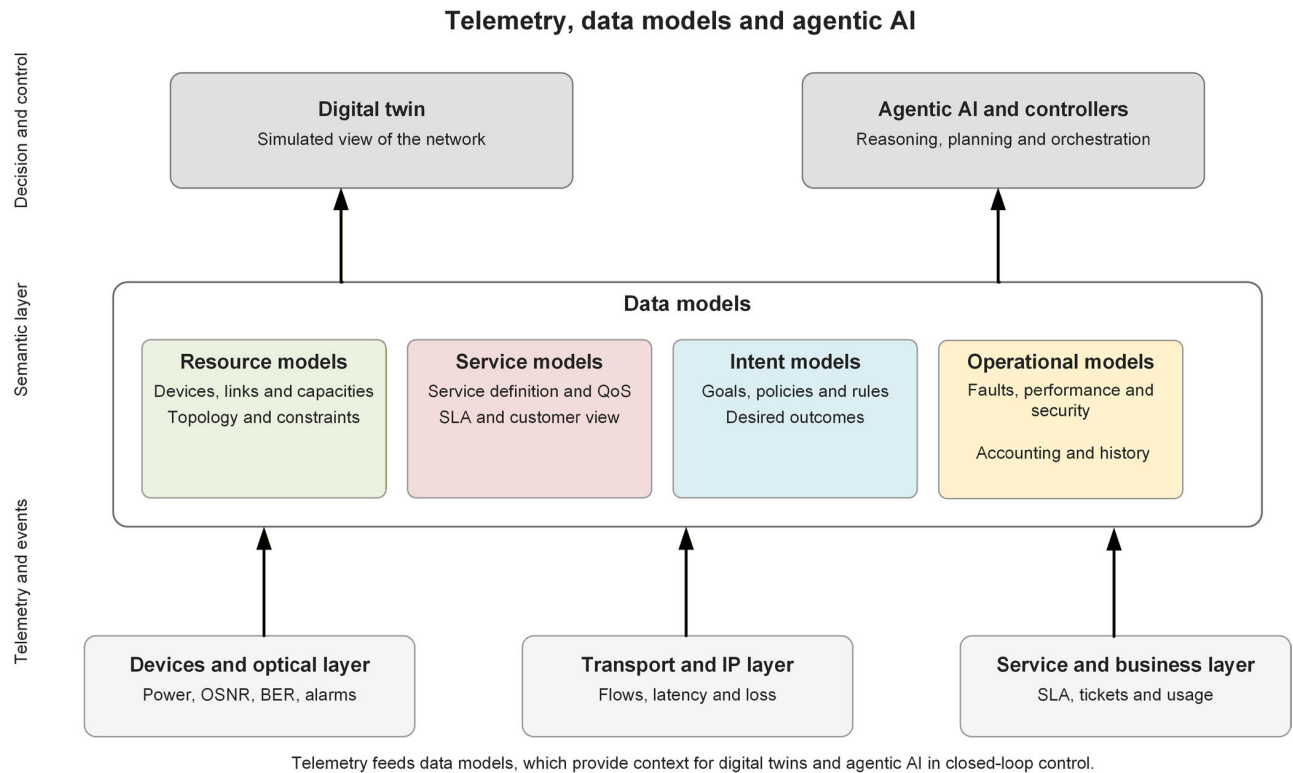
They also provide the data context used by DTs to maintain synchronization with physical networks and to evaluate the expected impact of planned actions before they are executed [136].

Together, these models support the semantic layer that binds telemetry to decision making. They ensure that data coming from heterogeneous equipment and multiple layers can be interpreted without ambiguity. They allow AI agents to reason about options, compare alternatives, and apply changes that satisfy intent and policy. They also allow DTs to build accurate internal states and run safe simulations of potential actions. By structuring the information exchanged between controllers, orchestrators, agents, and customers, these data models form the foundation of coordinated, cross domain, closed-loop automation.

Table 9 summarizes the four main groups of data models that support autonomous operation. The table highlights how

TABLE 9. Summary of data model types that support telemetry driven and agentic AI automation.

Model type	Main purpose	Typical contents	Role in automation
Resource models [131], [132], [133]	Describe physical and virtual network elements and their relationships	Device and link inventory, topology, capacities, constraints, performance indicators	Support placement and routing decisions, enable capacity planning, allow controllers and twins to understand how changes in one element affect the rest of the network
Service models [131], [134]	Describe how a service is assembled and the level of performance it is expected to deliver from the user perspective.	Service functions, QoS targets, protection rules, SLA parameters, customer facing attributes	Guide orchestration and restoration, relate telemetry deviations to specific services, help agents choose recovery actions that preserve agreed quality levels
Intent models [131], [135]	Capture business goals and policy rules in a form that systems can evaluate	Desired outcomes, performance objectives, policy constraints, security and compliance requirements	Translate high level objectives into concrete operations, allow agents to test candidate actions against policy and ensure that closed loops remain aligned with operator intent
Operational models [131], [136]	Organize information used for ongoing management and assurance	Fault records, performance histories, configuration and inventory snapshots, security rules, accounting and audit data	Enable predictive maintenance, automated healing and security enforcement, provide context for DTs and analytics to evaluate risk and impact before changes are applied

**FIGURE 14.** Relationship between telemetry sources, data models and agentic AI functions.

resource, service, intent, and operational models organize different layers of information. These structures give telemetry a clear meaning and allow agents, orchestration systems, and DTs to evaluate conditions, compare options, and trigger coordinated actions across domains.

Figure 14 provides a visual view of how telemetry, data models, and agentic AI interact. Telemetry from multiple layers feeds the semantic structures that describe resources,

services, intent, and operations. These models provide the context necessary for DTs and agents to interpret events, plan changes, and execute closed-loop control.

G. ILLUSTRATIVE IMPLEMENTATION EXAMPLES

The following examples illustrate how an automation framework can be applied in practice. Each case reflects patterns that are already present in reported implementations of

LLM based analysis, DT assisted evaluation, and telemetry driven control. The first example concerns soft failure detection. High rate telemetry based on gRPC and in memory preprocessing was used to stream power levels and OSNR measurements from ROADMs. A simple classifier running at the edge detected deviations from expected patterns, while an analysis agent compared the measurements with historical baselines. When a drift was confirmed, a planning agent evaluated alternative configurations in a DT before any change was applied to the live system. Similar ideas appear in recent deployments that use LLM agents for alarm reduction, root cause analysis, and anomaly detection with high accuracy [87].

A second example involves spectrum reallocation during maintenance windows. The operator used intent based commands to request a temporary reroute. The intent was translated into concrete constraints by an agent that consulted both service and resource models. The DT evaluated several candidate paths and confirmed that one option met QoS and margin requirements. An execution agent then applied the required configuration to the SDN controller, while a monitor agent tracked telemetry to verify that margins remained stable. Frameworks that translate intent into configuration actions and validate them with LLM reasoning and controller feedback show similar behavior [137].

A third example relates to amplifier gain adjustments. Telemetry streams carrying power levels, amplifier states, and alarms were processed by edge agents that summarized the data and forwarded key indicators. When a gain imbalance appeared, a control agent calculated a set of corrective actions that were validated in the DT system. Only after validation were the changes pushed to the network. This avoided unnecessary power shifts and reduced restoration time. Comparable architectures that use DTs to evaluate recovery options before execution have been reported in fault management studies [138].

A fourth scenario illustrates how agentic AI supports network expansion and equipment updates [139]. When new spans, OMS sections, or transponder types are introduced, the system must match planned designs with available hardware and software features. In the multi-agent framework, an agent interprets the operator's intent to extend the network. Another agent queries the DT to confirm that new services can be supported with proper routing, spectrum assignment, and margin. A management agent, equipped with documentation knowledge through retrieval or fine-tuned embeddings, identifies which commercial components match the required specifications, such as amplifier types or updated board modes. This mirrors the experimental result where the documentation-aware agent analyzed manuals, interpreted logs [140], and selected components across different versions of software and hardware. Telemetry then validates the behavior after the upgrade, ensuring consistency between predicted and observed performance. This example shows

how agentic AI integrates intent interpretation, product knowledge, simulation, and live monitoring into a coordinated workflow.

In addition to these four scenarios, several research groups have reported similar real-world demonstrations that combine telemetry, DTs, LLM-based reasoning, and agent-style workflows. Table 10 summarizes the examples described above and includes other published implementations that reflect the same principles [141], [142], [143], [144], [145], [146], [147]. Together, they offer a broader view of how current systems already use measurement data, modeling, and AI agents to support operation and decision making in optical networks.

VIII. CHALLENGES

Even with recent advances in telemetry, LLMs, DTs, and agentic AI, the transition to highly automated optical networks still faces several technical and organizational obstacles. These obstacles affect how data are collected and shared, how multi-vendor systems interoperate, how real-time decisions are taken, and how autonomous actions are governed under operator policies and regulatory rules [5], [6], [78].

A. DATA QUALITY AND CONSISTENCY

Most of the automation functions in this work depend on accurate models, and these models in turn depend on high quality data. In practice, telemetry and log data in optical networks are often incomplete, noisy, or unevenly sampled across devices and domains [78]. Multi-vendor deployments add further variability in naming, measurement units, and sampling strategies, making end-to-end correlation difficult.

For ML and LLM based components, these issues limit prediction accuracy and increase the risk of biased decisions, especially when training data do not reflect the full variety of operating conditions. Privacy and commercial constraints also restrict the sharing of raw data between operators or domains. Techniques such as data cleaning, normalization, and harmonization are mandatory, and approaches such as federated learning can help when models need to be trained in administrative domains without moving the underlying data [7]. Without consistent and curated datasets, even advanced agent workflows will struggle to generalize beyond narrow scenarios.

B. INTEROPERABILITY BETWEEN VENDORS

Optical networks combine equipment, controllers, and management systems from many vendors, leading to heterogeneous operational models and limited interoperability [8]. Proprietary device models, closed interfaces, and inconsistent control protocols further complicate integration efforts [8]. In such environments, agents and LLM-based tools must operate in different telemetry schemas, configuration formats, and command sets, increasing integration effort and slowing the automation deployment [8], [148].

TABLE 10. Overview of practical demonstrations that combine LLMs, DTs, and automation in optical networks.

Title	Focus area	Approach	Main results (with metrics)	Remarks
AlarmGPT [87]	Alarm stream reduction, correlation and failure prediction	ChatGPT with embedded alarm knowledge and vector retrieval	Compression of 30–85 percent, processing near 4 s, and 88 percent fault-prediction accuracy. The workflow resembles an agentic pipeline that retrieves, interprets and reasons over telemetry-derived alarms	Based on offline logs; depends on expert-built knowledge bases.
Alarm analysis [70]	Alarm ranking, grouping and resolution support	GPT-4 with LangChain, manuals, expert rules and BERT correlation	Evaluated on 2500 OTN alarms. Uses alarm telemetry and an LLM-guided sequence of correlation steps, reaching 96.8 percent accuracy. The workflow reflects an agentic chain coordinating rules and tools	Strong dependence on manuals and rules; limited portability across vendors.
LLM-based optical network log analysis [140]	Log parsing, anomaly detection and event classification	Instruction-tuned LLaMA2 (7B) with tool calls	Processes field log telemetry. Parses logs, detects anomalies and explains root causes with high accuracy (precision 0.97, recall 0.98, F1 0.97). Parsing reaches EMR 0.92, PA 0.93, RA 0.95. Workflow mirrors an agentic chain driven by telemetry content	Trained on curated logs; generalization depends on log diversity.
Open LLM pipeline for SDON configuration [148]	Intent translation into configuration tasks	Open-source pipeline with JSON schema, NETCONF/YANG and multi-step planning	Over 80 percent accuracy for converting intent into SDON commands. Pipeline follows an agentic pattern: intent interpretation, schema generation and controller feedback. Supports OSNR-monitoring intents tied to telemetry	Prototype limited to 3 nodes and small intent set.
LLM-driven AI Agent in SDN Controller [141]	Intent-based control and performance tuning	LLM agent within SDN controller using DT simulations and YANG models	Uses optical telemetry inside the DT to predict performance and compute optimized actions. Interprets intent, produces NETCONF commands and validates feedback. Achieved sub-2 s execution and 2.33 dB GSNR gain	Small-scale prototype relying on curated YANG; limited large-network validation.
IntentLLM [142]	Slice-intent handling in TeraFlowSDN	LLM chatbot interacting with SDN APIs and controller telemetry	Manages slice creation and queries by reading controller-side telemetry on slice attributes. Agentic flow: interpret request, validate KPIs and trigger slice actions. Demonstrated end-to-end provisioning	Proof-of-concept; limited metrics and no large-scale tests.
OptiComm-GPT [138]	Simulation and DSP workflow assistance	LLM with LangChain directing SSFM, DeepONet and DSP modules	Consumes OSNR traces, IQ constellations and DSP outputs. Acts as an agent selecting analysis steps and validating results. SSFM runtime near 28.3 s; accuracy improved from 31 to 100 percent; achieves 97 percent detection and 95 percent DSP verification	High SSFM cost; offline and simulation-dependent.
Local AI-Agents [139]	Lifecycle automation (design, maintenance, upgrades)	Locally fine-tuned LLMs with DTs on-premise	Reaches 100 percent natural-language to API translation; 83 percent model-size reduction; 1 dB SNR gain after power re-equalization; low hardware needs	Requires domain data; accuracy depends on fine-tuning quality.
DTs with Cloud-Native SDN [143]	Network supervision and optimization with DTs	Cloud-native SDN controller with virtualized DTs and real-time telemetry	Uses TAPI and Kafka telemetry to align DT with physical network. DT deploys in 2.38 s and removes in 0.85 s. Telemetry updates reach 0.009 s for link faults, supporting agent-like closed-loop validation	High virtualization overhead; scaling and DT accuracy require further study.
TAPI-based telemetry [144]	Telemetry streaming and closed-loop anomaly handling	TAPI telemetry with gRPC, Kafka and ML diagnosis	Provides continuous KPIs via unified TAPI model. gRPC beats REST for real-time updates. Telemetry drives ML detection about 100 time units before disruption and triggers TeraFlowSDN reconfiguration in an agent-like loop	Manual link setup and vendor-specific models complicate multi-domain integration.
DRL-based spectrum management [145]	Defragmentation in elastic optical networks	Deep RL agent via T-API DT system	Uses T-API telemetry to query spectrum state, choose reallocation steps and validate in the twin. Reduces blocking vs heuristics and supports real-time DT evaluation	Training relies on simulations; adaptability to production remains limited.
Measurement-informed models and DT [146]	Survey of DT methods for optical systems	Hybrid physics and data-driven DT architectures	Highlights role of OSNR, BER and spectral measurements for aligning DTs with the real network. Telemetry-informed DTs support future agentic controllers and optimization tasks	Survey only; no experimental or real-time pipeline.
Building a DT [64]	Tutorial on DT construction	Hybrid ML and physics-based modeling	Shows that DT accuracy depends on continuous telemetry (OSNR, power, impairments). DT lifecycle forms the predictive core supporting agentic decision loops	Conceptual guidance; no full closed-loop implementation.

TABLE 10. (Continued). Overview of practical demonstrations that combine LLMs, DTs, and automation in optical networks.

Improved QoT estimations [147]	QoT enhancement in disaggregated networks	Refined power-measurement calibration	Uses live telemetry on channel power, OSNR and load to recalibrate QoT parameters. Reduces prediction error from 0.6337 to 0.2377 dB and strengthens DT support for agentic control	Single-network validation; requires periodic retraining.
QoT estimation via GPT-4 [70]	Autonomous QoT prediction and optimization	GPT-4 with LangChain, GN-model parameters and optimization	Consumes span count, launch power and modulation telemetry to predict QoT with 95 percent accuracy across 2400 cases. Follows agentic loop: interpret measurements, estimate GSNR and suggest settings	Compute-heavy; depends on domain libraries and lab tests.
OCATA DT [118]	Time-domain DT for QoT and soft-failure detection	DNNs applied to IQ-level telemetry	Uses IQ traces and BER indicators to update DT and detect degradations. Achieves QoT prediction and soft-failure detection with BER error under 5 percent. Mirrors an agentic loop of telemetry ingestion and model update	Experimental testbed only; scaling to full networks remains open.

Open data models and APIs help reduce this friction, but they are not yet universal. Even when standard models are available, subtle differences in implementation can cause inconsistent behavior. As a result, multi-vendor interoperability remains a central challenge for any telemetry driven and agentic AI-based architecture. Achieving predictable behavior in heterogeneous domains still requires significant engineering effort and close coordination with suppliers [8].

C. IMPACT ON REAL-TIME TASKS

Many of the target tasks for telemetry and agentic AI are time sensitive, including QoT monitoring, protection switching, and fast restoration. These tasks have strict latency budgets, especially in high capacity backbones and DCI scenarios [5], [78]. At the same time, LLMs and complex agent workflows have a nontrivial computational footprint. Autoregressive generation, large parameter counts, and long contexts can all increase inference latency and memory use.

To use LLMs and agentic workflows in such settings, both model and system level optimizations are needed. Techniques such as pruning, quantization, distillation, and low rank adaptation reduce model size and speed up inference [9], [10], [11]. System level methods such as KV cache optimizations, sparse attention, batch aware scheduling, and speculative decoding further lower latency and resource use [39]. Even with these techniques, designers must still decide which decisions can be delegated to large models and which ones must remain with lightweight deterministic logic to meet strict real-time constraints.

D. RELIABILITY AND SAFETY OF AUTONOMOUS ACTIONS

As control moves from static scripts to autonomous agents, the risk associated with incorrect actions increases. Physical-layer behavior is complex, models are imperfect, and LLMs can produce outputs that are linguistically convincing but technically incorrect [2], [6], [78]. Misjudged QoT, incorrect fault hypotheses, or hallucinated configuration commands can lead to service degradation or outages if not filtered.

To keep autonomous actions safe, several safeguards are required. DTs and analytical QoT models should validate

changes before they reach the live network [12], [64]. RAG based grounding, domain fine tuning, and continuous learning reduce the risk of hallucinations, while error prediction modules can flag responses that are likely unreliable [2], [149]. Human in the loop checks remain essential for high impact operations, both to satisfy regulatory expectations and to build operator trust. Overall, automation must be designed as a supervised control layer, not as an unconstrained replacement for human judgment.

E. COMPUTATIONAL COST OF AGENT WORKFLOWS

Agentic AI pipelines depend on continuous telemetry ingestion, streaming infrastructure, ML models, and LLM inference. Running these components at scale, especially with large models and dense traffic, can place significant demands on compute, memory, and storage resources [5]. For operators with limited infrastructure, providing the required accelerators and high performance databases can be financially difficult.

When commercial LLM APIs are used, token-based pricing introduces another cost dimension. Tasks such as alarm correlation, continuous summarization, or configuration generation can involve thousands of prompts per day, causing costs to grow rapidly [150]. To keep deployments viable, designers must combine prompt compression, result caching, parameter-efficient fine-tuning (e.g., LoRA or QLoRA), and quantization, along with optimized inference kernels such as FlashAttention (an optimized attention algorithm that reduces memory access and latency by computing attention in a fused, block-wise manner directly on GPU memory) and fused operators (kernels that merge multiple neural-network operations into a single optimized execution to reduce memory movement and latency) [39], [151]. In some cases, hosting domain-tuned open-source models locally or in hybrid cloud-edge setups helps reduce recurring API costs while keeping sensitive telemetry on premises.

F. TRACEABILITY AND TRANSPARENCY

Regulated networks must provide clear explanations for actions that affect service, security, or customer traffic.

Many ML and LLM models are opaque, which complicates accountability and regulatory compliance [6]. If an autonomous agent misconfigures part of the network, operators need to understand how the decision was made, what data were used, and which models were involved.

To address this, automation frameworks must include strong observability features: detailed logs of agent decisions, traceable links between telemetry inputs and actions, and mechanisms to replay or audit decision paths. Explainable AI methods and structured reporting from LLMs can help by summarizing reasons and evidence in human readable form rather than only numerical outputs. DTs also support transparency by documenting which simulated scenarios were used to approve a given action [78]. Without these mechanisms, trust in autonomous operation will remain limited.

G. ALIGNMENT WITH OPERATOR POLICIES

Finally, even the best technical solution must align with operator policies, culture, and strategy. Automation changes existing workflows, alters responsibility boundaries, and raises concerns about job roles and required skills [6], [152]. Some teams may prioritize short term stability over long term gains, slowing the adoption of new tools. Budget constraints and uncertain return on investment also limit large upgrades to telemetry, compute platforms, and software stacks [153].

A practical path forward requires gradual deployment, clear governance, and explicit policy frameworks for what agents are allowed to do. DT based validation helps maintain human oversight and reduce perceived risk, while LLM based interfaces can make advanced capabilities accessible to staff without deep programming experience [78]. Training and reskilling programs are also needed so that operations teams can understand, supervise, and tune automation instead of viewing it as a black box. Aligning agentic AI and telemetry projects with operational goals and regulatory constraints is as important as solving the underlying technical problems.

IX. FUTURE DIRECTIONS

The combination of telemetry, DTs, LLMs, and agentic AI points to a next step in optical network automation in which control is carried out by distributed, cooperating software entities that operate close to the data and interact through standard, multi-vendor interfaces. This section outlines key directions that can shape such architectures and guide future research and field trials.

A. DYNAMIC ECOSYSTEMS OF AGENTS

Future automation frameworks are likely to move from single orchestrators to ecosystems of agents that coordinate through event driven fabrics and shared knowledge rather than static APIs [15], [92]. In this view, each agent carries a clear role and scope, but the overall behavior emerges from cooperation. Distributed agents will blend LLM based reasoning with lightweight models and rule engines, operating on streaming telemetry and stored context. Edge AI and hybrid edge cloud

designs will be important so that time critical tasks (for example, local QoT checks or fast restoration) can stay near the devices, while heavy planning or learning runs in more powerful environments [86]. Advances in efficient LLM architectures, such as Jet NemoTron style blocks that combine optimized attention with hardware aware tuning, show that future models can reach much higher throughput and lower energy use while keeping acceptable accuracy [154]. These models are natural candidates for on prem or operator hosted assistants that perform intent translation, telemetry summarization, or plan generation inside agent workflows. Over time, these ecosystems may evolve into marketplaces of specialized agents provided by different vendors or internal teams, all connected through shared streaming middleware and common safety and governance layers [89], [100].

B. COMPRESSION OF TELEMETRY STREAMS

As networks grow and more signals are monitored, raw telemetry rates will exceed what is practical to transport and store. This pushes architectures toward stronger compression, summarization, and event based reporting [3], [14], [18]. Future telemetry planes will likely combine several complementary techniques that reduce data volume while keeping the information needed for control. First, in the network, preprocessing will run directly on devices and agents. This can remove samples that carry little value, group short time windows into summaries, or report only the change since the previous update, reducing traffic without hiding important trends. Second, learned encoders and Joint Embedding Predictive Architecture (JEPA) style models can convert rich signals such as spectra or constellations into compact latent vectors [155]. These vectors capture the essential state of a channel while being far smaller than the raw measurements. Third, semantic aware telemetry will attach simple labels to each measurement, such as the related service, intent, or risk level. These tags help agents and LLMs decide which signals deserve attention and which can be processed with lower priority.

These techniques must preserve the information needed for QoT estimation, failure management, and capacity planning, while reducing bandwidth and storage. Designing compression schemes that remain interpretable, compatible with standards, and resilient to anomalies is an open problem. It ties directly to the cost and sustainability concerns around large AI workloads and continuous streaming [5].

C. DISTRIBUTED DIGITAL TWINS

DT solutions for optical networks are often built as centralized replicas. As networks expand and telemetry rates increase, this design becomes harder to sustain. As discussed in Section III, high-rate streaming, distributed preprocessing, and heterogeneous data sources place substantial load on any single processing point. A natural evolution is the move toward distributed DTs, where each domain maintains a partial model and synchronizes with others through shared

telemetry and control interfaces [61], [64]. Similar trends appear in other fields, including resource-constrained DTs for distributed health monitoring [156].

Local twins positioned near ROADMs, metro clusters, or edge nodes host reduced-detail models that provide fast assessments for short-horizon decisions. These instances rely on the edge processing techniques described in Sections III-G–III.I, allowing them to filter raw measurements, detect anomalies, and compress information before forwarding it to higher layers. Core or inter-domain twins maintain high-fidelity physics-based or data-driven models for design validation, impairment analysis, and long-term capacity studies, consistent with the roles introduced in Section IV (Digital Twins).

Coherence between these local and central twins depends on continuous synchronization. Local instances process full-rate telemetry but forward only segments that deviate from expected behavior or fall in regions of low model confidence, following the selective transmission principles outlined in Section III.I and also used in distributed DT frameworks for aerospace systems [156]. Central twins merge these updates with multi-domain histories, retrain high-detail models and propagate refined parameters back to the edge. This creates a bidirectional loop similar to the coordination cycle described in Section IV-C (Telemetry as a Source of Updates).

Multiple modeling styles can coexist within this hierarchy. Lightweight ML surrogates or reduced-order models are suited for embedded environments near optical nodes, while high-fidelity or physics-informed ML models [157] operate in the core. Quantum-assisted solvers may be used for specific large-scale tasks such as wideband spectrum allocation or amplifier planning [158]. The main challenge is defining clear boundaries and responsibilities so each twin answers its part of the global problem and exposes results through well-defined interfaces, consistent with the agent coordination mechanisms in Section VI (Agentic AI).

A distributed DT structure minimizes unnecessary data movement, reduces latency for operational functions and supports deeper computational analysis at upper layers. When combined with the telemetry foundations of Section III and the agentic AI mechanisms of Section VI, distributed twins form a scalable path for future optical network automation and contribute to the long-term research goals identified in this section.

D. SAFEGUARDS FOR AUTONOMOUS ACTIONS

As autonomous functions expand to include provisioning, restoration, and parameter tuning, safety mechanisms must be treated as core architectural elements rather than optional add-ons. Moving toward higher levels of automation requires clear methods to express operational limits, enforce them during decision making, and audit outcomes to ensure that agents remain predictable and verifiable. This aligns with broader AI guidance, such as the IEEE initiative on ethically

aligned design, which stresses that autonomy must be paired with mechanisms that protect users and infrastructure [159].

Several complementary technologies can support these safeguards. Causal AI and knowledge graphs can capture relationships between faults, impairments, and services, allowing agents to reason about cause and effect rather than only statistical patterns [160]. Reinforcement learning and DRL can be constrained by such causal structures and by DT based checks so that learned policies remain inside defined safety envelopes and service constraints [161]. Memory augmented systems can retain past incidents and decisions, improving long term consistency and helping agents avoid repeated mistakes [162].

Supervision must also be distributed across agents rather than centralized in a single controller. Work on multi-agent self-healing systems demonstrates that agents can cross-validate proposed actions, flag inconsistencies, and isolate problematic decisions while preserving global safety rules [163]. These studies show that structured communication, cross checking, and rule-driven restoration logic are effective safeguards against unsafe or premature actions in autonomous environments.

Explainability plays an equally important role. Operators and governance teams need human-readable accounts of why an action was proposed, which telemetry signals were considered, and how DT simulations influenced the decision [164]. This level of transparency is also emphasized in ethical design standards, which highlight traceability as essential for responsible deployment of autonomous systems [159].

Overall, reliable safeguards will require clear patterns for distributing responsibility across LLMs, classical ML models, DTs, and rule engines. High-level reasoning and interpretation can rely on the flexibility of LLMs, but final actuation should be anchored in validated models, explicit policies, and cross-agent review [2], [6]. A combination of structured rules, distributed oversight, and policy-constrained learning is likely to form the backbone of future safety design in agentic optical networks.

E. STANDARDIZATION FOR MULTI VENDOR TELEMETRY

Standardized telemetry models and interfaces are essential to make agentic AI and DTs practical in multi vendor networks. While current work on model driven management and transport APIs has made clear progress, many operators still face fragmented schemas and proprietary extensions [8], [18]. Future efforts will likely deepen and extend existing frameworks so that telemetry definitions cover a wider range of optical parameters, failure signatures, and DT related metrics, while remaining vendor neutral. This includes convergence between YANG based models, T-API like abstractions, and streaming formats suitable for Kafka, MQTT, and similar platforms [3], [18]. Federated learning and privacy preserving schemes will influence how telemetry is shared between domains or operators, allowing joint model training and benchmarking without exposing raw data [7].

TABLE 11. Summary of future research directions for telemetry-, DT-, and agentic-AI-driven optical networks.

Direction	Core idea	Key enabling elements	Refs.
Dynamic ecosystems of agents	Shift from single controllers to cooperating software agents that act on streaming telemetry and shared context. Edge agents handle fast tasks while heavier planning runs in cloud environments.	Event driven fabrics, LLM reasoning, lightweight local models, edge AI, hybrid cloud–edge execution.	[15], [92], [86], [154]
Compression of telemetry streams	Future networks require filtered, compressed and semantic telemetry to handle rising data volumes without losing information needed for QoT, failure detection or planning.	In device preprocessing, delta reporting, semantic tags for intent or service context.	[3], [18], [14]
Distributed DTs	DTs evolve from centralized replicas to distributed models per domain or region, synchronized through shared telemetry. Models mix physics guidance with learned behavior.	Regional twins, physics informed ML, wideband DT models, quantum assisted solvers for complex optimization.	[61], [64], [157], [158]
Safeguards for autonomous actions	Autonomous agents require strict safety layers, causal reasoning and clear governance to justify and constrain decisions related to provisioning, restoration and tuning.	Causal AI, knowledge graphs, constrained DRL, XAI, memory augmented systems, DT based validation.	[5], [6], [160], [161], [162]
Standardization for multi vendor telemetry	Broader telemetry standards are needed so AI agents and DTs can operate across domains without vendor specific extensions.	Unified YANG models, TAPI abstractions, Kafka/MQTT formats, federated learning, privacy preserving sharing.	[18], [8], [3], [7]
Integration with emerging optical fabrics	Next generation optical fabrics require richer telemetry, faster control loops and energy aware automation aligned with new modulation, wideband amplification and IP optical integration.	LLM based intent handling, edge inference, real-time control hooks, spectrum–power tuning, sustainability aware optimization.	[5], [154]

Blockchain style ledgers may be explored to provide integrity and non repudiation for critical telemetry and control traces, especially where multiple operators or regulatory bodies are involved [61]. Standard bodies and open communities will have an important role in making sure that these models are not only technically sound but also implementable at scale by equipment vendors and software suppliers.

F. INTEGRATION WITH EMERGING OPTICAL FABRICS

Finally, future work must align telemetry and agentic AI with the evolution of optical fabrics themselves. Higher baud rates, new modulation formats, ultra wideband amplification, and tighter coupling between IP and optical layers change both the metrics that matter and the time scales at which control must act [5]. AI ready optical fabrics will expose richer telemetry at line card and slice level, provide programmable knobs for spectrum, modulation, and power, and support fast reconfiguration paths that agents can use in soft and hard recovery scenarios. DTs and QoT models will have to adapt to wider spectra and denser channel interactions, while resource management agents must handle more frequent and finer grained adjustments. Efficient LLM designs and edge level inference will be needed to keep intent processing, troubleshooting assistance, and documentation generation close to these fabrics without adding excessive latency or cost [154]. At the same time, sustainability pressures will push designers to couple automation strategies with energy aware modulation, amplifier control, and capacity planning so that performance gains do not lead to uncontrolled power growth [5].

A set of emerging directions indicates how optical networks can evolve toward greater autonomy and clarity.

These include multi-agent ecosystems, compressed and semantic telemetry, distributed DTs, stronger safeguards for automated actions, common telemetry standards, and closer integration with new optical fabrics. Table 11 summarizes these directions and links each one to enabling technologies and representative references.

X. COMMON TRENDS IN TELEMETRY, AGENTIC AI, AND AUTOMATION

A clear set of themes emerges when recent work on telemetry, agentic AI, intent based management, and autonomous control is viewed together. The literature indicates a strong convergence toward architectures that fuse real-time telemetry, distributed intelligence, and intent driven automation as the foundation for next generation optical networks.

One recurring trend is the adoption of standardized and streaming telemetry as the primary source of operational truth. Works such as [13], [14], [20], [22], [26], [144], and [165] show how models based on YANG, OpenConfig, OpenROADM, and T-API provide uniform data structures, while high rate gRPC, Kafka, MQTT, and Redis channels deliver fine grain KPIs with low latency. These studies highlight the shift from slow, controller centric polling to continuous streaming that enables early detection of soft failures, multi layer visibility, and rapid reconfiguration across domains.

A second trend is the move from centralized automation to architectures built on distributed and local agents. Experiments reported in [14], [13], [18], [20], [26], and [139] show that placing lightweight agents near ROADMs, amplifiers, and transponders reduces control delay, improves resilience, and supports closed loop reactions. Local agents can filter

telemetry, run small ML models, and apply local corrections without waiting for remote controllers. This pattern aligns with the broader evolution toward low latency, multi-agent automation in optical transport systems.

Intent driven and LLM enabled interaction forms a third common direction. References [16], [18], [20], [70], [131], [135], and [148] demonstrate that intents, natural language interfaces, and LLM based reasoning are becoming practical mechanisms for expressing goals, generating actions, validating plans, and coordinating tasks across heterogeneous systems. These works show LLMs translating user intent into structured steps, interpreting vendor specific configurations, automating workflows, and orchestrating smaller ML components.

A fourth trend is the tight coupling of telemetry with ML and automated control loops. Studies such as [14], [22], [26], [34], [139], [144], and [165] integrate streaming telemetry with anomaly detection, QoT estimation, traffic driven reconfiguration, and distributed monitoring. They demonstrate that continuous KPI streams can trigger local or global responses, support hybrid ML pipelines, and maintain service performance under dynamic conditions. Recent results, such as those in [34], also reveal clear trade offs between update rates, cache strategies, sparsity, and the accuracy of unsupervised anomaly detection.

Across these works, several challenges remain visible. References [18], [20], [34], [70], [131], [16], and [148] note concerns related to hallucinations in LLM outputs, data quality limitations, sparse telemetry reconstruction, hyperparameter tuning, and the need for consistent evaluation of autonomous behaviors. Organizational obstacles and interoperability constraints also appear in discussions on intent based systems and autonomous networks.

Overall, the literature shows a field moving toward telemetry centric, agentic, and intent driven automation. Standardized data models, pervasive streaming telemetry, distributed agents, and LLM based interfaces are forming a layered architecture where sensing, reasoning, and action are tightly integrated. These common trends provide a coherent foundation for future frameworks.

XI. CONCLUSION

Telemetry remains the primary source of truth for automation, giving a continuous and coherent view of physical impairments and higher layer conditions. Without this data stream, no AI model can act with accuracy. On top of this sensory layer, agentic AI provides the operational logic needed to evaluate conditions, plan actions, and coordinate tasks between distributed systems. DTs reinforce this structure by allowing routing changes, power adjustments, or recovery steps to be tested before reaching the live network, reducing risk and strengthening confidence in autonomous behavior. LLMs complement these components by translating operator intent into structured tasks, refining alarms, summarizing telemetry patterns, and improving communication between humans and automated systems. Although LLMs

introduce latency, energy consumption, and the possibility of inaccurate output, careful grounding, lightweight fine tuning, and human oversight help mitigate these concerns. Together, these layers enable structured automation tasks such as QoT monitoring, resource optimization, and failure management. Telemetry provides raw observations, agentic AI drives the decision process, DTs validate proposed actions, and LLMs help interpret and document the results. This combination supports the movement toward real-time, intent driven operation across multi vendor optical infrastructures.

Important challenges remain. Data quality varies across vendors, and real-time workloads demand efficient processing pipelines. Automated actions must also remain auditable and safe. Interoperability, model transparency, and alignment with operator policies are essential for practical implementation. Sustainability pressures add further constraints, requiring energy aware automation, lighter models, and efficient telemetry processing. Standardization will be central to addressing these issues. Multi vendor environments still face fragmented models and proprietary extensions. Future progress depends on expanding vendor neutral schemas, aligning YANG based abstractions with T-API style interfaces, and adopting streaming formats suited for platforms such as Kafka or MQTT. Privacy preserving learning and audit mechanisms may influence how telemetry is shared across domains, and integrity guarantees may become important when multiple operators or regulators rely on shared data and automated actions.

These developments must also be aligned with emerging optical fabrics. Higher baud rates, wider optical bands, new modulation formats, and tighter IP–optical integration change both the metrics that matter and the timescales at which control operates. Agents will need access to richer telemetry at the line card and slice level, faster reconfiguration paths, and more frequent adjustments. Efficient LLM designs and edge level inference will help keep intent processing and troubleshooting close to the optical layer without adding unnecessary delay or cost. At the same time, energy-aware modulation, amplifier control, and capacity planning will be needed to balance performance gains with sustainability objectives.

Overall, the convergence of continuous telemetry, distributed agents, digital replicas, and language based reasoning points toward long term progress in self-management optical systems. These technologies allow networks to observe, interpret, and adapt while remaining accountable and aligned with operator intent. As research advances and standards mature, telemetry-driven agentic automation provides a clear path to more resilient, efficient, and autonomous optical infrastructures.

REFERENCES

- [1] R. Gu, Z. Yang, and Y. Ji, "Machine learning for intelligent optical networks: A comprehensive survey," *J. Netw. Comput. Appl.*, vol. 157, May 2020, Art. no. 102576, doi: 10.1016/j.jnca.2020.102576.

- [2] S. Cruzes, "Revolutionizing optical networks: The integration and impact of large language models," *Opt. Switching Netw.*, vol. 57, Oct. 2025, Art. no. 100812, doi: [10.1016/j.osn.2025.100812](https://doi.org/10.1016/j.osn.2025.100812). [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1573427725000190>
- [3] L. Velasco, P. González, and M. Ruiz, "Distributed intelligence for pervasive optical network telemetry," *J. Opt. Commun. Netw.*, vol. 15, no. 9, pp. 676–686, 2023, doi: [10.1364/jocn.493347](https://doi.org/10.1364/jocn.493347).
- [4] J. W. Nevin, S. Nallaperuma, N. A. Shevchenko, X. Li, M. S. Faruk, and S. J. Savory, "Machine learning for optical fiber communication systems: An introduction and overview," *APL Photon.*, vol. 6, no. 12, Dec. 2021, Art. no. 121101, doi: [10.1063/5.0070838](https://doi.org/10.1063/5.0070838).
- [5] S. Cruzes, "Optical networks automation overview: A survey," *J. Sensors, Netw. Data Commun.*, vol. 3, no. 1, pp. 144–162, 2023.
- [6] F. N. Khan, "Non-technological barriers: The last frontier towards AI-powered intelligent optical networks," *Nature Commun.*, vol. 15, no. 1, p. 5995, Jul. 2024, doi: [10.1038/s41467-024-50307-y](https://doi.org/10.1038/s41467-024-50307-y).
- [7] M. Ibrahim, F. Temiz, F. Musumeci, and M. Tornatore, "Vertical federated learning for failure localization in partially disaggregated optical networks," in *Proc. IEEE 25th Int. Conf. High Perform. Switching Routing (HPSR)*, Jul. 2024, pp. 1–6, doi: [10.1109/HPSR62440.2024.10635921](https://doi.org/10.1109/HPSR62440.2024.10635921).
- [8] E. Le Rouzic, O. Renais, J. Meuric, T. Marcot, C. Betoule, G. Thouenon, A. Triki, M. Laye, N. Pelloquin, Y. Lagadec, E. Delfour, M. Ermel, J. Dost, and S. Turk, "Operator view on optical transport network automation in a multi-vendor context [invited]," *J. Opt. Commun. Netw.*, vol. 14, no. 6, pp. C11–C22, Jun. 2022, doi: [10.1364/JOCN.450008](https://doi.org/10.1364/JOCN.450008).
- [9] D. Wu, X. Wang, Y. Qiao, Z. Wang, J. Jiang, S. Cui, and F. Wang, "NetLLM: Adapting large language models for networking," in *Proc. ACM SIGCOMM Conf.*, Aug. 2024, pp. 661–678, doi: [10.1145/3651890.3672268](https://doi.org/10.1145/3651890.3672268).
- [10] X.-Y. Fu, M. Tahmid Rahman Laskar, E. Khasanova, C. Chen, and S. Bhushan, "Tiny titans: Can smaller large language models punch above their weight in the real world for meeting summarization?" 2024, *arXiv:2402.00841*.
- [11] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding," 2015, *arXiv:1510.00149*.
- [12] N. Andriolli, A. Giorgetti, P. Castoldi, G. Cecchetti, I. Cerutti, N. Sambo, A. Sgambelluri, L. Valcarenghi, F. Cugini, B. Martini, and F. Paolucci, "Optical networks management and control: A review and recent challenges," *Opt. Switching Netw.*, vol. 44, May 2022, Art. no. 100652, doi: [10.1016/j.osn.2021.100652](https://doi.org/10.1016/j.osn.2021.100652).
- [13] A. Sgambelluri, A. Pacini, F. Paolucci, P. Castoldi, and L. Valcarenghi, "Reliable and scalable kafka-based framework for optical network telemetry," *J. Opt. Commun. Netw.*, vol. 13, no. 10, pp. E42–E52, Oct. 2021, doi: [10.1364/JOCN.424639](https://doi.org/10.1364/JOCN.424639).
- [14] S. Shen, J. Han, K. Bardhi, H. Li, R. Yang, Y. Teng, V. Yokar, S. Yan, and D. Simeonidou, "Unified monitoring and telemetry platform supporting network intelligence in optical networks," *J. Opt. Commun. Netw.*, vol. 17, no. 2, pp. 139–151, 2025, doi: [10.1364/jocn.538552](https://doi.org/10.1364/jocn.538552).
- [15] R. Sapkota, K. I. Roumeliotis, and M. Karkee, "AI agents vs. Agentic AI: A conceptual taxonomy, applications and challenges," *Inf. Fusion*, vol. 126, Feb. 2026, Art. no. 103599, doi: [10.1016/j.inffus.2025.103599](https://doi.org/10.1016/j.inffus.2025.103599).
- [16] A. Bandi, B. Kongari, R. Naguru, S. Pasnoor, and S. V. Vilipala, "The rise of future AI: A review of definitions, frameworks, architectures, applications, evaluation metrics, and challenges," *Future Internet*, vol. 17, no. 9, p. 404, Sep. 2025, doi: [10.3390/fi17090404](https://doi.org/10.3390/fi17090404). [Online]. Available: <https://www.mdpi.com/1999-5903/17/9/404>
- [17] H. Zhuang, Y. Zhao, X. Yu, Y. Li, Y. Wang, and J. Zhang, "Machine-Learning-based alarm prediction with GANs-based self-optimizing data augmentation in large-scale optical transport networks," in *Proc. Int. Conf. Comput., Netw. Commun. (ICNC)*, Feb. 2020, pp. 294–298, doi: [10.1109/ICNC47757.2020.9049750](https://doi.org/10.1109/ICNC47757.2020.9049750).
- [18] P. Castoldi, F. Cugini, M. Gharbaoui, A. Giorgetti, F. Paolucci, A. L. Ruscelli, N. Sambo, A. Sgambelluri, and L. Valcarenghi, "Shaping the future of optical networks by integrating SDN, telemetry, and AI [Invited]," *J. Opt. Commun. Netw.*, vol. 17, no. 7, pp. C51–C61, Jul. 2025, doi: [10.1364/jocn.553843](https://doi.org/10.1364/jocn.553843). [Online]. Available: <https://opg.optica.org/jocn/abstract.cfm?URI=jocn-17-7-C51>
- [19] M. Björklund, (Oct. 2010). *YANG - a Data Modeling Language for the Network Configuration Protocol (NETCONF)*. [Online]. Available: <https://tools.ietf.org/html/rfc6020>
- [20] Ó. G. de Dios, P. A. Robles, L. Roelens, A. Muñoz-Da-Costa, I. de Miguel, R. J. D. Barroso, and J. P. Fernández-Palacios, "Automation of multi-layer multi-domain transport networks and the role of AI [Invited]," *J. Opt. Commun. Netw.*, vol. 17, no. 2, pp. A124–A133, Feb. 2025, doi: [10.1364/jocn.537463](https://doi.org/10.1364/jocn.537463). [Online]. Available: <https://opg.optica.org/jocn/abstract.cfm?URI=jocn-17-2-A124>
- [21] *GRPC: A High-Performance, Open Source Universal RPC Framework*. Accessed: Jul. 7, 2025. [Online]. Available: <https://grpc.io/>
- [22] F. Paolucci, A. Sgambelluri, F. Cugini, and P. Castoldi, "Network telemetry streaming services in SDN-based disaggregated optical networks," *J. Lightw. Technol.*, vol. 36, no. 15, pp. 3142–3149, Aug. 2018, doi: [10.1109/JLT.2018.2795345](https://doi.org/10.1109/JLT.2018.2795345).
- [23] E. B. Sanjuan, I. A. Cardiel, J. A. Cerrada, and C. Cerrada, "Message queuing telemetry transport (MQTT) security: A cryptographic smart card approach," *IEEE Access*, vol. 8, pp. 115051–115062, 2020, doi: [10.1109/ACCESS.2020.3003998](https://doi.org/10.1109/ACCESS.2020.3003998).
- [24] C. D'Ortona, D. Tarchi, and C. Raffaelli, "Open-source MQTT-based end-to-end IoT system for smart city scenarios," *Future Internet*, vol. 14, no. 2, p. 57, Feb. 2022.
- [25] (2024). *Data Ingestion: 6 Ways To Speed Up Your Application*. Accessed: May 23, 2025. [Online]. Available: <https://redis.io/blog/data-ingestion-speed-up-your-application/>
- [26] A. Sgambelluri, F. Paolucci, A. Giorgetti, D. Scano, and F. Cugini, "Exploiting telemetry in multi-layer networks," in *Proc. 22nd Int. Conf. Transparent Opt. Netw. (ICTON)*, Jul. 2020, pp. 1–4, doi: [10.1109/ICTON51198.2020.9203310](https://doi.org/10.1109/ICTON51198.2020.9203310).
- [27] F. E. Subhan, A. Yaqoob, C. H. Muntean, and G.-M. Muntean, "A survey on artificial intelligence techniques for improved rich media content delivery in a 5G and beyond network slicing context," *IEEE Commun. Surveys Tuts.*, vol. 27, no. 2, pp. 1427–1487, Apr. 2025, doi: [10.1109/COMST.2024.3442149](https://doi.org/10.1109/COMST.2024.3442149).
- [28] L. Wu, W. A. Hanafy, A. Souza, T. Abdelzaher, G. Verma, and P. Shenoy, "Enhancing resilience in distributed ML inference pipelines for edge computing," in *Proc. IEEE Mil. Commun. Conf. (MILCOM)*, Oct. 2024, pp. 1–6. [Online]. Available: <https://lass.cs.umass.edu/papers/pdf/milcom24resilience.pdf>
- [29] W. Takase, T. Nakamura, Y. Watase, and T. Sasaki, "A solution for secure use of kibana and elasticsearch in multi-user environment," 2017, *arXiv:1706.10040*.
- [30] (Mar. 2024). *Message Queuing Telemetry Transport (MQTT)*. [Online]. Available: <https://www.symstic.com/en-us/what-is/mqtt>
- [31] S. Kirešová, M. Guzan, B. Sobota, V. Fedák, R. Bača, and D. Bakšič, "The use of time series database in measurements," in *Proc. Int. Conf. Electr. Drives Power Electron. (EDPE)*, Sep. 2023, pp. 1–8, doi: [10.1109/EDPE58625.2023.10274054](https://doi.org/10.1109/EDPE58625.2023.10274054).
- [32] (Aug. 2020). *Technical Specification TR-547—TAPI V2.1.3 Reference Implementation Agreement*. [Online]. Available: <https://opennetworking.org/wp-content/uploads/2020/08/TR-547-TAPI-v2.1.3-Reference-Implementation-Agreement-1.pdf>
- [33] (Dec. 2022). *Technical Specification TR-548—TAPI Streaming Reference Implementation Agreement V2.0*. [Online]. Available: <https://opennetworking.org/wp-content/uploads/2022/12/TR-548-TAPI-ReferenceImplementationAgreement-Streaming2.0.pdf>
- [34] P. Lechowicz, C. Natalino, V. Karunakaran, A. Autenrieth, T. Bauschert, and P. Monti, "Trade-offs in implementing unsupervised anomaly detection with TAPI-based streaming telemetry," in *Proc. IEEE 25th Int. Conf. High Perform. Switching Routing (HPSR)*, Jul. 2024, pp. 13–18, doi: [10.1109/HPSR62440.2024.10635922](https://doi.org/10.1109/HPSR62440.2024.10635922).
- [35] S. Xiong, Y. Ihlamur, F. Alican, and A. Ontoyin Yin, "GPTree: Towards explainable decision-making via LLM-powered decision trees," 2024, *arXiv:2411.08257*.
- [36] J. Pan, C. Wang, K. Zheng, Y. Li, Z. Wang, and B. Feng, "SmoothQuant+: Accurate and efficient 4-bit post-training WeightQuantization for LLM," 2023, *arXiv:2312.03788*.
- [37] (May 23, 2024). *Comparing NVIDIA H100 Vs A100 GPUs for AI Workloads*. [Online]. Available: <https://openmetal.io/resources/blog/nvidia-h100-vs-a100-gpu-comparison/>
- [38] C. Zhou, S. Zhang, Y. Zhou, Q. Qiao, J. Gao, S. Weng, W. Zhang, and C. Jin, "Balancing fidelity and plasticity: Aligning mixed-precision fine-tuning with linguistic hierarchies," 2025, *arXiv:2505.03802*.
- [39] R. Li, D. Fu, C. Shi, Z. Huang, and G. Lu, "Efficient LLMs training and inference: An introduction," *IEEE Access*, vol. 13, pp. 32944–32970, 2025, doi: [10.1109/ACCESS.2024.3501358](https://doi.org/10.1109/ACCESS.2024.3501358).

- [40] M. Yan, S. Agarwal, and S. Venkataraman, "Decoding speculative decoding," 2024, *arXiv:2402.01528*.
- [41] (Aug. 15, 2023). *How Continuous Batching Enables 23x Throughput in LLM Inference While Reducing P50 Latency*. [Online]. Available: <https://bestofai.com/article/how-continuous-batching-enables-23x-throughput-in-llm-inference-while-reducing-p50-latency-anyscale>
- [42] K. Kobayashi and S. B. Alam, "Explainable, interpretable, and trustworthy AI for an intelligent digital twin: A case study on remaining useful life," *Eng. Appl. Artif. Intell.*, vol. 129, Mar. 2024, Art. no. 107620, doi: [10.1016/j.engappai.2023.107620](https://doi.org/10.1016/j.engappai.2023.107620).
- [43] K. Waehner. (May 23, 2021). *Low Latency Data Streaming With Apache Kafka and Cloud-Native Telco Infrastructure*. [Online]. Available: <https://www.kai-waehner.de/blog/2021/05/23/apache-kafka-cloud-native-telco-infrastructure-low-latency-data-streaming-5g-aws-wavelength/>
- [44] S. Jackson, N. Cummings, and S. Khan, "Streaming technologies and serialization protocols: Empirical performance analysis," *IEEE Access*, vol. 12, pp. 158025–158039, 2024, doi: [10.1109/ACCESS.2024.3486054](https://doi.org/10.1109/ACCESS.2024.3486054).
- [45] T. P. Raptis and A. Passarella, "A survey on networked data streaming with apache kafka," *IEEE Access*, vol. 11, pp. 85333–85350, 2023, doi: [10.1109/ACCESS.2023.3303810](https://doi.org/10.1109/ACCESS.2023.3303810).
- [46] S. Sheng, Q. Huang, and P. P. C. Lee, "DeltaINT: Toward general in-band network telemetry with extremely low bandwidth overhead," in *Proc. IEEE 29th Int. Conf. Neww. Protocols (ICNP)*, Nov. 2021, pp. 1–11, doi: [10.1109/ICNP52444.2021.9651963](https://doi.org/10.1109/ICNP52444.2021.9651963).
- [47] T. Miller. (2022). *Data ingestion: 6 ways to speed up your application*. [Online]. Available: <https://redis.io/blog/data-ingestion-speed-up-your-application/>
- [48] *GRPC Vs REST API*. [Online]. Available: <https://www.toptal.com/devlopers/grpc-vs-rest-api>
- [49] (Mar. 25, 2025). *GRPC Vs. REST: Key Similarities and Differences*. [Online]. Available: <https://blog.dreamfactory.com/grpc-vs-rest-how-does-grpc-compare-with-traditional-rest-apis>
- [50] N. McDonald, "Modular, hierarchical machine learning for sequential goal completion," 2024, *arXiv:2404.19060*.
- [51] B. Gamsa, O. Krieger, and M. Stumm, "Optimizing IPC performance for shared-memory multiprocessors," in *Proc. Int. Conf. Parallel Process. (ICPP)*, Aug. 1994, pp. 208–211.
- [52] S. Shen, H. Li, A. Tyrovolas, Y. Teng, R. Nejabati, S. Yan, and D. Simeonidou, "End-to-end QoT predictions enhanced by GNPpy-based digital twin with network telemetry," in *Proc. Opt. Fiber Commun. Conf. (OFC)*, 2024, pp. 1–3.
- [53] R. Scolati, "Measuring the responsiveness of WebAssembly in edge network applications," Master's thesis, KTH Royal Inst. Technol., Stockholm, Sweden, 2023. [Online]. Available: <https://www.diva-portal.org/smash/get/diva2:1783251/FULLTEXT01.pdf>
- [54] *How Caching Helps Improve the LLM Inference?*. Accessed: Nov. 28, 2025. [Online]. Available: <https://adasci.org/how-caching-helps-improve-the-llm-inference/>
- [55] *How Continuous Batching Enables 23x Throughput in LLM Inference While Reducing P50 Latency - Anyscale*. Accessed: Nov. 28, 2025. [Online]. Available: <https://bestofai.com/article/how-continuous-batching-enables-23x-throughput-in-llm-inference-while-reducing-p50-latency-anyscale>
- [56] *Boosting Speed of Scikit-Learn Regression Algorithms*. Accessed: Nov. 28, 2025. [Online]. Available: <https://www.timetoact-group.at/en/techblog/techblog/boosting-speed-of-scikit-learn-regression-algorithms>
- [57] (2025). *Comparing NVIDIA H100 Vs A100 GPUs for AI workloads*. [Online]. Available: <https://openmetal.io/resources/blog/nvidia-h100-vs-a100-gpu-comparison/>
- [58] *Digital Twin Implementation With Commercial Equipment in Optical Networks*. Accessed: Nov. 28, 2025. [Online]. Available: <https://www.allegro-he.eu/2025/03/03/digital-twin-implementation-with-commercial-equipment-in-optical-networks/>
- [59] R. G. Goriparthi, "Hybrid AI frameworks for edge computing: Balancing efficiency and scalability," *J. Adv. Eng. Trends Innov.*, vol. 2, no. 1, pp. 110–125, 2024. [Online]. Available: <https://ijaeti.com/index.php/Journal/article/download/689/718/1274>
- [60] S. Mihai, M. Yaqoob, D. V. Hung, W. Davis, P. Towakel, M. Raza, M. Karamanoglu, B. Barn, D. Shetve, R. V. Prasad, H. Venkataraman, R. Trestian, and H. X. Nguyen, "Digital twins: A survey on enabling technologies, challenges, trends and future prospects," *IEEE Commun. Surv. Tut.*, vol. 24, no. 4, pp. 2255–2291, 2022, doi: [10.1109/COMST.2022.3208773](https://doi.org/10.1109/COMST.2022.3208773).
- [61] D. Wang, Y. Song, Y. Zhang, X. Jiang, J. Dong, F. N. Khan, T. Sasai, S. Huang, A. P. T. Lau, M. Tornatore, and M. Zhang, "Digital twin of optical networks: A review of recent advances and future trends," *J. Lightw. Technol.*, vol. 42, no. 12, pp. 4233–4259, Jun. 2024, doi: [10.1109/jlt.2024.3401419](https://doi.org/10.1109/jlt.2024.3401419).
- [62] C. Janz, Y. You, M. Hemmati, Z. Jiang, A. Javadtalab, and J. Mitra, "Digital twin for the optical network: Key technologies and enabled automation applications," in *Proc. IEEE/IFIP Netw. Operations Manage. Symp.*, Apr. 2022, pp. 1–6, doi: [10.1109/NOMS54207.2022.9789844](https://doi.org/10.1109/NOMS54207.2022.9789844).
- [63] D. Wang, Z. Zhang, M. Zhang, M. Fu, J. Li, S. Cai, C. Zhang, and X. Chen, "The role of digital twin in optical communication: Fault management, hardware configuration, and transmission simulation," *IEEE Commun. Mag.*, vol. 59, no. 1, pp. 133–139, Jan. 2021, doi: [10.1109/MCOM.001.2000727](https://doi.org/10.1109/MCOM.001.2000727).
- [64] Q. Zhuge, X. Liu, Y. Zhang, M. Cai, Y. Liu, Q. Qiu, X. Zhong, J. Wu, R. Gao, L. Yi, and W. Hu, "Building a digital twin for intelligent optical networks [invited tutorial]," *J. Opt. Commun. Netw.*, vol. 15, no. 8, pp. 242–262, 2023, doi: [10.1364/jocn.483600](https://doi.org/10.1364/jocn.483600).
- [65] H. Xu, F. Omitaomu, S. Sabri, S. Zlatanova, X. Li, and Y. Song, "Leveraging generative AI for urban digital twins: A scoping review on the autonomous generation of urban data, scenarios, designs, and 3D city models for smart city advancement," 2024, *arXiv:2405.19464*.
- [66] K. Muhammad, T. David, G. Nassissid, and T. Farus, "Integrating generative AI with network digital twins for enhanced network operations," 2024, *arXiv:2406.17112*.
- [67] Y. Song, M. Zhang, Y. Zhang, Y. Shi, S. Shen, B. Guo, S. Huang, and D. Wang, "Implementing digital twin in field-deployed optical networks: Uncertain factors, operational guidance, and field-trial demonstration," *IEEE Netw.*, vol. 38, no. 1, pp. 38–45, Jan. 2024, doi: [10.1109/MNET.2023.3332893](https://doi.org/10.1109/MNET.2023.3332893).
- [68] S. Cruzes. (Jan. 2024). *Disaggregated Optical Networks: A Survey*. [Online]. Available: <https://www.techrxiv.org/users/693957/articles/705861-disaggregated-optical-networks-a-survey>
- [69] G. Borracini, S. Straullu, A. Giorgetti, R. Ambrosone, E. Virgillito, A. D'Amico, R. D'Ingillo, F. Aquilino, A. Nespola, N. Sambo, F. Cugini, and V. Curri, "Experimental demonstration of partially disaggregated optical network control using the physical layer digital twin," *IEEE Trans. Netw. Service Manage.*, vol. 20, no. 3, pp. 2343–2355, Sep. 2023, doi: [10.1109/TNSM.2023.3288823](https://doi.org/10.1109/TNSM.2023.3288823).
- [70] D. Wang, Y. Wang, X. Jiang, Y. Zhang, Y. Pang, and M. Zhang, "When large language models meet optical networks: Paving the way for automation," *Electronics*, vol. 13, no. 13, p. 2529, Jun. 2024, doi: [10.3390/electronics13132529](https://doi.org/10.3390/electronics13132529).
- [71] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," 2017, *arXiv:1706.03762*.
- [72] S. Latif, A. Zaidi, H. Cuayahuitl, F. Shamshad, M. Shoukat, M. Usama, and J. Qadir, "Transformers in speech processing: A survey," 2023, *arXiv:2303.11607*.
- [73] H. Naveed, A. U. Khan, S. Qiu, M. Saqib, S. Anwar, M. Usman, N. Akhtar, N. Barnes, and A. Mian, "A comprehensive overview of large language models," 2023, *arXiv:2307.06435*.
- [74] G. Marvin, N. Hellen, D. Jjingo, and J. Nakatumba-Nabende, "Prompt engineering in large language models," in *Data Intelligence and Cognitive Informatics*, I. J. Jacob, S. Piramuthu, and P. Falkowski-Gilski, eds., Singapore, Springer, 2024, pp. 387–402.
- [75] Y. Tang, R. Puduppully, Z. Liu, and N. Chen, "In-context learning of large language models for controlled dialogue summarization: A holistic benchmark and empirical analysis," in *Proc. 4th New Frontiers Summarization Workshop*, Singapore, Dec. 2023, pp. 56–67, doi: [10.18653/v1/2023.newssum-1.6](https://doi.org/10.18653/v1/2023.newssum-1.6).
- [76] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, "Retrieval-augmented generation for large language models: A survey," 2023, *arXiv:2312.10997*.

- [77] H. Zheng, L. Shen, A. Tang, Y. Luo, H. Hu, B. Du, Y. Wen, and D. Tao, "Learning from models beyond fine-tuning," 2023, *arXiv:2310.08184*.
- [78] S. Cruzes, "Failure management overview in optical networks," *IEEE Access*, vol. 12, pp. 169170–169193, 2024, doi: [10.1109/ACCESS.2024.3498704](https://doi.org/10.1109/ACCESS.2024.3498704).
- [79] J. Su, C. Jiang, X. Jin, Y. Qiao, T. Xiao, H. Ma, R. Wei, Z. Jing, J. Xu, and J. Lin, "Large language models for forecasting and anomaly detection: A systematic literature review," 2024, *arXiv:2402.10350*.
- [80] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, "Chain-of-Thought prompting elicits reasoning in large language models," 2022, *arXiv:2201.11903*.
- [81] S. Yao, D. Yu, J. Zhao, I. Shafraan, T. L. Griffiths, Y. Cao, and K. Narasimhan, "Tree of thoughts: Deliberate problem solving with large language models," in *Proc. 37th Conf. Neural Inf. Process. Syst.*, 2023. [Online]. Available: <https://openreview.net/forum?id=5Xc1ecxOIh>
- [82] S. Sakthivel, M. R. Belgaum, A. A. B. Sajaj, M. M. Alam, and M. M. Su'ud, "Machine learning-driven QoT prediction for enhanced optical networks in DWDM system," *IEEE Access*, vol. 13, pp. 80445–80460, 2025, doi: [10.1109/ACCESS.2025.3565502](https://doi.org/10.1109/ACCESS.2025.3565502).
- [83] W. Xin Zhao et al., "A survey of large language models," 2023, *arXiv:2303.18223*.
- [84] H. Dong and S. Xie, "Large language models (LLMs): Deployment, tokenomics and sustainability," 2024, *arXiv:2405.17147*.
- [85] J. Zhang, H. Bu, H. Wen, Y. Liu, H. Fei, R. Xi, L. Li, Y. Yang, H. Zhu, and D. Meng, "When LLMs meet cybersecurity: A systematic literature review," 2024, *arXiv:2405.03644*.
- [86] F. Liang, Z. Zhang, H. Lu, V. C. M. Leung, Y. Guo, and X. Hu, "Communication-efficient large-scale distributed deep learning: A comprehensive survey," 2024, *arXiv:2404.06114*.
- [87] Y. Wang, J. Li, Y. Pang, Y. Song, L. Zhang, M. Zhang, and D. Wang, "AlarmGPT: An intelligent operation assistant for optical network alarm analysis using ChatGPT," in *Proc. 49th Eur. Conf. Opt. Commun. (ECOC)*, vol. 2023, 2023, pp. 1330–1333, doi: [10.1049/ICP.2023.2538](https://doi.org/10.1049/ICP.2023.2538).
- [88] Y. Huang, M. Xu, X. Zhang, D. Niyato, Z. Xiong, S. Wang, and T. Huang, "AI-generated network design: A diffusion model-based learning approach," *IEEE Netw.*, vol. 38, no. 3, pp. 202–209, May 2024, doi: [10.1109/MNET.2023.3321538](https://doi.org/10.1109/MNET.2023.3321538).
- [89] J. Wang and Z. Duan, "Research on the application of spark streaming real-time data analysis system and large language model intelligent agents," 2024, *arXiv:2501.14734*.
- [90] A. Dorri, S. S. Kanhere, and R. Jurdak, "Multi-agent systems: A survey," *IEEE Access*, vol. 6, pp. 28573–28593, 2018, doi: [10.1109/ACCESS.2018.2831228](https://doi.org/10.1109/ACCESS.2018.2831228).
- [91] T. Zeeshan, A. Kumar, S. Pirttikangas, and S. Tarkoma, "Large language model based multi-agent system augmented complex event processing pipeline for Internet of Multimedia Things," 2025, *arXiv:2501.00906*.
- [92] H. Cao, X. Yang, and R. Deng, "Ontology-based holonic event-driven architecture for autonomous networked manufacturing systems," *IEEE Trans. Autom. Sci. Eng.*, vol. 18, no. 1, pp. 205–215, Jan. 2021, doi: [10.1109/TASE.2020.3025784](https://doi.org/10.1109/TASE.2020.3025784).
- [93] P. Hu and X. Ying, "Unified mind model: Reimagining autonomous agents in the LLM era," 2025, *arXiv:2503.03459*.
- [94] (2025). *A Guide to Event-Driven Design for Agents Multi-Agent Systems* [Online]. Available: <https://www.confluent.io/resources/ebook/guide-to-event-driven-agents/>
- [95] K. M. Chandy, "Event driven architecture," in *Encyclopedia of Database Systems*. New York, NY, USA: Springer, 2009, pp. 1040–1044.
- [96] C. Xu, Z. Tang, H. Yu, P. Zeng, and L. Kong, "Digital twin-driven collaborative scheduling for heterogeneous task and edge-end resource via multi-agent deep reinforcement learning," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 10, pp. 3056–3069, Oct. 2023, doi: [10.1109/JSAC.2023.3310066](https://doi.org/10.1109/JSAC.2023.3310066).
- [97] D. Vijay and V. Ethiraj, "Graph-symbolic policy enforcement and control (G-SPEC): A neuro-symbolic framework for safe agentic AI in 5G autonomous networks," 2025, *arXiv:2512.20275*.
- [98] Z. Xu, S. Jain, and M. Kankanhalli, "Hallucination is inevitable: An innate limitation of large language models," 2024, *arXiv:2401.11817*.
- [99] (2025). *Four Design Patterns for Event-Driven, Multi-Agent Systems*. [Online]. Available: <https://www.confluent.io/blog/event-driven-multi-agent-systems/>
- [100] Z. Asgar, M. Nguyen, and S. Katti, "Efficient and scalable agentic AI with heterogeneous systems," 2025, *arXiv:2507.19635*.
- [101] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-rank adaptation of large language models," 2021, *arXiv:2106.09685*.
- [102] H. Dizdarevic, S. Dizdarevic, M. Skrbic, and N. Hadžiahmetovic, "A survey on physical layer impairments aware routing and wavelength assignment algorithms in transparent wavelength routed optical networks," in *Proc. 39th Int. Conv. Inf. Commun. Technol., Electron. Microelectron. (MIPRO)*, May 2016, pp. 530–536, doi: [10.1109/MIPRO.2016.7522201](https://doi.org/10.1109/MIPRO.2016.7522201).
- [103] E. Rivera Hartling, A. Pilipetskii, D. Evans, E. Mateo, M. Salsi, P. Pecci, and P. Mehta, "Design, acceptance and capacity of subsea open cables," *J. Lightw. Technol.*, vol. 39, no. 3, pp. 742–756, Feb. 2021, doi: [10.1109/JLT.2020.3045389](https://doi.org/10.1109/JLT.2020.3045389).
- [104] (2025). *What is Q-Factor and What is Its Importance?*. Accessed: Aug. 13, 2025. [Online]. Available: <https://mapyourtech.com/what-is-q-factor-and-what-is-its-importance/>
- [105] R. Ayassi, A. Triki, N. Crespi, R. Minerva, and M. Laye, "Survey on the use of machine learning for quality of transmission estimation in optical transport networks," *J. Lightw. Technol.*, vol. 40, no. 17, pp. 5803–5815, Sep. 2022, doi: [10.1109/JLT.2022.3184178](https://doi.org/10.1109/JLT.2022.3184178).
- [106] P. Poggiolini, "The GN model of non-linear propagation in uncompensated coherent optical systems," *J. Lightw. Technol.*, vol. 30, no. 24, pp. 3857–3879, Dec. 2012, doi: [10.1109/JLT.2012.2217729](https://doi.org/10.1109/JLT.2012.2217729).
- [107] A. Carena, G. Bosco, V. Curri, Y. Jiang, P. Poggiolini, and F. Forghieri, "On the accuracy of the GN-model and on analytical correction terms to improve it," 2014, *arXiv:1401.6946*.
- [108] F. Musumeci, C. Rottondi, A. Nag, I. Macaluso, D. Zibar, M. Ruffini, and M. Tornatore, "An overview on application of machine learning techniques in optical networks," *IEEE Commun. Surveys Tuts.*, vol. 21, no. 2, pp. 1383–1408, 2nd Quart., 2019, doi: [10.1109/COMST.2018.2880039](https://doi.org/10.1109/COMST.2018.2880039).
- [109] J. Lu, G. Zhou, Q. Fan, D. Zeng, C. Guo, L. Lu, J. Li, C. Xie, C. Lu, F. N. Khan, and A. P. T. Lau, "Performance comparisons between machine learning and analytical models for quality of transmission estimation in wavelength-division-multiplexed systems [invited]," *J. Opt. Commun. Netw.*, vol. 13, no. 4, pp. B35–B44, Apr. 2021, doi: [10.1364/JOCN.410876](https://doi.org/10.1364/JOCN.410876).
- [110] D. Wang, X. Jiang, Y. Song, M. Fu, Z. Zhang, X. Chen, and M. Zhang, "Applications of physics-informed neural network for optical fiber communications," *IEEE Commun. Mag.*, vol. 60, no. 9, pp. 32–37, Sep. 2022, doi: [10.1109/MCOM.001.2100961](https://doi.org/10.1109/MCOM.001.2100961).
- [111] D. Wang, D. Wang, C. Zhang, L. Wang, S. Liu, and M. Zhang, "Machine learning for optical layer failure management," in *Proc. Opto-Electron. Commun. Conf. (OECC)*, Jul. 2021, pp. 1–3.
- [112] H. Lun, Q. Zhuge, M. Fu, Y. Wu, Q. Liu, M. Cai, X. Zeng, and W. Hu, "Soft failure identification in optical networks based on convolutional neural network," in *Proc. 45th Eur. Conf. Opt. Commun. (ECOC)*, Sep. 2019, pp. 1–3, doi: [10.1049/CP.2019.1138](https://doi.org/10.1049/CP.2019.1138).
- [113] R. Wang, J. Zhang, Z. Gu, M. Ibrahimi, B. Zhang, F. Musumeci, M. Tornatore, and Y. Ji, "Digital-twin-assisted meta learning for soft-failure localization in ROADM-based optical networks," *J. Opt. Commun. Netw.*, vol. 16, no. 7, pp. C11–C19, 2024, doi: [10.1364/jocn.517551](https://doi.org/10.1364/jocn.517551).
- [114] S. Aladin, S. Allogba, A. V. S. Tran, and C. Tremblay, "Recurrent neural networks for short-term forecast of lightpath performance," in *Proc. Opt. Fiber Commun. Conf. Exhib. (OFC)*, Mar. 2020, pp. 1–3.
- [115] F. Musumeci and M. Tornatore, "Failure management in optical networks with ML: A tutorial on applications, challenges, and pitfalls [invited]," *J. Opt. Commun. Netw.*, vol. 17, no. 8, pp. C144–C155, 2025, doi: [10.1364/jocn.551910](https://doi.org/10.1364/jocn.551910).
- [116] S. Allogba, B. L. M. Yaméogo, and C. Tremblay, "Extraction and early detection of anomalies in lightpath SNR using machine learning models," *J. Lightw. Technol.*, vol. 40, no. 7, pp. 1864–1872, Apr. 2022, doi: [10.1109/JLT.2021.3134098](https://doi.org/10.1109/JLT.2021.3134098).
- [117] L. E. Kruse, S. Köhl, A. Dochhan, and S. Pachnicke, "Experimental investigation of machine-learning-based soft-failure management using the optical spectrum," *J. Opt. Commun. Netw.*, vol. 16, no. 2, p. 94, Jan. 2024, doi: [10.1364/jocn.500930](https://doi.org/10.1364/jocn.500930).
- [118] M. Devigili, M. Ruiz, N. Costa, C. Castro, A. Napoli, J. Pedro, and L. Velasco, "Applications of the OCATA time domain digital twin: From QoT estimation to failure management," *J. Opt. Commun. Netw.*, vol. 16, no. 2, pp. 221–232, Feb. 2024, doi: [10.1364/jocn.507128](https://doi.org/10.1364/jocn.507128).

- [119] A. Pacini, F. Boitier, A. May, V. Bajaj, A. Sgambelluri, A. Giorgetti, L. Valcarengi, and P. Layec, "Longitudinal power profile monitoring telemetry enabling self-healing optical networks," *J. Opt. Commun. Netw.*, vol. 17, no. 7, pp. 580–589, Jul. 2025, doi: [10.1364/jocn.555840](#).
- [120] S. Behera, T. Panayiotou, and G. Ellinas, "Modeling soft-failure evolution for triggering timely repair with low QoT margins," in *Proc. IEEE Global Commun. Conf.*, Dec. 2022, pp. 2140–2145, doi: [10.1109/GLOBECOM48099.2022.10000690](#).
- [121] Z. Wang, M. Zhang, D. Wang, C. Song, M. Liu, J. Li, L. Lou, and Z. Liu, "Failure prediction using machine learning and time series in optical network," *Opt. Exp.*, vol. 25, no. 16, pp. 18553–18565, Aug. 2017, doi: [10.1364/oe.25.018553](#).
- [122] Q. Pham-Van, V. López, A. M. Lopez-de-Lerma, R. Szwedowski, K. Mrówka, S. Auer, H.-T. Thieu, Q.-H. Tran, D. Verchere, G. Atkinson, A. Autenrieth, S. Neidlinger, and L. Tancevski, "Demonstration of alarm correlation in partially disaggregated optical networks," in *Proc. Opt. Fiber Commun. Conf. Exhib. (OFC)*, Mar. 2020, pp. 1–3.
- [123] B. Shariati, M. Ruiz, J. Comellas, and L. Velasco, "Learning from the optical spectrum: Failure detection and identification," *J. Lightw. Technol.*, vol. 37, no. 2, pp. 433–440, Jan. 2019, doi: [10.1109/JLT.2018.2859199](#).
- [124] A. P. Vela, M. Ruiz, F. Fresi, N. Sambo, F. Cugini, G. Meloni, L. Poti, L. Velasco, and P. Castoldi, "BER degradation detection and failure identification in elastic optical networks," *J. Lightw. Technol.*, vol. 35, no. 21, pp. 4595–4604, Nov. 2017, doi: [10.1109/JLT.2017.2747223](#).
- [125] B. C. Chatterjee, N. Sarma, and E. Oki, "Routing and spectrum allocation in elastic optical networks: A tutorial," *IEEE Commun. Surv. Tut.*, vol. 17, no. 3, pp. 1776–1800, 3rd Quart., 2015, doi: [10.1109/COMST.2015.2431731](#).
- [126] M. Doherty, R. Matzner, R. Sadeghi, P. Bayvel, and A. Beghelli, "Reinforcement learning for dynamic resource allocation in optical networks: Hype or hope?" 2025, *arXiv:2502.12804*.
- [127] M. A. Amirabadi, S. A. Nezamalhosseini, M. H. Kahaei, and L. R. Chen, "A comprehensive survey on machine and deep learning for optical communications," *IEEE Access*, vol. 13, pp. 88794–88846, 2025, doi: [10.1109/ACCESS.2025.3569559](#).
- [128] Y. Zhang, J. Xin, X. Li, and S. Huang, "Overview on routing and resource allocation based machine learning in optical networks," *Opt. Fiber Technol.*, vol. 60, Dec. 2020, Art. no. 102355, doi: [10.1016/j.yofte.2020.102355](#).
- [129] S. Cruzes, "Enhancing optical network efficiency: A survey on machine learning-driven quality of transmission estimation," 2024, doi: [10.36227/techrxiv.171340665.50082312/v2](#).
- [130] (May 2019). *Autonomous Networks: Empowering Digital Transformation for the Telecoms Industry*. [Online]. Available: <https://www.tmforum.org/wp-content/uploads/2019/05/22553-Autonomous-Networks-whitepaper.pdf>
- [131] *AI in the Evolution of Autonomous Networks*. [Online]. Available: <https://www.etsi.org/images/files/ETSIWhitePapers/ETSI-WP-69-AI-inthe>
- [132] Y. Lee, G. Bernstein, D. Li, and W. Imajuku. (Feb. 2015). *Routing and Wavelength Assignment Information Model for Wavelength Switched Optical Networks*. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc7446>
- [133] M. Dallaglio, N. Sambo, F. Cugini, and P. Castoldi, "YANG models for vendor-neutral optical networks, reconfigurable through state machine," *IEEE Commun. Mag.*, vol. 55, no. 8, pp. 170–178, Aug. 2017, doi: [10.1109/MCOM.2017.1600733](#).
- [134] T. Faisal, J. A. O. Lucena, D. R. Lopez, C. Wang, and M. Dohler, "How to design autonomous service level agreements for 6G," *IEEE Commun. Mag.*, vol. 61, no. 3, pp. 80–85, Mar. 2023, doi: [10.1109/MCOM.001.2200131](#).
- [135] S. S. Mwanje, A. Banerjee, J. Goerge, A. Abdelkader, G. Hannak, P. Szilagyi, T. Subramanya, J. Gosser, and T. Foth, "Intent-driven network and service management: Definitions, modeling and implementation," *ITU J. Future Evolving Technol.*, vol. 3, no. 3, pp. 555–569, 2022, doi: [10.52953/aoia2456](#).
- [136] J. Ma, Y. Guo, C. Fang, and Q. Zhang, "Digital twin-based zero-touch management for IoT," *Electronics*, vol. 11, no. 24, p. 4104, Dec. 2022, doi: [10.3390/electronics11244104](#).
- [137] Y. Huang, H. Du, X. Zhang, D. Niyato, J. Kang, Z. Xiong, S. Wang, and T. Huang, "Large language models for networking: Applications, enabling techniques, and challenges," *IEEE Netw.*, vol. 39, no. 1, pp. 235–242, Jan. 2025, doi: [10.1109/MNET.2024.3435752](#).
- [138] X. Jiang, M. Zhang, Y. Song, Y. Zhang, Y. Wang, C. Ju, and D. Wang, "OptiComm-GPT: A GPT-based versatile research assistant for optical fiber communication systems," *Opt. Exp.*, vol. 32, no. 12, pp. 20776–20796, Jun. 2024, doi: [10.1364/oe.522026](#).
- [139] C. Sun, X. Yang, N. Di Cicco, R. Ayassi, V. Virajit Garbhapu, P. A. Stavrou, M. Tornatore, G. Charlet, and Y. Pointurier, "Experimental demonstration of local AI-agents for lifecycle management and control automation of optical networks," *J. Opt. Commun. Netw.*, vol. 17, no. 8, pp. C82–C92, 2025, doi: [10.1364/jocn.550286](#).
- [140] Y. Pang, M. Zhang, Y. Liu, X. Li, Y. Wang, Y. Huan, Z. Liu, J. Li, and D. Wang, "Large language model-based optical network log analysis using LLaMA2 with instruction tuning," *J. Opt. Commun. Netw.*, vol. 16, no. 11, pp. 1116–1132, 2024, doi: [10.1364/jocn.527874](#).
- [141] A. Zhou, Y. Song, Y. Zhang, M. Zhang, and D. Wang, "Large language model-driven AI agent in SDN controller towards intent-based management of optical networks," in *Proc. ECOC 50th Eur. Conf. Opt. Commun.*, 2024, pp. 1595–1598.
- [142] D. Adanza, C. Natalino, L. Gifre, R. Muñoz, P. Alemany, P. Monti, and R. Vilalta, "IntentLLM: An AI chatbot to create, find, and explain slice intents in TeraFlowSDN," in *Proc. IEEE 10th Int. Conf. Netw. Softwarization (NetSoft)*, Jun. 2024, pp. 307–309, doi: [10.1109/NET-SOFT60951.2024.10588917](#).
- [143] R. Vilalta, L. Gifre, R. Casellas, R. Muñoz, R. Martínez, A. Mozo, A. Pastor, D. López, and J. P. Fernández-Palacios, "Applying digital twins to optical networks with cloud-native SDN controllers," *IEEE Commun. Mag.*, vol. 61, no. 12, pp. 128–134, Dec. 2023, doi: [10.1109/MCOM.003.2300105](#).
- [144] V. Karunakaran, C. Natalino, B. Shariati, P. Lechowicz, J. K. Fischer, A. Autenrieth, P. Monti, and T. Bauschert, "TAPI-based telemetry streaming in multi-domain optical transport network," in *Proc. Opt. Fiber Commun. Conf. (OFC)*, 2024, p. M3Z.9.
- [145] E. Etezadi, C. Natalino, V. Karunakaran, R. Diaz, A. Lindgren, S. Melin, A. Autenrieth, L. Wosinska, P. Monti, and M. Furdek, "Demonstration of DRL-based intelligent spectrum management over a T-API-enabled optical network digital twin," in *Proc. 49th Eur. Conf. Opt. Commun. (ECOC)*, vol. 2023, 2023, pp. 1730–1733, doi: [10.1049/icp.2024.1801](#).
- [146] M. S. Faruk and S. J. Savory, "Measurement informed models and digital twins for optical fiber communication systems," *J. Lightw. Technol.*, vol. 42, no. 3, pp. 1016–1030, Feb. 2024, doi: [10.1109/jlt.2023.3328765](#).
- [147] Y. He, Z. Zhai, L. Dou, L. Wang, Y. Yan, C. Xie, C. Lu, and A. P. T. Lau, "Improved QoT estimations through refined signal power measurements and data-driven parameter optimizations in a disaggregated and partially loaded live production network," *J. Opt. Commun. Netw.*, vol. 15, no. 9, pp. 638–648, 2023, doi: [10.1364/jocn.496720](#).
- [148] N. D. Cicco, M. Ibrahim, S. Troia, F. Musumeci, and M. Tornatore, "Open implementation of a large language model pipeline for automated configuration of software-defined optical networks," in *Proc. ECOC*, 2024, pp. 1591–1594.
- [149] R. Kamoi, S. Snigdha Sarathi Das, R. Lou, J. Janice Ahn, Y. Zhao, X. Lu, N. Zhang, Y. Zhang, R. Haoran Zhang, S. Reddy Vummanthala, S. Dave, S. Qin, A. Cohan, W. Yin, and R. Zhang, "Evaluating LLMs at detecting errors in LLM responses," 2024, *arXiv:2404.03602*.
- [150] K. Hau, S. Hassan, and S. Zhou, "LLMs in mobile apps: Practices, challenges, and opportunities," 2025, *arXiv:2502.15908*.
- [151] F. Wang, Z. Zhang, X. Zhang, Z. Wu, T. Mo, Q. Lu, W. Wang, R. Li, J. Xu, X. Tang, Q. He, Y. Ma, M. Huang, and S. Wang, "A comprehensive survey of small language models in the era of large language models: Techniques, enhancements, applications, collaboration with LLMs, and trustworthiness," 2024, *arXiv:2411.03350*.
- [152] J. Freund. (Mar. 21, 2024). *Top Automation Challenges for 2024 (and How To Mitigate Them)*. [Online]. Available: <https://www.forbes.com/councils/forbestechcouncil/2024/03/21/top-automation-challenges-for-2024-and-how-to-mitigate-them/>
- [153] (Feb. 1, 2024). *Optical Network Automation Can Save Operators 81% in Network & Service Management*. [Online]. Available: <https://www.communicationstoday.co.in/optical-network-automation-can-save-operators-81-in-network-service-management/>
- [154] Y. Gu, Q. Hu, S. Yang, H. Xi, J. Chen, S. Han, and H. Cai, "Jet-nemotron: Efficient language model with post neural architecture search," 2025, *arXiv:2508.15884*.

- [155] A. Dawid and Y. LeCun, "Introduction to latent variable energy-based models: A path toward autonomous machine intelligence," *J. Stat. Mech., Theory Exp.*, vol. 2024, no. 10, Oct. 2024, Art. no. 104011, doi: [10.1088/1742-5468/ad292b](https://doi.org/10.1088/1742-5468/ad292b).
- [156] A. Hartwell, F. Montana, W. Jacobs, V. Kadirkamanathan, N. Ameri, and A. R. Mills, "Distributed digital twins for health monitoring: Resource constrained aero-engine fleet management," *Aeronaut. J.*, vol. 128, no. 1325, pp. 1556–1575, Jul. 2024, doi: [10.1017/aer.2024.23](https://doi.org/10.1017/aer.2024.23).
- [157] K. G. Barman et al., "Large physics models: Towards a collaborative approach with large language models and foundation models," 2025, *arXiv:2501.05382*.
- [158] E. Zeydan, C. D. Alwis, R. Khan, Y. Turk, A. Aydeger, T. R. Gadekallu, and M. Liyanage, "Quantum technologies for beyond 5G and 6G networks: Applications, opportunities, and challenges," *IEEE Open J. Commun. Soc.*, vol. 6, pp. 6383–6420, 2025, doi: [10.1109/OJCOMS.2025.3591842](https://doi.org/10.1109/OJCOMS.2025.3591842).
- [159] K. Shahriari and M. Shahriari, "IEEE standard review—Ethically aligned design: A vision for prioritizing human wellbeing with artificial intelligence and autonomous systems," in *Proc. IEEE Canada Int. Humanitarian Technol. Conf. (IHTC)*, Toronto, ON, Canada, Jul. 2017, pp. 197–201, doi: [10.1109/IHTC.2017.8058187](https://doi.org/10.1109/IHTC.2017.8058187).
- [160] E. Kiciman, R. Ness, A. Sharma, and C. Tan, "Causal reasoning and large language models: Opening a new frontier for causality," 2023, *arXiv:2305.00050*.
- [161] Y. Cao, H. Zhao, Y. Cheng, T. Shu, Y. Chen, G. Liu, G. Liang, J. Zhao, J. Yan, and Y. Li, "Survey on large language model-enhanced reinforcement learning: Concept, taxonomy, and methods," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 6, pp. 9737–9757, Jun. 2025, doi: [10.1109/TNNLS.2024.3497992](https://doi.org/10.1109/TNNLS.2024.3497992).
- [162] S. Khosla, Z. Zhu, and Y. He, "Survey on memory-augmented neural networks: Cognitive insights to AI applications," 2023, *arXiv:2312.06141*.
- [163] J. Feng, T. Yu, K. Zhang, and L. Cheng, "Integration of multi-agent systems and artificial intelligence in self-healing subway power supply systems: Advancements in fault diagnosis, isolation, and recovery," *Processes*, vol. 13, no. 4, p. 1144, Apr. 2025, doi: [10.3390/pr13041144](https://doi.org/10.3390/pr13041144). [Online]. Available: <https://www.mdpi.com/2227-9717/13/4/1144>
- [164] P. Linardatos, V. Papastefanopoulos, and S. Kotsiantis, "Explainable AI: A review of machine learning interpretability methods," *Entropy*, vol. 23, no. 1, p. 18, Dec. 2020.
- [165] F. Paolucci and A. Sgambelluri, "Telemetry in disaggregated optical networks," in *Proc. Int. Conf. Opt. Netw. Design Model. (ONDM)*, May 2020, pp. 1–3, doi: [10.23919/ONDM48393.2020.9132992](https://doi.org/10.23919/ONDM48393.2020.9132992).



SERGIO CRUZES (Member, IEEE) received the M.B.A. degree in project management and technological innovation from FIPE, Brazil, in 2010, and the M.Sc. degree in electrical engineering from the University of São Paulo (USP), with a focus on optical communications and developed a prototype for video transmission over fiber optics using frequency modulation. He is currently an Optical Network Engineer with Ciena, Brazil. His research interests include optical network automation, machine learning, high-spectral-efficiency coherent transmission systems, disaggregated optical networks, quality of transmission in optical networks, and failure management in optical networks.

...